

10주차 - Greedy Algorithm

Dynamic Programming에서의 최적화 문제

답을 합치면서 subproblem의 복잡한 문제를 푸는 것

1. 문제를 여러 개의 작은 부분문제(subproblem)로 쪼갬다.
2. 각 부분문제마다 가능한 "모든 선택지"를 전부 고려하여
3. 그 부분문제를 최적으로 해결할 수 있는 해답을 구하고,
>> 이들을 모아 전체 문제의 최적해(optimal solution)를 구성한다.

모든 작은 경우를 다 계산해 놓고 그중 최적이었던 옵션을 나중에 골라낸다. >> 오버헤드가 커짐

결론 : DP를 쓰는 것은 너무 과하다. 시간(자원)을 많이 사용한다.

Greedy Algorithm

어떤 특정 순간에서 최선의 선택을 하는 알고리즘

'Locally' 최선의 선택을 함. but **Global**한 최적이 되도록 "기대"하는 방식

항상 최적해를 보장하지는 않음. 하지만 많은 문제에서는 좋은 해 또는 최적해를 얻을 수 있음.

DP는 Subproblems를 전부 해결한 뒤에 선택을 하지만, **Greedy**는 **먼저 선택**을 하고 진행함.

* 문제 1 : Activity-selection 문제

- 간단하지만 직관적이지는 않은 문제. Greedy 방식으로 효율적으로 최적해를 구할 수 있는 대표 사례.

* Greedy 접근의 요소

- Greedy-choice 특성
- Optimal Substructure

* 문제 2 : 허프만 코드

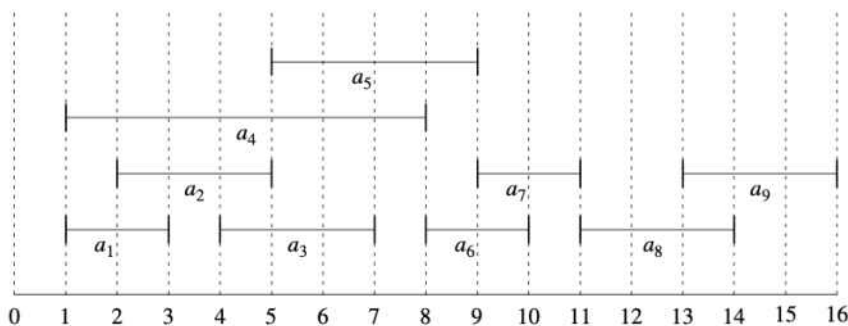
- 데이터 압축에서 널리 쓰이는 알고리즘. Greedy의 성공적인 예시

Activity-selection 문제

상호 호환되는 활동들을 최대한 많이 선택하는 방법

각각의 액티비티 는 시작시간 s_i , 종료시간 f_i 를 가지며 $0 \leq s_i < f_i$ 인 관계임.

a_i 와 a_j 는 $[s_i, f_i)$ and $[s_j, f_j)$ do not overlap 오버랩되지 않으면 **Compatible**하다.



i	1	2	3	4	5	6	7	8	9
s_i	1	2	4	1	5	8	9	11	13
f_i	3	5	7	8	9	10	11	14	16

Sorted in increasing order by finish time

How to organize a set of lectures so that the classroom (resource) is utilized as much as possible?

>> 가장 최대로 사용하려면 어떻게 해야 할까?

Compatible : a_2-a_5 $a_2=a_5-a_8$

NOT Compatible : $a_1-a_4-a_6$

S1 = (a2,a5,a8) : Mutually compatible activities하지만, Maximum-size set이 아니다

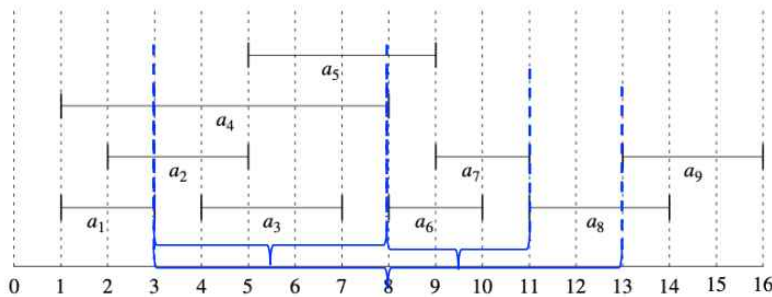
S2 = (a1,a3,a6,a8) : **Mutually compatible activities**하고, **Maximum-size set**

S3 = (a2,a5,a7,a9) : 또다른 **Mutually compatible activities**하고, **Maximum-size set**

Activity-Selection 문제의 Optimal Substructure

j 는 활동 i 가 끝난 후, 활동 a_j 가 시작되기 전에 호환 가능한 활동들의 집합.

$$S_{ij} = \{a_k \in S : \boxed{f_i \leq s_k} < \boxed{f_k \leq s_j}\}$$



$$S_{16} = \{a_3\}$$

$$S_{48} = \{a_6, a_7\}$$

$$S_{19} = \{a_3, a_5, a_6, a_7\}$$

Subproblem : S_{ij} 에서 최대 크기의 mutually compatible 집합을 찾는 것

S_{16} a1이 끝나고 난 후 ~ a6이 시작하기 전

S_{48} a4가 끝나고 난 후 ~ a8이 시작하기 전

S_{19} a1이 끝나고 난 후 ~ a9가 시작하기

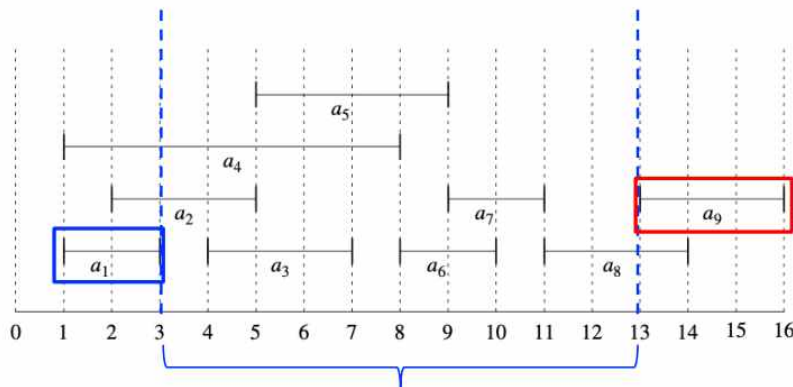
A_{ij} 는 S_{ij} 사이에 있는 a의 집합들 중 하나.

■ A_{ij} denotes an optimal solution to S_{ij} for $i \leq j$

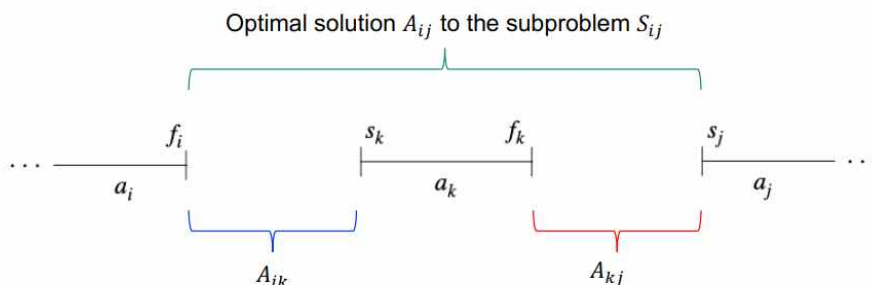
□ A maximum-size set of mutually compatible activities

$$S_{19} = \{a_3, a_5, a_6, a_7\}$$

$$A_{19} = \{a_3, a_6\} \text{ or } \{a_3, a_7\} \text{ or } \{a_5, a_7\}$$



■ If A_{ij} includes a_k , $A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$, $|A_{ij}| = |A_{ik}| + |A_{kj}| + 1$



The subproblem S_{ij} can be divided into the two subproblems S_{ik} and S_{kj}

$|A_{ij}| = |A_{ik}| + |A_{kj}| + 1$: 이는 A_{ij} 에 포함된 활동의 개수($|A_{ij}|$)가 A_{ik} 에 포함된 활동의 개수($|A_{ik}|$), A_{kj} 에 포함된 활동의 개수($|A_{kj}|$), 그리고 중간에 선택된 활동 a_k 하나를 더한 것과 같다는 것을 의미함

절댓값을 씌운 것은 집합의 갯수를 나타낸 것임.

>> Subproblem j 는 S_{ik} 와 S_{kj} 2개로 나눌 수 있다!

Activity-Selection을 DP로 해결

tabulation을 함. Bottom-up 방법.

$c[i,j]$: A_{ij} 의 활동의 최댓값

$$c[i, j] = \begin{cases} 0 & \text{if } S_{ij} = \emptyset, \\ \max_{a_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\} & \text{if } S_{ij} \neq \emptyset. \end{cases}$$

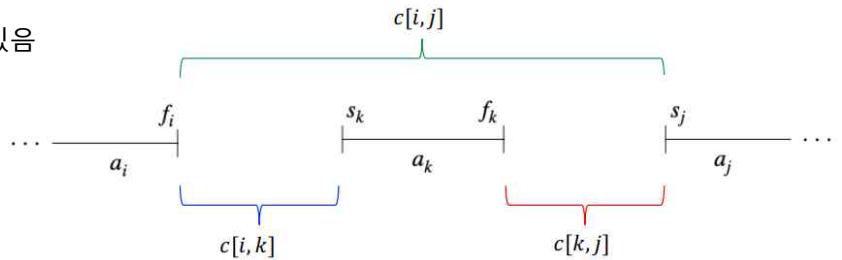
We have to examine all possible activities a_k in S_{ij} to make the best choice

재귀를 사용해서 테이블 C를 채울 수 있음

모든 subproblems가 필요함.

>> 다른 효율적인 방법이 없을까?

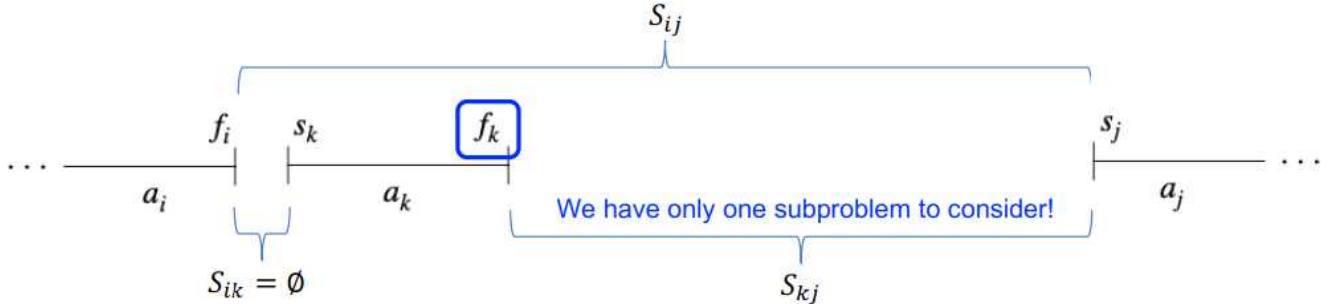
-> GREEDY!



Greedy 알고리즘으로 Activity-Selection 해결

모든 SP 필요없이, Greedy한 선택(특정 순간에서 최선의 선택)을 해 보자

각각의 반복에서 **finish time(종료시간)이 가장 빠른 것을 선택**하는 것. 활동이 종료되는 시간이 빠를수록, 자원을 다른 활동들에 더 많이 할당할 수 있기 때문.



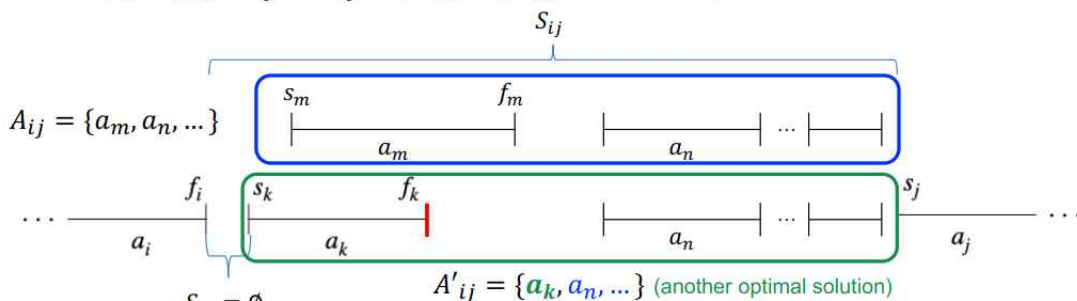
S_{ik} is empty since a_k have the earliest finish time f_k

이렇게 함으로서 해결해야 할 SP를 두개에서 하나로 줄인다!

Greedy한 선택이 Main problem이 **항상** 최적해를 구하는 방향으로 될까?

A_{ij} 는 S_{ij} 에서 mutually compatible activities의 최대크기 subset으로, m 은 A_{ij} 에서 가장 빠른 finish time인 a이다.

■ Since $f_k \leq f_m$, $A'_{ij} = A_{ij} - \{a_m\} \cup \{a_k\}$ is also an optimal solution



A'_{ij} 는 A_{ij} 에서 a_m 을 빼고 a_k 를 넣은 것인데 이것 또한 최적해이다

>> 가장 빨리 끝나는 활동으로 대체해도 여전히 최적해를 유지한다.

>> 가장 빨리 끝나는 활동은 최적 해를 찾을 때 포함되어야 하는 중요한 요소이다.

The activity with the earliest finish time should be included in some optimal solutions!

Greedy Algorithm Pseudo code

우선 Activity들을 finish time 오름차순으로 정렬.

i	1	2	3	4	5	6	7	8	9
s_i	1	2	4	1	5	8	9	11	13
f_i	3	5	7	8	9	10	11	14	16

GREEDY-ACTIVITY-SELECTOR(s, f)

```

1   $n = s.length$ 
2   $A = \{a_1\}$  << 은 무조건 포함. 그것이 Greedy이니까
3   $k = 1$  << 마지막으로 선택된 활동의 인덱스
4  for  $m = 2$  to  $n$ 
5      if  $s[m] \geq f[k]$  Check if they are compatible
6           $A = A \cup \{a_m\}$   $a_m$ 도 포함시키기
7           $k = m$  k를 업데이트
8  return  $A$ 
```

The greedy approach solves the activity-selection problem with a set of n activities in $\Theta(n)$ time

예시 적용:

1. 활동들을 종료 시간(f_i) 기준으로 정렬합니다.
2. $A = \{a_1\}$ (활동 1 선택, $f_1 = 3$)
3. $k = 1$
4. 활동 2 ($s_2 = 2, f_2 = 5$): $s_2(2) < f_1(3)$ 이므로 선택 불가.
5. 활동 3 ($s_3 = 4, f_3 = 7$): $s_3(4) > f_1(3)$ 이므로 선택 가능. $A = \{a_1, a_3\}$, $k = 3$.
6. 활동 4 ($s_4 = 1, f_4 = 8$): $s_4(1) < f_3(7)$ 이므로 선택 불가.
7. 활동 5 ($s_5 = 5, f_5 = 9$): $s_5(5) < f_3(7)$ 이므로 선택 불가.
8. 활동 6 ($s_6 = 8, f_6 = 10$): $s_6(8) > f_3(7)$ 이므로 선택 가능. $A = \{a_1, a_3, a_6\}$, $k = 6$.
9. 활동 7 ($s_7 = 9, f_7 = 11$): $s_7(9) < f_6(10)$ 이므로 선택 불가.
10. 활동 8 ($s_8 = 11, f_8 = 14$): $s_8(11) > f_6(10)$ 이므로 선택 가능. $A = \{a_1, a_3, a_6, a_8\}$, $k = 8$.
11. 활동 9 ($s_9 = 13, f_9 = 16$): $s_9(13) < f_8(14)$ 이므로 선택 불가.

Activity selection문제에서 Greedy Approach의 시간복잡도 $\Theta(n)$ 임.

DP vs Greedy Algorithm

DP : SP가 해결된 후에 선택을 함
여러 SP(부분문제)가 만들어질 수 있다
구현, 실행 측면에서 더 복잡하고 느리다.
SP의 결과를 재사용한다

Time Complexity: (n^3) = $O(n^2)$ SP의 개수 * $O(n)$ 각 SP를 계산하는데 드는 시간

Greedy : SP가 해결되기 전에 선택을 함
1가지의 SP만 생성된다.
구현, 실행 측면에서 더 간단하고, 빠르다
SP의 결과를 재사용하지 않는다
★ main problem에 대해 최적해를 제공하지 않을 수도 있다.

Time Complexity: $O(n)$ but, f_i 를 sorting하기 위해서는 $O(n \lg n)$ 이 필요.

□ Exercise 1:

- Solve the following activity selection problem using the greedy approach

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

정답 : { , a_4, a_8, a_{11} }

Greedy Approach의 적용성

언제 적용할 수 있을까?

Activity-Selection 문제에서 얻은 통찰.

우선 SP_{ij} 와 최적 부분문제(Optimal Substructure)를 정의함.

Greedy한 선택이 딱 1개 나옴을 봄. $S_{k,j}$ 라는 유일한 SP가 만들어짐.

($S_{i,k}, S_{k,j}$ 이렇게 두개가 만들어지지 않음!)

Greedy한 선택이 **안전하다**는 것을 증명함.

>> 실제로 전체 문제의 최적해 중 하나에 반드시 포함함.

>> 일반적인 Greedy Algorithm에서는 최적해를 보장하지 않는 경우가 많지만, 이 경우는 보장

결론적으로 Greedy 선택을 통해 하나의 $SP(S_{k,j})$ 로 축소시키고, 축소된 부분에 대해서도 같은 방식으로 반복.

Greedy-Choice 특성

Global한 최적해는 **locally optimal choice**를 함으로서 구성될 수 있다.

이것이 핵심.

Greedy는 **지금까지의(So far)**선택에 의존할 수 있지만, **미래의 선택에는 의존할 수 없다**.

미래에 일어날 부분은 전혀 고려하지 않음. 현재 가장 최적의 선택을 내림.

Optimal Substructure 속성

문제에 대한 최적해는 부분문제들에 대한 최적해를 포함한다.

DP와 Greedy 알고리즘 모두에서 해당되고 중요

큰 문제의 최적 해가 더 작은(동일한 형태의) 부분 문제들의 최적 해를 조합하여 얻어질 수 있다

Greedy의 미묘한 점(Subtleties)

0-1 배낭(Knapsack) 문제

도둑이 물건을 훔치려고 한다. i 번째 물건은 v_i 달러 가치이고, w_i 파운드이다.

도둑은 총 W 파운드를 옮길 수 있다. 어떤 걸 훔쳐야 할까?

물건은 쪼갤 수 없다.

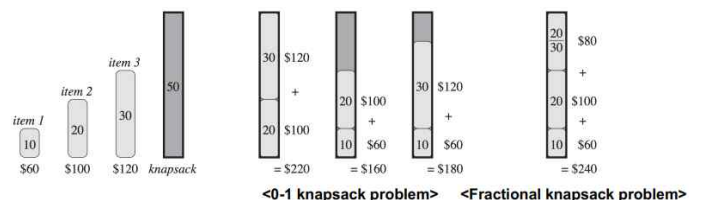
>> Greedy로 Optimal Solution **보장 불가**

분할가능한(Fractional) 배낭 문제

물건을 쪼갤 수 있다.

>> Greedy로 Optimal Solution **보장 가능**. 단위무게당 가치가 높은 물건부터 순서대로 채우면 됨

Greedy의 속성 + Optimal Substructure의 속성을 모두 만족! 최적해 보장.



We can solve the fractional knapsack problem by the greedy approach, but cannot solve the 0-1 problem

Huffman Codes 허프만 코드

Greedy-approach를 사용하는 **데이터 압축 방식**. 보통 20~90% 공간을 절약할 수 있음.

어떻게 Encode/Decode를 하는가?

10,000개의 짝 찬 문자가 있다고 하자. 문자들은 a~f까지이고 아래같은 분포를 띤다.
2진수로 바꾼다.

- One possible way to represent this file: **binary character code** (**code** for short)

□ e.g., abc: 000 001 010, acef: 000 010 100 101

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101

Each character should be represented by a unique binary string (**codeword**) for decoding!

고정 길이 코드 Fixed-Length Code

같은 길이의 binary 코드를 모든 글자에 할당한다. 6글자를 표현하려면 최소 3비트가 필요함.

$$\begin{aligned} &= (45,000 + 13,000 + 12,000 + 16,000 + 9,000 + 5,000) * 3 \\ &= \mathbf{300,000 \text{ bits}} \end{aligned}$$

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101

>> 자주 나오는 글자에 짧은 코드워드를 부여하고, 덜 자주 나오는 글자에 긴 코드워드를 부여해보자!

→ Giving **short** codewords to **frequent** characters,
while giving **long** codewords to **infrequent** characters

가변 길이 코드 Variable-Length Code

빈도에 따라서 다른 길이의 binary 코드를 문자에 할당한다.

$$\begin{aligned} &= (45,000 * 1 + 13,000 * 3 + 12,000 * 3 + 16,000 * 3 + 9,000 * 4 + 5,000 * 4) \\ &= \mathbf{224,000 \text{ bits}} \end{aligned}$$

Saving about 25% of space!

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Variable-length codeword	0	101	100	111	1101	1100

Q. How can we construct optimal variable-length codewords?

>> Huffman Coding을 사용하면 됨!

Prefix 코드 (Prefix-Free 코드)

만약, 'a'를 0이라 하고 'b'를 01이라하고 'c'를 1이라하면 001이 aac인지 ab인지 **모호성**이 생긴다.
그래서 어떤 codeword도 다른 codeword와 **prefix가 겹치지 않게** 해야 한다. >> huffman

- We can unambiguously decode the encoded data

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Variable-length codeword	0	101	100	111	1101	1100

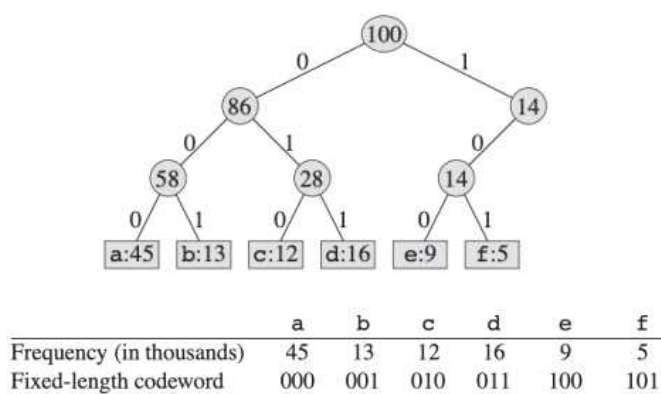
Prefix 코드를 Binary 트리로 표현하기

문자에 대한 Prefix Code는 Binary 트리에서 Root에서 Leaf까지의 단순 경로로 해석될 수 있다. 루트->리프
 각 leaf 노드는 루트에서 다른 노드까지의 단순 경로에 포함될 수 없다(Prefix)

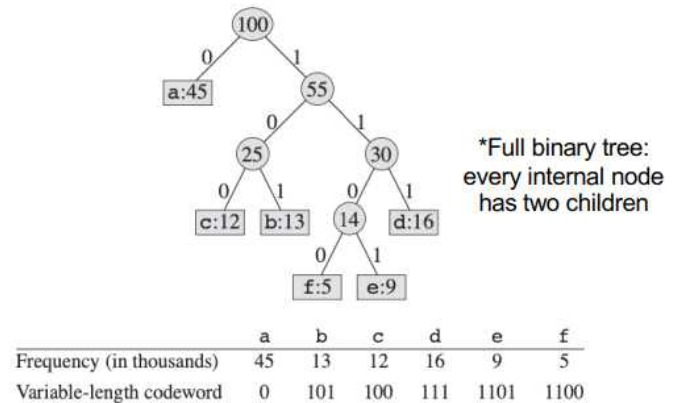
>> 그래서 어떤 문자를 나타내는 코드는 항상 leaf 노드가 되어야 함.

Full Binary Tree로 표현하는 Prefix Codes

데이터에 대한 최적 코드는 항상 **완전 이진(Full Binary) 트리**(자식이 0 or 2)로 표현된다.



A binary tree for 3-bit fixed-length codes,
 where all leaf nodes are placed in the last level



a **FULL** binary tree for the variable-length codes,
 where leaf nodes are placed in different levels

완전이진 트리로 만든 다음에 빈도가 높은 것을 길이가 짧은 코드를 할당

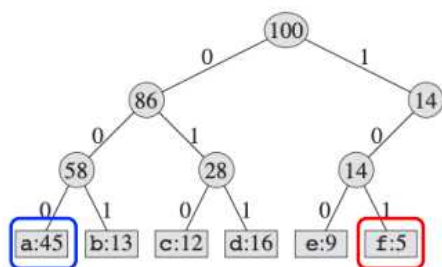
어떤 문자집합 C가 있다고 하면, $|C|$ (문자의 총 개수) == leaf 노드의 개수
 내부 노드의 개수 (leaf 제외 노드) == $|C|-1$ (문자의 총 개수-1)

트리 T의 비용 : B(T)

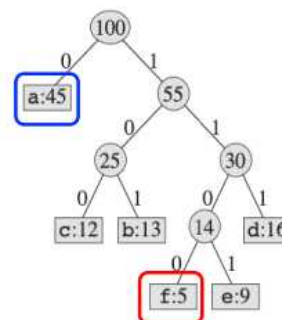
B(T) : 주어진 이진 트리 T를 사용해서 데이터를 인코딩할 때 필요한 비트 수

$$B(T) = \sum_{c \in C} c.freq \cdot d_T(c)$$

문자집합 C에 해당하는 모든 문자 c를 더해라
 ≡
 c.freq : c문자의 빈도 수
 $d_T(c)$: c문자의 코드워드의 수(몇 비트인가?)
 ~~~root로부터의 거리. 거리가 1이면 1비트, 2면 2비트...



VS



>> 우리의 목표는 B(T)를 최소화시키는 완전이진트리(Full binary Tree)를 만드는 것이다!

## 허프만 코드 Huffman Code란?

: Optimal한 Prefix code를 찾기 위해서 완전이진트리를 만드는 Greedy 알고리즘.

다뤄야 하는 것

**Greedy 알고리즘의 방법론** : Greedy 알고리즘을 사용해서 어떻게 동작하는지 살펴봐야 함

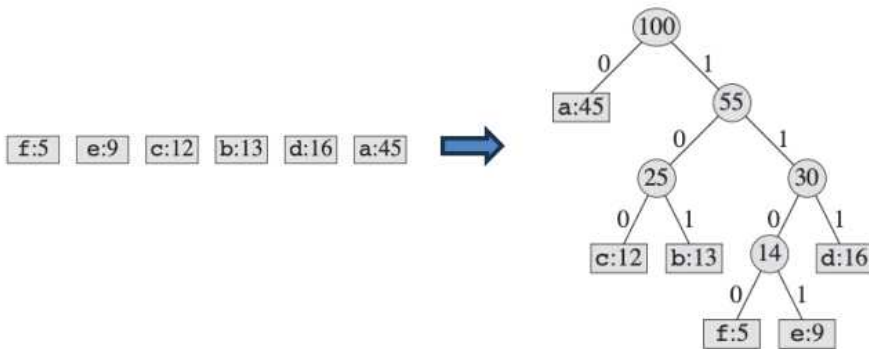
**Greedy 알고리즘의 정확도** : Greedy한 선택 특성을 갖는가? + Optimal Substructure인가?

## 허프만 코드 방법론

|C| leaf 노드에서 시작해서 |C|-1번의 'Merging 병합' 작업을 한다.

**Greedy한 선택** : 우선 2개의 가장 작은 빈도(least-frequent)의 문자를 합친다.

>> 빈도가 낮은 문자를 트리의 깊은 곳에 배치해야 B(T)가 최소화되기 때문!



### HUFFMAN(C)

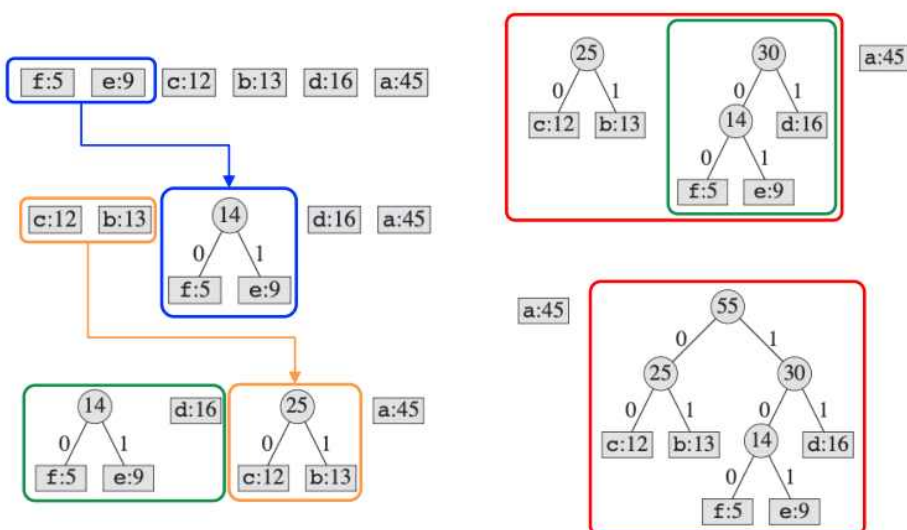
```

1  n = |C|           //Q는 Priority Queue
2  Q = C
3  for i = 1 to n - 1
4      allocate a new node z
5      z.left = x = EXTRACT-MIN(Q)
6      z.right = y = EXTRACT-MIN(Q)
7      z.freq = x.freq + y.freq
8      INSERT(Q, z)
9  return EXTRACT-MIN(Q)

```

의사코드는 허프만 코드를 생성하는 알고리즘의 단계별 논리적 흐름을 보여줍니다.

1.  $n = |C|$  : 전체 문자의 개수를  $n$  에 저장합니다.
2.  $Q = C$  : 최소 우선순위 큐  $Q$  를 초기화합니다. 처음에는 모든 문자(각자의 빈도와 함께)가 이 큐에 노드로 들어 갑니다.
3. for  $i = 1$  to  $n - 1$  :  $n-1$  번의 병합 작업을 반복합니다. (이것이 위에서 설명한  $|C| - 1$  번의 병합 작업입니다.)
4. allocate a new node  $z$  : 새로운 내부 노드  $z$  를 생성합니다.
5.  $z.left = x = \text{EXTRACT-MIN}(Q)$  : 큐  $Q$  에서 가장 빈도가 낮은 노드를 추출하여  $x$  에 할당하고,  $z$  의 왼쪽 자식으로 설정합니다.
6.  $z.right = y = \text{EXTRACT-MIN}(Q)$  : 큐  $Q$  에서 (남아있는 노드들 중) 두 번째로 빈도가 낮은 노드를 추출하여  $y$  에 할당하고,  $z$  의 오른쪽 자식으로 설정합니다.
7.  $z.freq = x.freq + y.freq$  : 새로 생성된 노드  $z$  의 빈도는  $x$  와  $y$  의 빈도를 합한 값으로 설정합니다. (이것이 트리의 중간 노드에 표시되는 합산된 빈도입니다. 예:  $c:12$  와  $b:13$  이 합쳐져  $25$  가 되는 것처럼).
8.  $\text{INSERT}(Q, z)$  : 새로 생성된 노드  $z$  를 다시 우선순위 큐  $Q$  에 삽입합니다. 이제  $z$  는 다른 노드들과 함께 빈도에 따라 정렬되어 다음 병합 과정의 후보가 됩니다.
9. return  $\text{EXTRACT-MIN}(Q)$  : 모든  $n-1$  번의 병합 작업이 끝나면, 큐  $Q$  에는 단 하나의 노드(최종적으로 완성된 트리의 루트)만 남아 있게 됩니다. 이 루트 노드를 반환하여 최종 허프만 트리를 완성합니다.



|                          |    |     |     |     |      |      |
|--------------------------|----|-----|-----|-----|------|------|
|                          | a  | b   | c   | d   | e    | f    |
| Frequency (in thousands) | 45 | 13  | 12  | 16  | 9    | 5    |
| Variable-length codeword | 0  | 101 | 100 | 111 | 1101 | 1100 |

\*Each merging generates an internal node with the sum of their frequencies



Huffman Codes의 복잡도 -  $n \lg n$

Build min-heap : finish 값 기준으로 정렬하기

$O(n)$

Merge and Min-Heapify : (n-1)번의 병합을 함. 각각의 병합동안 Min-Heapify함.

$O(n \lg n)$

## Huffman Codes의 정확도

Greedy-choice property를 만족하는가? - Lemma 1

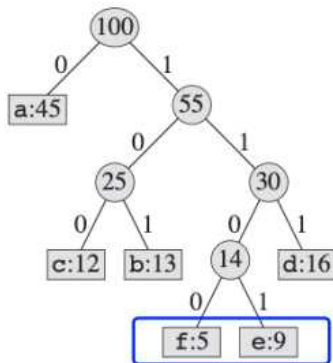
Optimal substructure를 만족하는가? - Lemma 2

### Lemma1 : Greedy-Choice Property

C가 문자의 집합이라고 하자. 여기서  $c \in C$ 는 c.freq 빈도수를 가진다.

x와 y가 가장 낮은 빈도를 가진 문자라고 하자.

그러면, C에 대한 최적의 Prefix code가 존재하는데, 이 둘은 codeword의 길이는 같고 마지막 비트만 다르다!



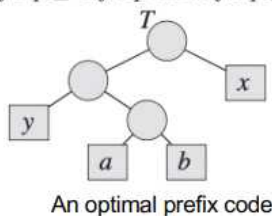
|                          | a  | b   | c   | d   | e    | f    |
|--------------------------|----|-----|-----|-----|------|------|
| Frequency (in thousands) | 45 | 13  | 12  | 16  | 9    | 5    |
| Variable-length codeword | 0  | 101 | 100 | 111 | 1101 | 1100 |

### 증명 아이디어

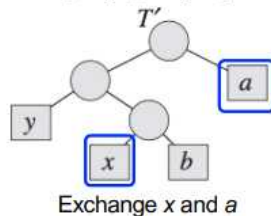
1. 임의의(arbitrary) optimal prefix tree T가 있다고 하고, (이미 애는 최적)

2. 이 T를 수정해서 새로운 최적 접두어 코드 트리 T'를 만든다. 이때, 가장 낮은 빈도수를 가진 두 문자 x와 y가 새 트리 T'에서 최대 깊이의 형제 leaf(sibling leaves of max depth)로 정한다.

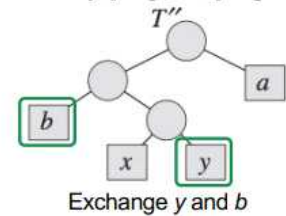
Assume that  
 $a.freq \leq b.freq$  and  $x.freq \leq y.freq$



$B(T) \geq B(T')$   
 $x.freq \leq a.freq$



$B(T') \geq B(T'')$   
 $y.freq \leq b.freq$



1. 위 가정에 따라 -----  $B(T) \geq B(T') \geq B(T'')$

2. 최적 prefix code이므로  $\therefore T$  is an optimal prefix code  $B(T) \leq B(T'')$

>>따라서 1,2를 모두 만족해야하므로 >>>>>>  $\rightarrow B(T) = B(T'')$

Another optimal prefix code,  
 where x and y have the maximum depth

$\therefore$  매 단계에서 가장 작은 빈도수를 가진 두 문자를 묶는 선택을 하는 것이 optimal prefix code를 구하는 데 문제가 없고, 오히려 도움이 된다.

### Lemma 2 : Optimal Substructure

C를 문자의 집합이라고 가정하고, x와 y를 가장 낮은 빈도를 가진 문자라 하자.

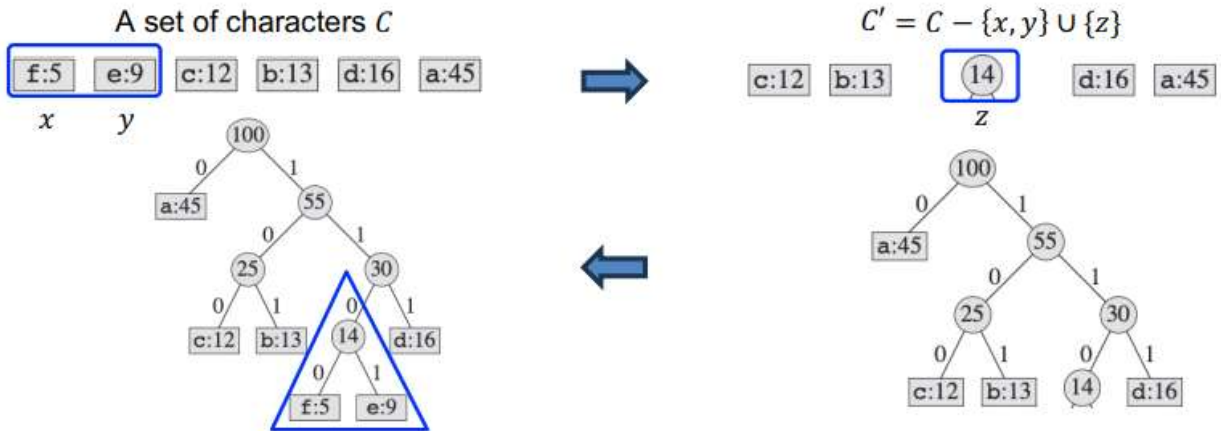
새로운 문자집합을 C'라 하고,  $C' = C - \{x, y\} \cup \{z\}$  {x,y}를 제거하고 {z}를 넣은 것이다.

z의 빈도수는 x+y의 빈도수이다.

T'을 C'의 트리라 하자.

>> C에 대한 Optimal prefix code tree T를 T'로부터 얻어올 수 있다.

>>>>Optimal Substructure 특성을 만족한다.



## Exercise 2-1:

- What is an optimal Huffman code for the following set of frequencies?

The first 8 Fibonacci numbers

a:1 b:1 c:2 d:3 e:5 f:8 g:13 h:21

## Exercise 2-2:

- Generalize your answer to find the optimal code when the frequencies are the first  $n$  Fibonacci numbers

피보나치 수열을 빈도로 쓰면, 허프만 코드가 한쪽으로 치우친(스큐드) 형태가 되어, 코드 길이가 차례로 1씩 짧아집니다.

## 11주차 - Amortized Analysis

### Amortized Analysis란?

많은 알고리즘에서 일부 연산은 느리지만, 대부분의 연산은 빠르다

ex) 커피만들기 - 느림      커피 붓기 - 빠름

(느린) 단일 연산을 분석하기 보다는, **일련의 연산(Sequence of operations)**에 걸친 **시간복잡도**를 분석하는 방법.

KEY IDEA : 비싼 연산의 비용을 **여러 연산에 걸쳐서 분산**시키는 것.

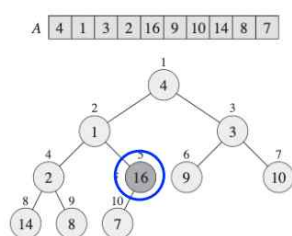
일련의 연산을 수행하는 데 필요한 비용을 **평균**내는 것

>> 최악의 경우에도 각 연산의 **평균 performance**를 보장한다.

### Build-Max-Heap의 예제

최악의 경우 :  $n \lg n$

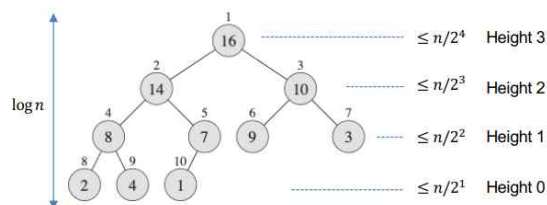
Amortized 분석 :  $O(n)$ . 실제로는  $O(n \lg n)$ 보다 훨씬 적게 시간이 걸림.



BUILD-MAX-HEAP( $A$ )  $\rightarrow O(n \lg n)$

- 1  $A.heap-size = A.length$
- 2 **for**  $i = \lfloor A.length/2 \rfloor$  **downto** 1  $\rightarrow O(n)$
- 3 MAX-HEAPIFY( $A, i$ )  $\rightarrow O(\lg n)$

Worst-case analysis



$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left[ \frac{n}{2^{h+1}} \right] O(h) = O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right) = O(n)$$

# of nodes of height  $h$

The cost of Max-Heapify on a node of height  $h$

The average performance of each operation is  $O(1)$

Amortized analysis

## 단순히 평균 구하는 것과의 차이

평균 케이스 분석은 입력 데이터의 **확률 분포(probability distribution)**가 필요하다.

평균은 최악의 경우 분석이나 amortized 분석보다 복잡할 수 있다. (더 복잡한 계산이 필요하기 때문에)

>> Amortized 분석은 다른 분석 도구들을 보완할 수 있다.

## Amortized 분석의 장점

### 1. 더 정확한 분석.

최악의 경우에도 각 연산(여러 연산)의 평균 performance를 제공함.

### 2. 효율적인 알고리즘을 디자인하는데 유용함.

일련의 연산에 걸쳐서 알고리즘이 **어떻게 동작하는지 이해**하는 데 도움이 됨.

자료 구조의 **적절한 크기**를 결정할 수 있음.(e.g. 스택, 해시, 배열,...) => **dynamic 테이블!**

## Amortized 분석 Overview

### 3가지 Amortized Analysis의 방법

1. Aggregation Analysis
2. Accounting Method
3. Potential Method

### 2가지 예시

MULTIPOP이라는 추가적인 연산을 가진 Stack. 한번에 여러개 POP.

INCREMENT라는 단일 연산으로 0부터 세어 올라가는 이진 카운터.

## Aggregation Analysis

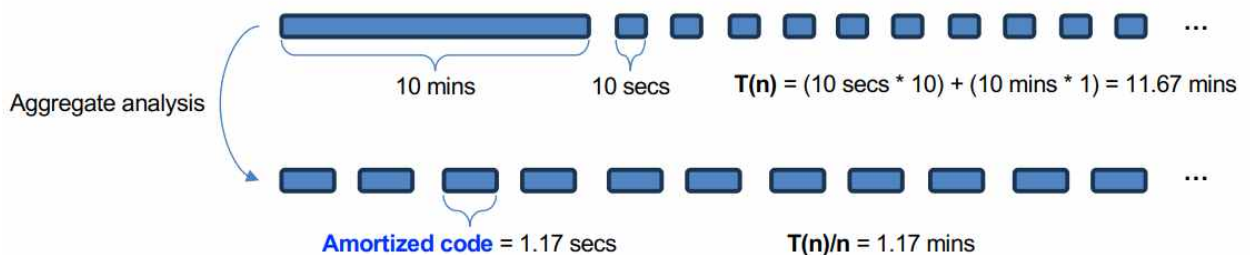
### 1. 최악의 경우에 n개의 연산 시퀀스에 대한 **총 비용 T(n)**을 계산합니다.

n개의 연산 시퀀스를 수행하는 데 드는 실제 비용을 모두 합친 것

### 2. 총 비용을 평균하여 연산당 **Amortized 비용**을 결정한다.

Amortized cost =  $T(n)/n$

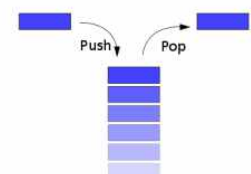
이 Amortized cost는 여러가지 유형의 연산이 있더라도 모든 연산에 적용된다. 즉, 비싼 연산이든, 싼 연산이든 모두 평균된 비용을 가진다고 간주함.



We can obtain a better upper bound of the total cost than the worst-case analysis

>> worst-case 분석보다 총 비용에 대한 **더 나은 Upper bound**를 얻을 수 있다.

worst case보다 **더 현실적**이다.



## MULTIPOP Stack

### : Aggregation

----->

### Stack Operations

- $Push(S, x)$ : push an element  $x$  into stack  $S$ 
  - Cost:  $O(1)$
- $Pop(S)$ : pop the top element of stack  $S$  and return it
  - Cost:  $O(1)$
- $Multipop(S, k)$ : pop the top- $k$  elements of stack  $S$ 
  - Cost:  $O(k)$  ( $1 \leq k \leq n$ )

MULTIPOP( $S, k$ )

```
1 while not STACK-EMPTY( $S$ ) and  $k > 0$ 
2   POP( $S$ )
3    $k = k - 1$ 
```

## Aggregate Analysis

직관 : 스택에서 원소는 **스택이 비어있지 않을 때만 pop**을 할 수 있다. >> pop을 하려면 push된게 있어야 함.  
Pop연산의 합은 Push연산의 합을 넘을 수 없다.

$\langle \text{Pop}, \text{Pop} \rangle \quad \langle \text{Pop}, \text{Pop}, \text{Pop} \rangle$   
 $\langle \text{Push}, \text{Push}, \text{Pop}, \text{Push}, \text{Multipop}(2), \dots, \text{Multipop}(3), \text{Pop} \rangle$   
 $n \text{ operations}$

$$\#(\text{Pop}) \leq \#(\text{Push}) \leq n$$

$$\#(\text{Pop}) + \#(\text{Push}) \leq 2n$$

The total cost of a sequence of operations is  $O(n)$

→ **Amortized cost** is  $O(n)/n = O(1)$

Amortized analysis 中 Aggregate analysis는 현실적인 제약을 고려한다!

>> 실제 연산 시퀀스에서 발생하는 비용의 총합은  $O(n)$ 을 넘지 않으며,  
n으로 나누면 **Amortized cost**는  $O(1)$ 이다.

## Binary Counter

### ■ Consider the problem of implementing a $k$ -bit binary counter

- It counts upward from 0
- We can use an array  $A[0, 1, \dots, k-1]$  as the counter

|          |     |        |        |        |
|----------|-----|--------|--------|--------|
| $A[k-1]$ | ... | $A[2]$ | $A[1]$ | $A[0]$ |
|----------|-----|--------|--------|--------|

Highest-order bit

Lowest-order bit

| Counter value | $A[k-1]$ | ... | $A[2]$ | $A[1]$ | $A[0]$ |
|---------------|----------|-----|--------|--------|--------|
| 0             | 0        | ... | 0      | 0      | 0      |
| 1             | 0        | ... | 0      | 0      | 1      |
| 2             | 0        | ... | 0      | 1      | 0      |
| 3             | 0        | ... | 0      | 1      | 1      |

| Counter value | $A[k-1]$ | ... | $A[2]$ | $A[1]$ | $A[0]$ |
|---------------|----------|-----|--------|--------|--------|
| 4             | 0        | ... | 1      | 0      | 0      |
| 5             | 0        | ... | 1      | 0      | 1      |
| 6             | 0        | ... | 1      | 1      | 0      |
| 7             | 0        | ... | 1      | 1      | 1      |

What is the cost of INCREMENT operation in this binary counter?

**최악의 경우** : 모든 비트가 1일 때, INCREMENT 연산을 수행하면, 모든 비트가 0으로 바뀌고 가장 높은 자리 비트가 1이 된다. 따라서  $k$ 개의 비트가 모두 뒤집히므로  $O(k)$ 이다.

**대부분의 경우** : 가장 오른쪽 비트( $A[0]$ )만 뒤집거나, 두 비트( $A[0], A[1]$ )만 뒤집는 비트 플립이 발생하므로  $O(1)$ 에 가깝다.

>> INCREMENT 연산의 비용은 **bit flip**의 수에 비례한다!

- Cost of INCREMENT operation is **proportional to the number of bit flips**

INCREMENT( $A$ )

```

1   $i = 0$ 
2  while  $i < A.length$  and  $A[i] == 1$ 
3     $A[i] = 0 \rightarrow O(1)$ 
4     $i = i + 1$ 
5  if  $i < A.length$ 
6     $A[i] = 1 \rightarrow O(1)$ 
    
```

The number of bit flips can vary depending on counter values!

비트 플립 수는 카운터 값에 따라서 바뀐다.  
매우 불규칙적.

| Counter value | $A[4]$ | $A[3]$ | $A[2]$ | $A[1]$ | $A[0]$ |    |
|---------------|--------|--------|--------|--------|--------|----|
| 0             | 0      | 0      | 0      | 0      | 0      |    |
| 1             | 0      | 0      | 0      | 0      | 1      | +1 |
| 2             | 0      | 0      | 0      | 1      | 0      | +1 |
| 3             | 0      | 0      | 0      | 1      | 1      | +1 |
| 4             | 0      | 0      | 1      | 0      | 0      | +1 |
| 5             | 0      | 0      | 1      | 0      | 1      | +1 |
| 6             | 0      | 0      | 1      | 1      | 0      | +1 |
| 7             | 0      | 0      | 1      | 1      | 1      | +1 |
| 8             | 0      | 1      | 0      | 0      | 0      | +1 |



- Cost at each step is proportional to the number of bit flips

| Counter value | A[3] | A[2] | A[1] | A[0] | Cost | Total Cost |
|---------------|------|------|------|------|------|------------|
| 0             | 0    | 0    | 0    | 0    | 0    | 0          |
| 1             | 0    | 0    | 0    | 1    | 1    | 1          |
| 2             | 0    | 0    | 1    | 0    | 2    | 3          |
| 3             | 0    | 0    | 1    | 1    | 1    | 4          |
| 4             | 0    | 1    | 0    | 0    | 3    | 7          |
| 5             | 0    | 1    | 0    | 1    | 1    | 8          |
| 6             | 0    | 1    | 1    | 0    | 2    | 10         |
| 7             | 0    | 1    | 1    | 1    | 1    | 11         |
| 8             | 1    | 0    | 0    | 0    | 4    | 15         |

**Worst case** : k자리수인데 k비트가 바뀌는 경우. 따라서  $O(nk)$ 가 worst case이다.

|   |   |   |   |   |   |    |
|---|---|---|---|---|---|----|
| 8 | 1 | 0 | 0 | 0 | 4 | 15 |
|---|---|---|---|---|---|----|

Thus, a sequence of  $n$  INCREMENT operations takes  $O(nk)$  time in the worst case.  
However, this is not tight upper bound!

>> 왜 Tight upper bound가 아닌가? > 이렇게 k자리가 모두 flip되는 경우는 드물기 때문에.

## □ Observation

- Computing the number of bit flips of  $A[i]$

- $A[0]$ :  $n$  (flip at every step)
- $A[1]$ :  $\frac{n}{2}$  (flip at every two steps)
- $A[2]$ :  $\frac{n}{4}$  (flip at every four steps)
- ...

## □ Aggregate Analysis

- The total number of bit flips in the sequence

$$\sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n,$$

- Thus, the total cost is  $O(n)$

→ **Amortized cost per operation** is  $O(n)/n = O(1)$

| Counter value | A[3] | A[2] | A[1] | A[0] | Cost |
|---------------|------|------|------|------|------|
| 0             | 0    | 0    | 0    | 0    | 0    |
| 1             | 0    | 0    | 0    | 1    | 1    |
| 2             | 0    | 0    | 1    | 0    | 2    |
| 3             | 0    | 0    | 1    | 1    | 1    |
| 4             | 0    | 1    | 0    | 0    | 3    |
| 5             | 0    | 1    | 0    | 1    | 1    |
| 6             | 0    | 1    | 1    | 0    | 2    |
| 7             | 0    | 1    | 1    | 1    | 1    |
| 8             | 1    | 0    | 0    | 0    | 4    |

$\frac{n}{8}$   $\frac{n}{4}$   $\frac{n}{2}$   $n$   
At every eight steps At every four steps At every two steps At every step

At every four steps

At every eight steps

## □ Exercise 1

- Suppose we perform a sequence of  $n$  operations on a data structure where the  $i$ -th operation costs  $i$  if  $i$  is an exact power of 2, and 1 otherwise. Use aggregate analysis to **determine the amortized cost per operation**.

$$c_i = \begin{cases} i & \text{if } i \text{ is an exact power of 2,} \\ 1 & \text{otherwise.} \end{cases}$$

| Operation | Cost |
|-----------|------|
| 1         | 1    |
| 2         | 2    |
| 3         | 1    |
| 4         | 4    |
| 5         | 1    |
| 6         | 1    |
| 7         | 1    |
| 8         | 8    |
| 9         | 1    |
| 10        | 1    |
| ...       | ...  |

$$\begin{aligned} &= 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + \dots + 1 + 1 \\ &= 2^0 + 2^1 + 0 + 2^2 + 0 + 0 + 0 + 2^3 + \dots + 2^{\lg n} \end{aligned}$$

$$T(n) = \sum_{i=1}^n c_i \leq \boxed{n} + \sum_{j=0}^{\lg n} 2^j = n + (2n - 1) < 3n$$

<<3n을 넘지는 않음  
증명 어느정도 외우기

→ **Amortized cost per operation** is  $O(n)/n = O(1)$

The total actual cost is  $O(n)$



## Accounting Method

**Aggregated analysis와의 차이점** : Aggregate 분석은 모든 연산에 대해 동일한 평균 상환 비용을 할당하는 반면, 회계 분석은 각 연산 유형에 따라 상환 비용을 다르게 설정.

일부에는 실제 비용보다 많이 청구하고, 다른 연산에는 실제보다 적게 청구

**Credit(Prepaid cost 미리 적립시켜놓는 비용) :**

**amortized cost( $\hat{c}_i$ )와 실제 비용( $c_i$ ) 사이의 차이** ( $\hat{c}_i > c_i$ )

나중에 실제 cost가 amortized cost보다 더 높은(비싼) 연산의 비용을 지불하는 데 사용됩니다.

**어떻게 연산들의 Amortized cost를 고를 것인가?**

일련의 연산에 대한 총 Amortized cost는 총 **실제 비용의 Upper bound**를 제공하기 위해 총 실제 비용보다 크거나 같아야 한다.

□  $c_i$ : Actual cost of the  $i$ -th operation

□  $\hat{c}_i$ : Amortized cost of the  $i$ -th operation

$$\sum_{i=1}^n \hat{c}_i - \sum_{i=1}^n c_i \geq 0 \Rightarrow \sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$$

The total credits should be non-negative

Credit은 음수가 될 수 없다!

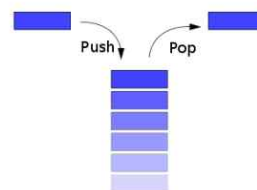
## MULTIPOP Stack : Accountin Method

### Actual Cost of Stack Operations

■  $Push(S, x)$ : 1

■  $Pop(S)$ : 1

■  $Multipop(S, k)$ :  $k$  ( $1 \leq k \leq n$ )



### Amortized Cost of Stack Operations

■  $Push(S, x)$ :  $1 + 1(\text{credit}) = 2$  → It is **larger** than the actual cost →

■  $Pop(S)$ : 0

■  $Multipop(S, k)$ : 0

→ They are **less** than the actual cost →

**Prepaying** the cost for popping the element pushed

Thus, we can charge **nothing** for Pop and Multipop operations

The prepaid credits are always **enough** to pay for Pop and Multipop operations

$$\sum_{i=1}^n \hat{c}_i - \sum_{i=1}^n c_i \geq 0$$

The total credit is non-negative

$$\Rightarrow \sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$$

The total amortized cost can provide an upper bound on the total actual cost

### Amortized cost

**Push** : 2 == 실제 비용에서 1 많음

>> Pop 할 때의 비용을 미리 지불!

**Pop** : 0 == 실제 비용보다 1 적음

>> Pop과 multipop 연산에서는 아무것도 바뀌지 않음

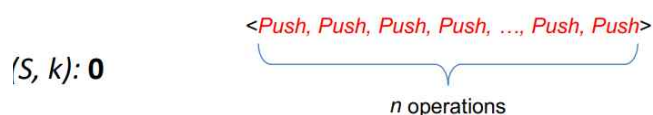
**Multipop** : 0 == 실제 비용보다 1 적음

>> Pop과 multipop 연산에서는 아무것도 바뀌지 않음

>> 미리 지불한 Credit은 Pop/Multipop 연산하기에 항상 충분하다! (음수가 되지 않음)

Worst-case일때 시퀀스의 Total amortized cost는  $O(2n)$ 이다.

>> 연산당 Amortized cost :  $O(n) / n = O(1)$



The total amortized cost of the sequence is  $O(2n)$  in the worst case

→ Amortized cost per operation is  $O(n)/n = O(1)$

## Binary Counter : Accounting Method

Binary counter 연산에서의 실제 비용

Bit flip (0→1) : 1

Bit flip (1→0) : 1

$$\sum_{i=1}^n \hat{c}_i - \sum_{i=1}^n c_i \geq 0$$

$$\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$$

The total amortized cost can provide an upper bound on the total actual cost

Binary counter 연산에서의 Amortized 비용

Bit flip (0→1) : **1+1 (credit)** >> flip될걸 미리 지불!

Bit flip (1→0) : 0 >> 아무것도 바뀌지 않음 (1→0)

>> 미리 지불한 credit은 다시 뒤집기 연산 (1→0)을 하기에 충분하다.

## Analysis of $n$ INCREMENT operations

■ Bit flip (0 → 1): 1 + **1 (credit)**, Bit flip (1 → 0): 0

$$\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$$

| Counter value | A[3] | A[2] | A[1] | A[0] | Amortized Cost $\hat{c}$ | Total Cost | Actual Cost $c$ | Total Cost |
|---------------|------|------|------|------|--------------------------|------------|-----------------|------------|
| 0             | 0    | 0    | 0    | 0    | 0                        | 0          | 0               | 0          |
| 1             | 0    | 0    | 0    | 1    | 2                        | 2          | 1               | 1          |
| 2             | 0    | 0    | 1    | 0    | 2                        | 4          | 2               | 3          |
| 3             | 0    | 0    | 1    | 1    | 2                        | 6          | 1               | 4          |
| 4             | 0    | 1    | 0    | 0    | 2                        | 8          | 3               | 7          |
| 5             | 0    | 1    | 0    | 1    | 2                        | 10         | 1               | 8          |
| 6             | 0    | 1    | 1    | 0    | 2                        | 12         | 2               | 10         |
| 7             | 0    | 1    | 1    | 1    | 2                        | 14         | 1               | 11         |
| 8             | 1    | 0    | 0    | 0    | 2                        | 16         | 4               | 15         |

The total amortized cost of  $n$  INCREMENT operations is  $O(n) = O(2n)$  in the worst case

→ **Amortized cost per operation** is  $O(n)/n = O(1)$

Worst-case일때 시퀀스의 Total amortized cost는  $O(2n)$ 이다.

>> 연산당 Amortized cost :  $O(n) / n = O(1)$

## Exercise 2

■ Suppose we perform a sequence of  $n$  operations on a data structure where the  $i$ -th operation costs  $i$  if  $i$  is an exact power of 2, and 1 otherwise. Use the accounting method to **determine the amortized cost per operation**.

$$c_i = \begin{cases} i & \text{if } i \text{ is an exact power of 2,} \\ 1 & \text{otherwise.} \end{cases}$$

| Operation | Cost | Actual cost | Credit remaining |
|-----------|------|-------------|------------------|
| 1         | 3    | 1           | 2                |
| 2         | 3    | 2           | 3                |
| 3         | 3    | 1           | 5                |
| 4         | 3    | 4           | 4                |
| 5         | 3    | 1           | 6                |
| 6         | 3    | 1           | 8                |
| 7         | 3    | 1           | 10               |
| 8         | 3    | 8           | 5                |
| 9         | 3    | 1           | 7                |
| 10        | 3    | 1           | 9                |
| ⋮         | ⋮    | ⋮           | ⋮                |

Set the amortized cost  $\hat{c}_i = 3$   $\sum_{i=1}^n \hat{c}_i = 3n$

$$\sum_{i=1}^n c_i < 3n \quad (\text{by the previous exercise})$$

$$\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i \quad \text{The total credit is always non-negative}$$

→ **Amortized cost per operation** is  $O(1)$

The total actual cost is  $O(n)$

What is the meaning of the amortized cost '3'? → Dynamic Tables

## Potential Method

각 연산에 의해 자료 구조의 '포텐셜 에너지(potential energy)'를 분석  
accounting method의 **credit**과 유사

각 연산의 **Amortized cost**를 어떻게 할당할까?

i번째 연산의 Amortized cost :  $\hat{c}_i$

$\Phi(D_i)$ : i번째 연산 후의 자료 구조 상태  $D_i$ 에 대한 포텐셜 값.

$\Phi(D_{i-1})$ : i번째 연산 전의 자료 구조 상태  $D_{i-1}$ 에 대한 포텐셜 값.

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) \rightarrow \text{Actual cost} + \text{the changed potential due to the operation}$$

□  $D_0$ : Initial data structure

$\Phi(D_i) - \Phi(D_{i-1}) > 0$ : overcharge to the i-th operation

□  $D_i$ : Data structure after i-th operation

$\Phi(D_i) - \Phi(D_{i-1}) < 0$ : undercharge to the i-th operation

□  $\Phi(D_i)$ : Potential associated with data structure  $D_i$

- $\Phi(D_i) - \Phi(D_{i-1}) > 0$  (포텐셜 증가): 이 경우, 실제 비용( $c_i$ )에 \*\*추가 요금(overcharge)\*\*이 부과되어 상환 비용( $\hat{c}_i$ )이 실제 비용보다 높아집니다. 이 추가 요금이 자료 구조에 "포텐셜 에너지"로 축적됩니다. 이는 마치 회계 분석에서 크레딧을 적립하는 것과 유사합니다. 자료 구조가 "정리되지 않은" 상태로 가거나, 미래의 비싼 연산을 위해 "에너지를 축적"하는 상황으로 볼 수 있습니다.
- $\Phi(D_i) - \Phi(D_{i-1}) < 0$  (포텐셜 감소): 이 경우, 실제 비용( $c_i$ )에서 \*\*할인(undercharge)\*\*이 적용되어 상환 비용( $\hat{c}_i$ )이 실제 비용보다 낮아집니다. 이 할인은 이전에 축적된 포텐셜 에너지를 사용하여 비용을 지불하는 것입니다. 이는 회계 분석에서 크레딧을 사용하는 것과 유사합니다. 자료 구조가 "정리되는" 상태로 가거나, 축적된 에너지를 소모하여 비싼 연산을 수행하는 상황으로 볼 수 있습니다.

■ Total amortized cost:

$$\begin{aligned} \sum_{i=1}^n \hat{c}_i &= \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1})) \\ &= \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \end{aligned}$$

Potential 함수 를 어떻게 정의해야 하는가?

일련의 연산에 대한 총 Amortized 비용이 총 실제 비용의 Upper Bound를 제공해야 한다.

$$\begin{aligned} \sum_{i=1}^n \hat{c}_i &\geq \sum_{i=1}^n c_i \\ \sum_{i=1}^n \hat{c}_i &= \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1})) \\ &= \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \rightarrow \Phi(D_n) \geq \Phi(D_0) \end{aligned}$$

$\Phi(\quad) = ?$ 의 실제 정확한 연산의 수를 아나요??

>> NO, 보통 알고리즘을 분석할 때 미래에 몇 번의 연산이 더 수행될지 정확히 알 수 없습니다.

따라서, 우리는 모든  $i$ 에 대해  $\Phi(D_i) \geq \Phi(D_0)$ 가 되도록 정의해야 합니다.)

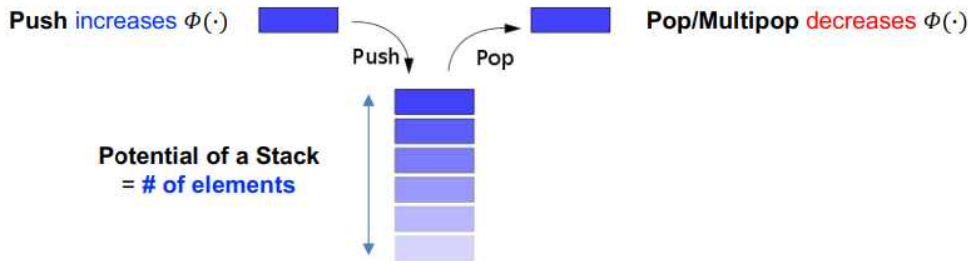
• 더 구체적으로는,

- $\Phi(D_0) = 0$  (초기 자료 구조의 포텐셜은 0으로 설정합니다. 이는 기준점 역할을 합니다.)
- $\Phi(D_i) \geq 0$  for all  $i$  (모든 중간 상태  $D_i$ 의 포텐셜도 항상 0보다 크거나 같도록 정의합니다.)



( )는 스택에 있는 원소의 개수로 정의할 수 있다.

- $\Phi(D_0) = 0$ :  $D_0$ 는 빈 스택을 나타냅니다. 빈 스택에는 원소가 없으므로 포텐셜은 0입니다. 이 조건이 만족됩니다.
- $\Phi(D_i) \geq 0$ :  $D_i$ 는  $i$ 번째 연산 후의 스택을 나타냅니다. 스택에 있는 원소의 개수는 항상 0 이상이므로, 이 조건도 만족됩니다.



When we push elements into the stack, we need to save **potential enough for Pop and Multipop operations**

Amortized Cost :  $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$  (실제 비용 + 포텐셜 변화량)

### MULTIPOP Stack : Potential Method

#### ■ $Push(S, x)$ : $2 = 1 + 1$

- Actual cost  $c_i = 1$
- The changed potential  $\Phi(D_i) - \Phi(D_{i-1}) = (s + 1) - s = 1$

#### ■ $Pop(S)$ : $0 = 1 - 1$

- Actual cost  $c_i = 1$
- The changed potential  $\Phi(D_i) - \Phi(D_{i-1}) = (s - 1) - s = -1$

#### ■ $Multipop(S, k)$ : $0 = k - k$

- Actual cost  $c_i = k$  ( $1 \leq k \leq n$ )
- The changed potential  $\Phi(D_i) - \Phi(D_{i-1}) = (s - k) - s = -k$

#### 1. $Push(S, x)$ 연산의 상환 비용:

- 실제 비용 ( $c_i$ ): 1
  - 원소를 스택에 하나 넣는 데 드는 실제 비용입니다.
- 포텐셜 변화량 ( $\Phi(D_i) - \Phi(D_{i-1})$ ):
  - 연산 전 스택 원소 수:  $s$
  - 연산 후 스택 원소 수:  $s + 1$  (하나의 원소를 Push 했으므로)
  - 변화량:  $(s + 1) - s = 1$
  - 포텐셜이 1 증가했습니다. 이는 미래에 Pop 할 비용을 미리 적립
- 상환 비용 ( $\hat{c}_i$ ):
  - $\hat{c}_i = c_i + (\text{포텐셜 변화량})$
  - $\hat{c}_i = 1 + 1 = 2$

#### 2. $Pop(S)$ 연산의 상환 비용:

- 실제 비용 ( $c_i$ ): 1
  - 원소를 스택에서 하나 빼는 데 드는 실제 비용입니다.
- 포텐셜 변화량 ( $\Phi(D_i) - \Phi(D_{i-1})$ ):
  - 연산 전 스택 원소 수:  $s$
  - 연산 후 스택 원소 수:  $s - 1$  (하나의 원소를 Pop 했으므로)
  - 변화량:  $(s - 1) - s = -1$
  - 포텐셜이 1 감소했습니다. 이는 이전에 Push 할 때 적립해둔 포텐셜(에너지)을 사용하는 것입니다.
- 상환 비용 ( $\hat{c}_i$ ):
  - $\hat{c}_i = c_i + (\text{포텐셜 변화량})$
  - $\hat{c}_i = 1 + (-1) = 0$

#### 3. $Multipop(S, k)$ 연산의 상환 비용:

- 실제 비용 ( $c_i$ ):  $k$ 
  - $k$  개의 원소를 스택에서 빼는 데 드는 실제 비용입니다.
- 포텐셜 변화량 ( $\Phi(D_i) - \Phi(D_{i-1})$ ):
  - 연산 전 스택 원소 수:  $s$
  - 연산 후 스택 원소 수:  $s - k$  ( $k$ 개의 원소를 Multipop 했으므로)
  - 변화량:  $(s - k) - s = -k$
  - 포텐셜이  $k$  감소했습니다. 이는 이전에 Push 할 때  $k$  개의 원소에 대해 적립해둔 포텐셜을 사용하는 것입니다.
- 상환 비용 ( $\hat{c}_i$ ):
  - $\hat{c}_i = c_i + (\text{포텐셜 변화량})$
  - $\hat{c}_i = k + (-k) = 0$

>> accounting method와 같은 amortized cost. ----->

The total amortized cost is  $O(n)$   
The amortized cost per operation is  $O(1)$

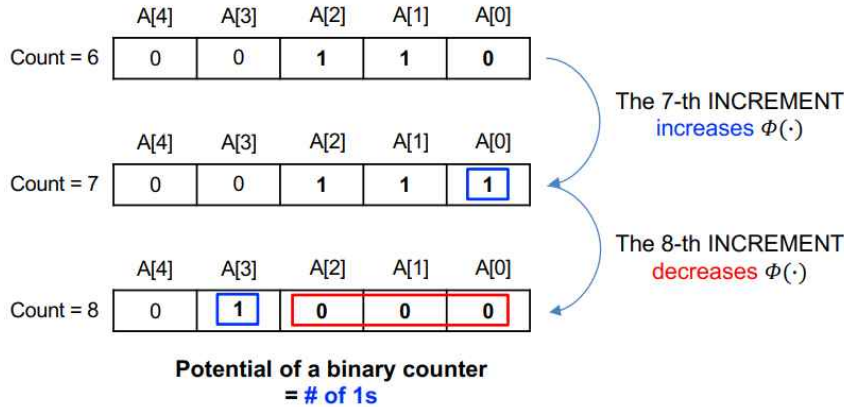


## Binary counter : Potential Method

( )는 카운터에 있는 1의 개수로 정의할 수 있다.

$\Phi(\cdot)$ 는 스택에 있는 원소의 개수로 정의할 수 있다.

- $\Phi(D_0) = 0$ :  $D_0$ 는 카운팅 시작 전의 카운터 (즉, 모든 비트  $A[i]$ 가 0인 상태)를 나타냅니다. 이 상태에서는 1의 개수가 0이므로 포텐셜은 0입니다. 이 조건이 만족됩니다.
- $\Phi(D_i) \geq 0$ :  $D_i$ 는  $i$ 번째 INCREMENT 연산 후의 카운터를 나타냅니다. 카운터에 있는 1의 개수는 항상 0 이상이므로, 이 조건도 만족됩니다.



### Amortized Cost of INCREMENT: $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$

- Suppose that the  $i$ -th INCREMENT operation resets  $t_i$  bits to be 0

- Actual cost  $c_i \leq t_i + 1$  (resetting  $t_i$  bits + setting at most one bit to 1)
- The changed potential  $\Phi(D_i) - \Phi(D_{i-1}) \leq (b_{i-1} - t_i + 1) - b_{i-1} = 1 - t_i$

- Amortized cost  $\hat{c}_i \leq (t_i + 1) + (1 - t_i) = 2$

| Counter | A[3] | A[2] | A[1] | A[0] |
|---------|------|------|------|------|
| 0       | 0    | 0    | 0    | 0    |
| 1       | 0    | 0    | 0    | 1    |
| 2       | 0    | 0    | 1    | 0    |
| 3       | 0    | 0    | 1    | 1    |
| 4       | 0    | 1    | 0    | 0    |
| 5       | 0    | 1    | 0    | 1    |
| 6       | 0    | 1    | 1    | 0    |
| 7       | 0    | 1    | 1    | 1    |
| 8       | 1    | 0    | 0    | 0    |

$c_8 \leq 3 + 1$

$t_8 = 3$

The number of 1 bits after the  $i$ -th operation

$$b_i = b_{i-1} - t_i + 1$$

$$= (\text{\# of 1 bits after the } (i-1)\text{-th operation}) - (\text{\# of bits to be 0}) + (\text{\# of bits to be 1})$$

The same amortized cost in the accounting method

The total amortized cost is  $O(n)$   
The amortized cost per operation is  $O(1)$

실제 변화량

INCREMENT 연산은  $t$  개의 비트를 1에서 0으로 뒤집고, 최대 한 개의 비트를 0에서 1로 뒤집음.

$$c_i = t_i + 1 \text{ (상한 값)}$$

Potential 변화량

$\Phi(D_{i-1})$ : 연산 전 1의 개수:  $b_{i-1}$

- $\Phi(D_i)$  (연산 후 1의 개수): (연산 전 1의 개수) - (0으로 리셋된 1의 개수) + (새로 1이 된 비트의 개수)
- $\Phi(D_i) = b_{i-1} - t_i + 1$
- 포텐셜 변화량:  $\Phi(D_i) - \Phi(D_{i-1}) = (b_{i-1} - t_i + 1) - b_{i-1} = 1 - t_i$

>> Amortized cost  $c_i = (t_i + 1) + (1 - t_i) = 2$

3. 상황 비용 ( $\hat{c}_i$ ): Amortized Cost

- $\hat{c}_i = c_i + (\text{포텐셜 변화량})$
- $\hat{c}_i = (t_i + 1) + (1 - t_i)$
- $\hat{c}_i = 2$

## □ Exercise 3: Potential Method

$$c_i = \begin{cases} i & \text{if } i \text{ is an exact power of 2,} \\ 1 & \text{otherwise.} \end{cases}$$

How do we define the potential function?

$$\Phi(D_i) = 2(i - 2^{\lceil \lg i \rceil}) \Rightarrow \text{*Trick to make the amortized cost be a Constant in the both cases}$$

Operation Cost

|    |   |
|----|---|
| 1  | 1 |
| 2  | 2 |
| 3  | 1 |
| 4  | 4 |
| 5  | 1 |
| 6  | 1 |
| 7  | 1 |
| 8  | 8 |
| 9  | 1 |
| 10 | 1 |
| ⋮  | ⋮ |

$$\text{if } i \neq 2^j, \text{ then } \Phi(D_i) - \Phi(D_{i-1}) = 2(i - 2^{\lceil \lg i \rceil}) - 2((i-1) - 2^{\lceil \lg(i-1) \rceil}) = 2$$

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 1 + 2 = 3$$

$$\text{if } i = 2^j, \text{ then } \Phi(D_i) - \Phi(D_{i-1}) = 2(i - 2^j) - 2((i-1) - 2^{j-1}) = -i + 2$$

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = i - i + 2 = 2$$

→ Amortized cost per operation is  $O(1)$

The total actual cost is  $O(n)$

Practical way to define the potential function  
→ Dynamic Tables

### 동적 테이블

실생활에서 발생할 수 있는 실용적인 문제

우리는 테이블 안에 얼마만큼의 객체가 저장될지 사전에 알 수 없다.

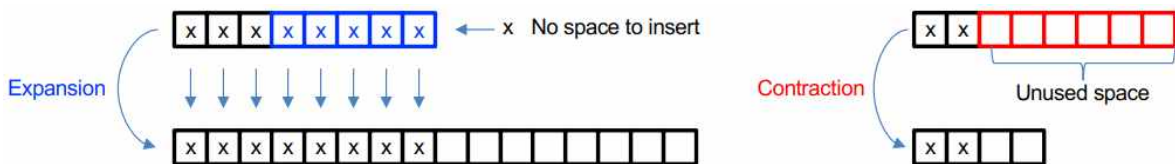
INSERT, DELETE, UPADTE 등의 일련의 연산이 수행된다.

**Table expansion** : INSERT 연산에 의해 테이블이 팽창할 수 있다.

>> 더 큰 메모리 공간으로 재할당을 해서, 객체들을 새 공간으로 옮긴다

**Table contraction** : DELETE 연산에 의해 충분히 작아질 수 있다.

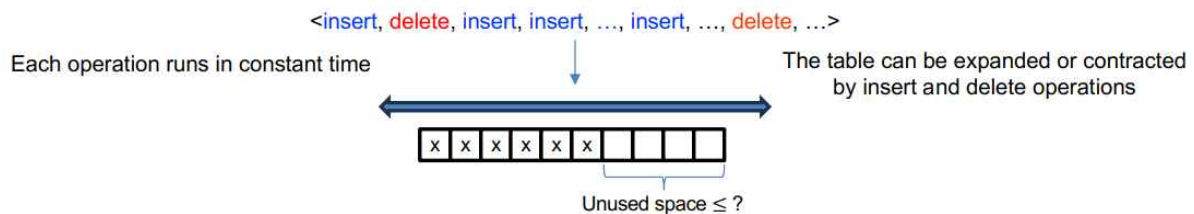
>> 더 작은 메모리 공간으로 재할당을 하는게 좋다.. 공간 절약



어떻게 하면 가능한 효율적으로 사용할 수 있을까?

1. 사용되지 않는 공간  $\leq$  할당된 공간의 상수비율(일정비율) - 낭비되는 공간을 일정량 아래로 하겠다.
2. Insertion과 Deletion의 Amortized cost는  $O(1)$ 이어야 한다.

Expansion + Contraction



>> 이 목표를 달성하기 위해서 어떻게 테이블을 동적으로 제어할까?

### Table Insertion 연산

**팍 찬 table**에 item을 insert하면, 테이블은 확장되어야 한다.

1. 더 많은 용량의 새로운 공간을 할당하고 -> 2. 모든 객체를 새로운 테이블로 옮겨야 한다.

그런데 몇 개를 늘려야 할까? -> 원래의 2배로 늘린다

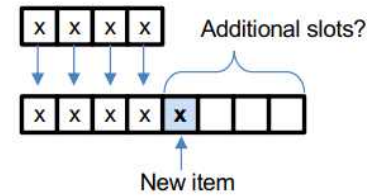
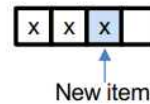
## TABLE-INSERT( $T, x$ )

```

1  if  $T.size == 0$ 
2      allocate  $T.table$  with 1 slot
3       $T.size = 1$ 
4  if  $T.num == T.size$ 
5      allocate  $new-table$  with  $2 \cdot T.size$  slots
6      insert all items in  $T.table$  into  $new-table$ 
7      free  $T.table$ 
8       $T.table = new-table$ 
9       $T.size = 2 \cdot T.size$ 
10 insert  $x$  into  $T.table$ 
11  $T.num = T.num + 1$ 

```

$T.table$ : a pointer to the table  
 $T.num$ : the number of items in the table  
 $T.size$ : the total number of slots in the table

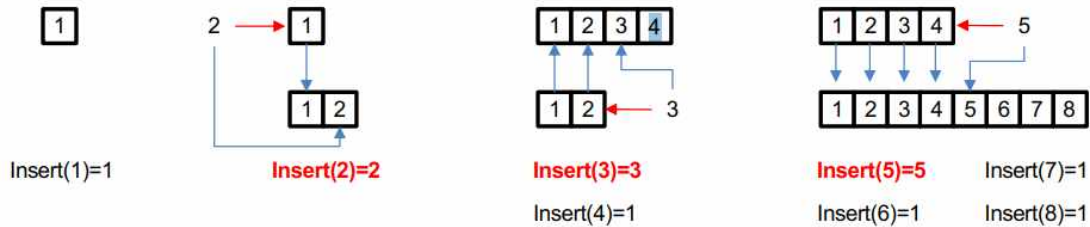


## Table Expansion : Aggregate Analysis

\* 현재 테이블에 새로운 아이템을 넣을 공간이 있는 경우 : =1 (삽입)

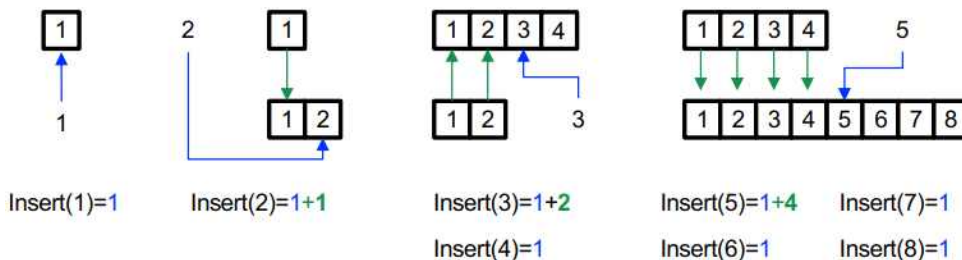
\* 현재 테이블이 꽉 찬 경우 :  $c_i = (i-1) + 1 = i$  (확장 and 삽입)

현재 테이블은  $(i-1)$ 개만큼의 아이템이 있고, 그만큼을 copy해야되기 때문에  $i-1$ 만큼이 듭.  
 $i$ -th 아이템은 새로운 테이블에 넣기.



Can we represent the actual cost  $c_i$ ?

>>네. 실제 cost  $c_i$ 를 나타낼 수 있음.



$$\begin{aligned}
 \text{Total cost} &= \sum_{i=1}^n c_i \\
 &\leq n + \sum_{j=0}^{\lfloor \lg n \rfloor} 2^j \\
 &= n + \frac{2^{\lfloor \lg n \rfloor + 1} - 1}{2 - 1} \\
 &< n + 2n \\
 &= 3n
 \end{aligned}$$

Insert the new item

Copying existing items

→ Amortized cost per operation is  $O(n)/n = O(1)$

## Table Expansion : Accounting Analysis

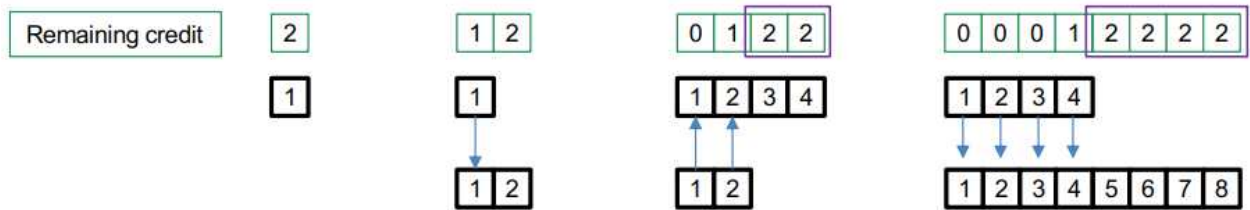
TABLE-INSERT 연산의 상환 비용이 왜 3이어야 하는지 분석해보자 -> Exercise 2 참고

TABLE-INSERT 연산에 3\$를 부과

1\$ for 새로운 아이템 삽입

1\$ for 향후 발사될 새 항목에 대한 비용(확장이 될 때 필요할 비용)

1\$ for 향후 복사될 다른 기존 항목에 대한 비용(확장 발생 시 필요)



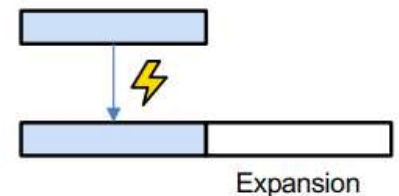
We always have **credits** enough for copying all existing items!

→ **Amortized cost per operation** is  $O(3n)/n = O(1)$

### Table Expansion : Potential Analysis

포텐셜 함수 에 대한 직관

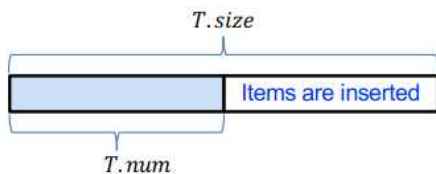
- \* 다음 확장을 하기 위해서 충분한 양의 potential을 저장해놔야 한다.
- \* 확장 이후에, potential이 즉시 0이 된다.



그럼 함수  $\Phi$ 를 어떻게 정의할까?

■  $\Phi(T) = 2 \cdot T.num - T.size$

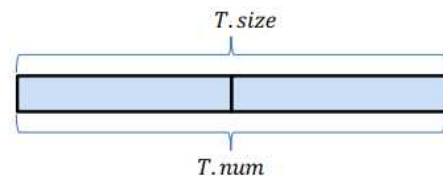
- $\Phi(T) \geq 0$  since the table is always at least half full ( $T.num - T.size/2$ )



**AFTER** an expansion:  
 $\Phi(T) = 2 \cdot T.num - T.size = 0$



Save the potential  
For the next expansion



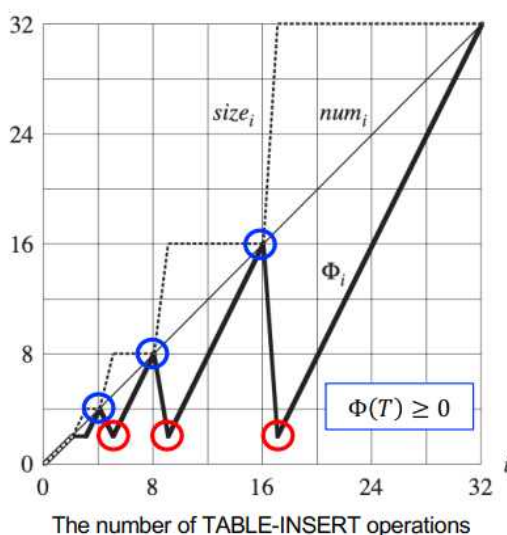
**BEFORE** an expansion:  
 $\Phi(T) = 2 \cdot T.num - T.size = T.num$

Ready to pay **the cost of copying all items ( $T.num$ )** to the new table

$\Phi(T) \geq 0$  인 이유: 테이블은 항상 절반 이상 채워져 있기 때문에, 0보다 크다!

BEFORE 상태에서는 모든 아이템을 Copy하는데 쓸 cost를 저장하고 있다!

### □ The Potential Energy w.r.t the Number of INSERT Operations



**BEFORE** an expansion:  
 $\Phi(T) = 2 \cdot T.num - T.size = T.num$

Ready to pay for the cost of copying all items ( $T.num$ )

**AFTER** an expansion:  
 $\Phi(T) = 2 \cdot T.num - T.size = 0$

Consuming all potential for copying all items

### i번째 TABLE-INSERT 연산에서의 Amortized Cost

Case 1: expansion을 하지 않는 경우

- $size_i = size_{i-1}$   
 □  $c_i = 1$

$$\begin{aligned} \hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (2 \cdot num_i - size_i) - (2 \cdot num_{i-1} - size_{i-1}) \\ &= 1 + (2 \cdot num_i - size_i) - (2(num_i - 1) - size_i) \\ &= 3. \end{aligned}$$

Case 1



## Case 2: expansion을 하는 경우

$$\square \text{ size}_i = 2 \cdot \text{size}_{i-1}$$

$$\square \text{ size}_{i-1} = \text{num}_{i-1} = \text{num}_i - 1$$

$$\square c_i = \text{num}_i$$

$$\hat{c}_i = c_i + \Phi_i - \Phi_{i-1}$$

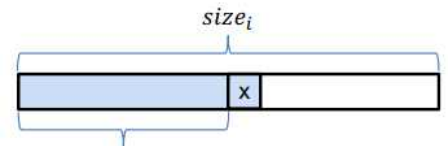
$$= \text{num}_i + (2 \cdot \text{num}_i - \text{size}_i) - (2 \cdot \text{num}_{i-1} - \text{size}_{i-1})$$

$$= \text{num}_i + (2 \cdot \text{num}_i - 2 \cdot (\text{num}_i - 1)) - (2(\text{num}_i - 1) - (\text{num}_i - 1))$$

$$= \text{num}_i + 2 - (\text{num}_i - 1)$$

$$= 3.$$

## Case 2



>> Amortized cost per operation is 3 => O(1)

## TABLE-DELETE 연산

안쓰는 slot이 많이 있으면 수축(contract)해야 한다.

1. 더 작은 사이즈의 테이블을 재할당하고
2. 원래 테이블의 데이터를 복사한다.

Load Factor  $\alpha = T.\text{num}/T.\text{size}$

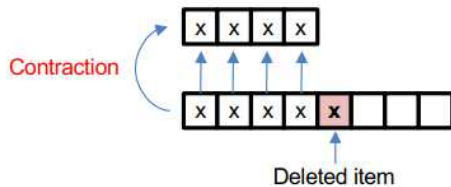
<- 언제 수축할지 정하는 기준.

$\alpha = 1$ : 테이블이 가득 찼습니다 (the table is full).

$\alpha = 1/2$ : 테이블이 절반만 찼습니다 (the table is half-full).

로트 팩터를 기준으로 언제 수축할지 정한다.

Copying all items into the new smaller table



We would like to preserve two properties:

- (1)  $\alpha$  is bounded below by a constant and
- (2) the amortized cost per operation is  $O(1)$  연산 비용이  $O(1)$ 이어야 함

## Naive한 접근

If  $\alpha \geq 1 \rightarrow$  the table is full, **expanding the table** with double size

If  $\alpha < 1/2 \rightarrow$  the table is less than  $1/2$  full, **contracting** the table with half size

이 전략의 한계 : 경계에서 계속 늘렸다 줄였다 하면 비효율성이 높아짐.

## Naive한 접근의 개선

If  $\alpha \geq 1 \rightarrow$  the table is full, expanding the table with double size (same as before)

If  $\alpha < 1/4 \rightarrow$  the table is less than  $1/4$  full, contracting the table with half size

확장이나 축소 이후에는 항상  $\alpha$ 가  $1/2$ 이 된다.

$\alpha$ 는 항상  $1/4 \leq \alpha \leq 1$ 이다.

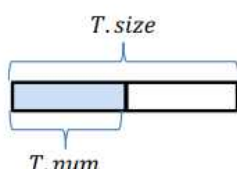
## 포텐셜 함수에 대한 직관.

1. 다음 확장을 위한 포텐셜 저장.

2. 다음 축소를 위한 포텐셜 저장

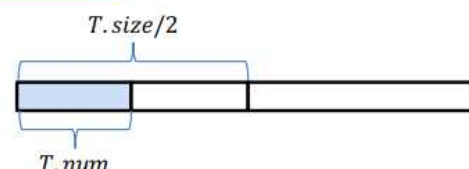
>> 포텐셜은 축소나 확장 이후 0으로 즉시 줄어든다.

$$\Phi(T) = \begin{cases} 2 \cdot T.\text{num} - T.\text{size} & \text{if } \alpha \geq 1/2, \\ T.\text{size}/2 - T.\text{num} & \text{if } \alpha < 1/2. \end{cases}$$



Immediately AFTER a contraction:

$$\Phi(T) = T.\text{size}/2 - T.\text{num} = 0$$

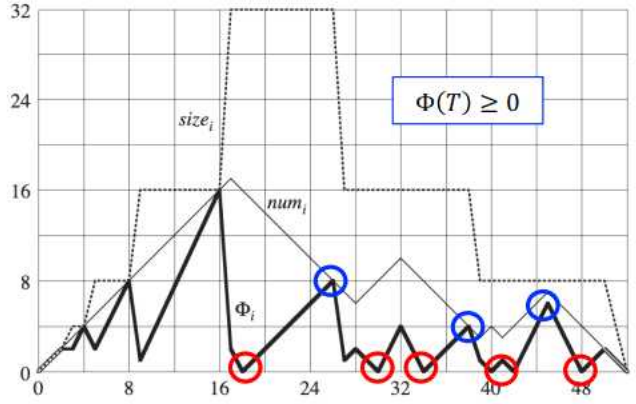


Immediately BEFORE a contraction:

$$\Phi(T) = T.\text{size}/2 - T.\text{num} = T.\text{num}$$

Ready to pay the cost of copying all items ( $T.\text{num}$ ) to the new table

## □ The Potential Energy w.r.t the Number of Operations



The number of INSERT and DELETE operations

Immediately **BEFORE** a contraction:  
 $\Phi(T) = T.size/2 - T.num = T.num$

Ready to pay for the cost of copying all items ( $T.num$ )

Immediately **AFTER** a contraction:  
 $\Phi(T) = T.size/2 - T.num = 0$

Consuming all potential for copying all items

## □ AMORTIZED COST OF THE $i$ -th TABLE-INSERT Operation

■ **Case 1:**  $\alpha_{i-1} \geq 1/2$  (the amortized cost is **at most 3** same as before)

■ **Case 2:**  $\alpha_{i-1} < 1/2$  (the table does not expand)

□  $size_i = size_{i-1}$

□  $num_{i-1} = num_i - 1$

$$\Phi(T) = \begin{cases} 2 \cdot T.num - T.size & \text{if } \alpha \geq 1/2, \\ T.size/2 - T.num & \text{if } \alpha < 1/2. \end{cases}$$

■ **Case 3:**  $\alpha_{i-1} < 1/2$ , but  $\alpha_i \geq 1/2$  (i.e., at the half line)

**Case 2**

$$\begin{aligned} \hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (size_i/2 - num_i) - (size_{i-1}/2 - num_{i-1}) \\ &= 1 + (size_i/2 - num_i) - (size_i/2 - (num_i - 1)) \\ &= \boxed{0}. \end{aligned}$$

**Case 3**

$$\begin{aligned} \hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (2 \cdot num_i - size_i) - (size_{i-1}/2 - num_{i-1}) \\ &= 1 + (2(num_{i-1} + 1) - size_{i-1}) - (size_{i-1}/2 - num_{i-1}) \\ &= 3 \cdot num_{i-1} - \frac{3}{2} size_{i-1} + 3 \\ &= 3\alpha_{i-1} size_{i-1} - \frac{3}{2} size_{i-1} + 3 \\ &< \frac{3}{2} size_{i-1} - \frac{3}{2} size_{i-1} + 3 \\ &= \boxed{3}. \end{aligned}$$

→ **Amortized cost per INSERT** is at most 3,  $O(1)$

## □ Amortized Cost of the $i$ -th TABLE-DELETE Operation

■ **Case 1:**  $\alpha_{i-1} < 1/2$  and does not trigger a contraction

□  $size_i = size_{i-1}$

■ **Case 2:**  $\alpha_{i-1} < 1/2$  and does trigger a contraction

$$\Phi(T) = \begin{cases} 2 \cdot T.num - T.size & \text{if } \alpha \geq 1/2, \\ T.size/2 - T.num & \text{if } \alpha < 1/2. \end{cases}$$

**Case 1**

$$\begin{aligned} \hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (size_i/2 - num_i) - (size_{i-1}/2 - num_{i-1}) \\ &= 1 + (size_i/2 - num_i) - (size_i/2 - (num_i + 1)) \\ &= \boxed{2}. \end{aligned}$$

**Case 2**

$$\begin{aligned} \hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= (num_i + 1) + (size_i/2 - num_i) - (size_{i-1}/2 - num_{i-1}) \\ &= (num_i + 1) + ((num_i + 1) - num_i) - ((2 \cdot num_i + 2) - (num_i + 1)) \\ &= \boxed{1}. \end{aligned}$$

→ **Amortized cost per DELETE** is bounded above by a constant.  $O(1)$

## 12주차 - Graph Algorithm

## Graph란?

개체(entities)와 그것들의 관례를 나타낸 것.

**사회네트워크 분석 : 영향력 최대화 - 바이럴 마케팅, 감염 통제**

**디지털 헬스케어 : 감염병 전달경로 탐색, 약물 부작용 예측**

### 추천 알고리즘 : 사용자의 관심을 추천

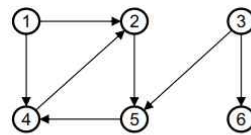
**사회 복지** : 가짜뉴스 감지, 메아리방 효과 완화

검색 증강 생성 : LLM 기술에 사용

## Graph 용어

**V : vertex(정점) 집합** - 한 객체를 나타냄

**E : edge(간선) 집합** - 여러 vertex간의 관계를 나타냄



### Directed graph

$$V = \{1, 2, 3, 4, 5, 6\}$$
$$E = \{(1,2), (1,4), (2,5), (3,5), (3,6), (4,2), (5,4)\},$$

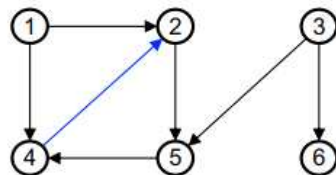
**Directed Graph** : Edge가 방향성이 있는 경우

### Undirected Graph : Edge가 방향성이 없는 경우

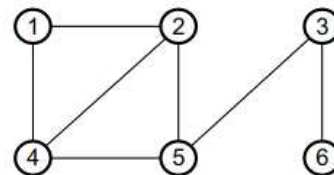
- If  $(u, v)$  is an edge, vertex  $u$  is **adjacent** to vertex  $v$  and vertex  $v$  is **adjacent** to vertex  $u$ 
  - e.g., Vertex 4 is adjacent to vertex 2, and Vertex 2 is also adjacent to vertex 4 (**symmetric**)
 

$(4,2) \neq (2,4)$

$(4,2) = (2,4)$



## Directed graph



### Undirected graph

## Directed Graph에서의 Degree

노드의 In-degree : 들어오는 edge의 개수

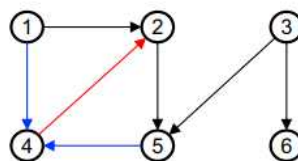
노드의 Out-degree : 나가는 edge의 개수

ex) vertex 4에서의 In-degree는 2, out-degree는 1이다.

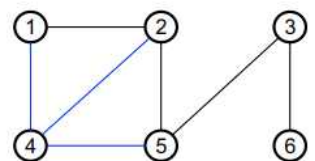
## Undirected Graph에서의 Degree

노드의 degree : edge의 개수

ex) vertex 4에서는 3이다.



### Directed graph



## Undirected graph

u vertex부터 v vertex까지의 경로

=u,  $v_k=v$ 라고 할 때 **vertices**의 시퀀스  $\langle v_0, v_1, \dots, v_k \rangle$ 를 **경로(path)**라고 한다.

**path의 길이는** path안의 edge의 갯수이다.

모든 vertex  $v_{i+1}$ 은  $v_i$ 에 인접하다(adjacent)

vertex  $u$ 에서 vertex  $v$ 까지의 경로가 존재한다면,  $v$ 는  $u$ 까지 **reachable**하다!

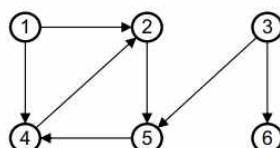
**Simple Path** : 경로 내의 모든 vertices이 서로 다른 경우. 한번 지나간 Vertex를 다시 거치지 않음

$\langle 1, 2, 5, 4 \rangle$  is a path

$\langle 1, 2, 4 \rangle$  is not a path

$\langle 1, 2, 5, 4 \rangle$  is a simple path

$\langle 1, 2, 5, 4, 2 \rangle$  is not a simple path



### Directed graph

Vertex 4 is reachable from vertices 1, 2, and 3

Vertex 3 is not reachable from vertex 1

**cycle** :  $v, v_1, \dots, v_k >$  일 때  $v_0 = v_k$  인 경우

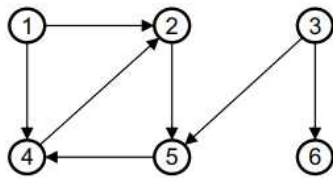
-하나 이상의 사이클을 포함할 경우

**Simple cycle** :  $v_0$ 을 제외한 사이클 내의 모든 vertices가 서로 달라야 한다.

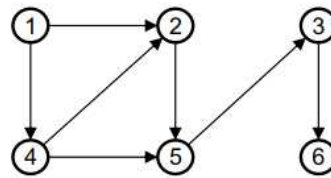
**acycle** : 사이클이 없는 그래프.

e.g., the path  $\langle 2, 5, 4, 2 \rangle$  is a simple cycle, but the path  $\langle 2, 5, 4, 2, 5, 4, 2 \rangle$  is **not a simple cycle**

>> 2, 5, 4가 여러번 반복되기 때문



Directed graph



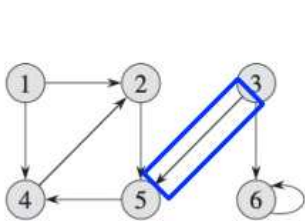
Directed acyclic graph

## Graph의 표현

### Adjacency-Matrix 표현

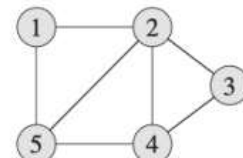
vertices  $i, j$  사이에 edge 가 있으면  $A[i][j]$ 가 1인  $|V| \times |V|$ 인 행렬 **A**

**undirected** graph는 **symmetry**(대칭)이다. 주대각(main diagonal)을 따라서 대칭/



Directed graph

|   | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 1 | 0 | 0 |
| 2 | 0 | 0 | 0 | 0 | 1 | 0 |
| 3 | 0 | 0 | 0 | 0 | 1 | 1 |
| 4 | 0 | 1 | 0 | 0 | 0 | 0 |
| 5 | 0 | 0 | 0 | 1 | 0 | 0 |
| 6 | 0 | 0 | 0 | 0 | 0 | 1 |



Undirected graph

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 0 | 1 |
| 2 | 1 | 0 | 1 | 1 | 1 |
| 3 | 0 | 1 | 0 | 1 | 0 |
| 4 | 0 | 1 | 1 | 0 | 1 |
| 5 | 1 | 1 | 0 | 1 | 0 |

공간 복잡도 :  $O(|V|^2)$

--- 존재하지 않는 edge에 대해서는 0으로 저장되어 있음!

시간 복잡도 : 어떤 edge  $A[i][j]$ 가 존재하는가?  $\rightarrow O(1)$

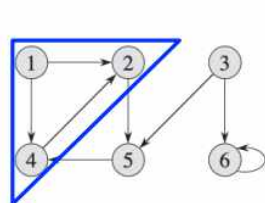
이웃 찾기 :  $O(|V|)$

모든 edge 찾기 :  $O(|V|^2)$

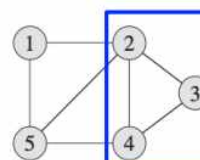
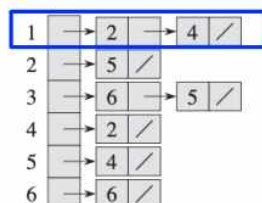
### Adjacency-List 표현

각 Vertex마다 하나의 리스트를 가짐. 총  $|V|$ 개의 리스트 배열

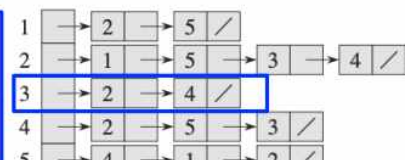
각 정점  $u$ 의 인접 리스트는  $u$ 에 인접한 모든 정점을 포함.



Directed graph



Undirected graph



공간 복잡도 :  $O(|V| + |E|)$

adjacency-matrix와 달리 존재하는 edge만 저장되어 있음

시간 복잡도 : 어떤 edge  $A[i][j]$ 가 존재하는가?  $\rightarrow O(\text{vertex } i \text{의 degree})$

이웃 찾기 :  $O(\text{vertex } i \text{의 degree})$

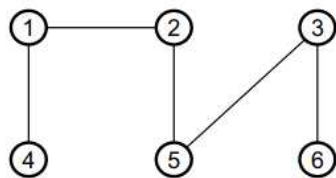
모든 edge 찾기 :  $O(|E|)$

>> 둘 중에 어떻게 나올까? Sparse  $\rightarrow$  Adjacency-List

Dense  $\rightarrow$  Adjacency-Matrix

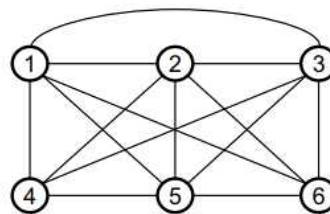


|                  | Time Complexity |                 |                 | Space Complexity |
|------------------|-----------------|-----------------|-----------------|------------------|
|                  | Edge present    | Visit neighbors | Visit all edges |                  |
| Adjacency-Matrix | $O(1)$          | $O( V )$        | $O( V )$        | $O( V ^2)$       |
| Adjacency-List   | $O(deg)$        | $O(deg)$        | $O( V  +  E )$  | $O( V  +  E )$   |



$$|E| \ll |V|^2$$

Sparse graph



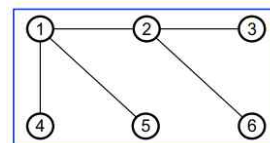
$$|E| \approx |V|^2$$

Dense graph

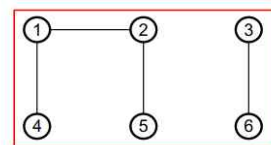
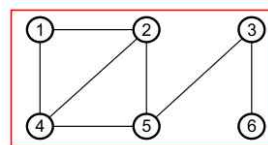
## Graph vs Tree

모든 트리는 그래프이다. 모든 그래프가 트리인 것은 아니다

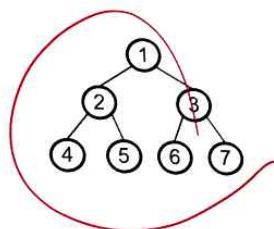
All Trees Are Graphs, but Not All Graphs Are Trees



: 트리는 cycle이 없고 모든 vertices가 연결되어 있는 그래프다.

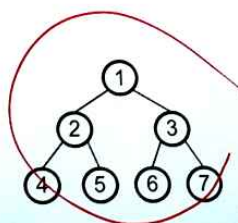


## Exercise 1: Adjacency-List Representation



```
1: 2,3
2: 1,4,5
3: 1,6,7
4: 2
5: 2
6: 3
7: 3.
```

## Exercise 2: Adjacency-Matrix Representation



$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

## Graph Searching

왜 그래프 서칭이 중요할까?

특정한 vertex를 그래프 안에서 찾기

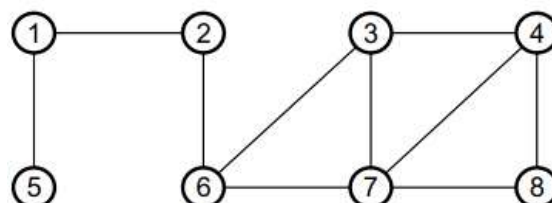
vertex u로부터 vertex v에 reachable한가?

vertex u부터 vertex v까지 가장 최단 경로는 무엇인가?

기초적인 알고리즘

Breath-first search(BFS)

Depth-first search(DFS)



## Breath-first Search(BFS)

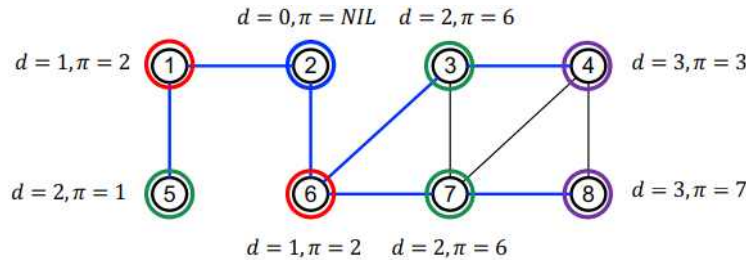
$G=(V,E)$  와 source(시작) vertex  $s$ 가 주어졌을 때,  $s$ 로부터 **reachable**한 모든 **vertex**를 탐색하는 알고리즘.

**거리(distance)** : 두 vertex  $u \rightarrow v$ 까지의 **최단 경로에 있는 edge의 개수**를 의미함. (가중치가 없을 때)

BFS는  $s$ 로부터의 **거리가 증가하는 순서로 vertices**를 탐색함.

1. **d(distance)** :  $d$ 로 표시.  $u \rightarrow v$ 까지의 최단 경로에 있는 edge 수의 합.

2. **predecessor(선행자)** :  $\pi$ 로 표시. 해당 vertex를 발견하게 한 직전의 vertex를 의미. 최단 경로를 역추적 가능.



vertex 2가  $d=0, \pi=NIL$ 이므로 시작. 빨간색이  $d=1$ , 초록색이  $d=2$ , 보라색이  $d=3$

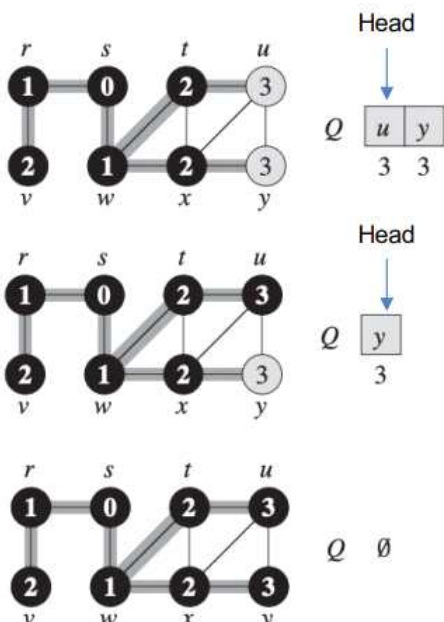
## □ Procedure of BFS

$BFS(G, s)$

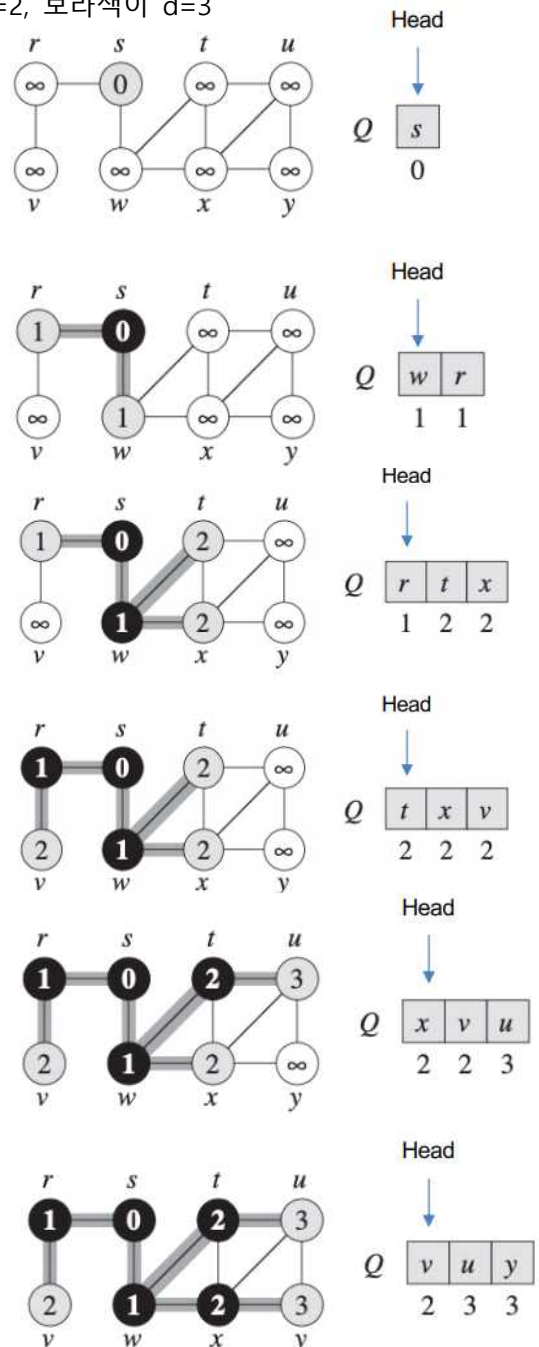
```

1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4     $u.\pi = NIL$ 
5   $s.color = GRAY$ 
6   $s.d = 0$ 
7   $s.\pi = NIL$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11    $u = DEQUEUE(Q)$ 
12   for each  $v \in G.Adj[u]$ 
13     if  $v.color == WHITE$ 
14        $v.color = GRAY$ 
15        $v.d = u.d + 1$ 
16        $v.\pi = u$ 
17       ENQUEUE( $Q, v$ )
18    $u.color = BLACK$ 
  
```

Initialization



<-----



## BFS의 복잡도

### 시간복잡도

Initialization(초기화) :  $O(|V|)$

그래프 탐색 :  $O(|V| + |E|)$

모든 vertex는 Queue에 최대 한번 enqueue됨

각 인접 리스트는 최대 한번 스캔된다. > b.o. 어떤 vertex를 dequeue한 후 u에 연결된 모든

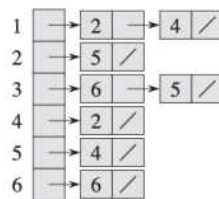
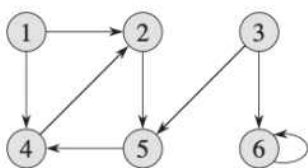
이웃을 확인하기 위해서 u의 adjacency list를 한번만 완전히 순회하기 때문.

## Time Complexity of BFS

■ Initialization:  $O(|V|)$

■ Exploring a graph:  $O(|V| + |E|)$

- Each vertex is enqueued (i.e., dequeued) at most once
- Each adjacency list is scanned at most once (→ Why?)



BFS( $G, s$ )

```

1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = WHITE$ 
3       $u.d = \infty$ 
4       $u.\pi = NIL$ 
5   $s.color = GRAY$ 
6   $s.d = 0$ 
7   $s.\pi = NIL$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = DEQUEUE(Q)$ 
12     for each  $v \in G.Adj[u]$ 
13         if  $v.color == WHITE$ 
14              $v.color = GRAY$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = BLACK$ 
    
```

$$V_\pi = \{v \in V : v.\pi \neq NIL\} \cup \{s\}$$

(선행자)가 NIL이 아닌 정점들. + 시작 정점  $s$

이 말은  $v$ 가 이미 BFS에 의해 방문되었다는 뜻.

$s$ 를 따로 추가해주는 이유는

- $s$ 는 어떤 선행자에도 포함이 안 되기 때문

>> 시작 vertex  $s$ 로부터 도달 가능한 모든 vertices

$$E_\pi = \{(v.\pi, v) : v \in V_\pi \setminus \{s\}\}$$

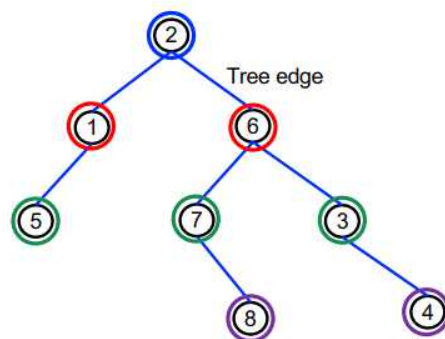
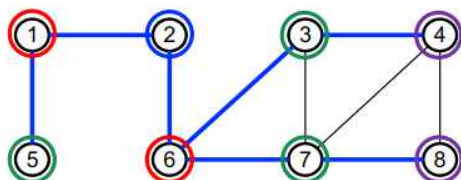
트리 에지의 집합

■  $V_\pi = \{v \in V : v.\pi \neq NIL\} \cup \{s\}$

- The vertices reachable from the source  $s$

■  $E_\pi = \{(v.\pi, v) : v \in V_\pi \setminus \{s\}\}$

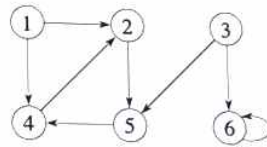
- A set of **tree edges** ( $|E_\pi| = |V_\pi| - 1$ )



Unique **simple** (also **shortest**) path from  $s$  to  $v$



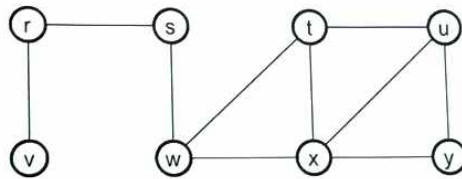
### Exercise 3: BFS on a Directed Graph



| vertex | d        | $\pi$ |
|--------|----------|-------|
| 1      | $\infty$ | NIL   |
| 2      | 3        | 4     |
| 3      | 0        | NIL   |
| 4      | 2        | 5     |
| 5      | 1        | 3     |
| 6      | 1        | 3     |

- Show the  $d$  (distance) and  $\pi$  (predecessor) values that result from running BFS on the following directed graph, using vertex 3 as the source

### Exercise 4: BFS on an Undirected Graph

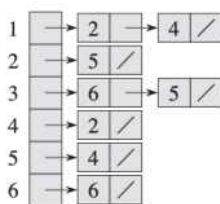
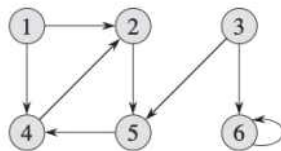


| Vertex | d | $\pi$ |
|--------|---|-------|
| r      | 4 | s     |
| s      | 3 | w     |
| t      | 1 | u     |
| u      | 0 | NIL   |
| v      | 5 | r     |
| w      | 2 | t     |
| x      | 1 | u     |
| y      | 1 | u     |

- Show the  $d$  (distance) and  $\pi$  (predecessor) values that result from running BFS on the following undirected graph, using vertex u as the source

### Exercise 5: Complexity of BFS in the Matrix Representation

- What is the running time of BFS if we represent its input graph by an adjacency matrix and modify the algorithm to handle this form of input?



|   | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 1 | 0 | 0 |
| 2 | 0 | 0 | 0 | 0 | 1 | 0 |
| 3 | 0 | 0 | 0 | 0 | 1 | 1 |
| 4 | 0 | 1 | 0 | 0 | 0 | 0 |
| 5 | 0 | 0 | 0 | 1 | 0 | 0 |
| 6 | 0 | 0 | 0 | 0 | 0 | 1 |

BFS( $G, s$ )

```

1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4     $u.\pi = NIL$ 
5   $s.color = GRAY$ 
6   $s.d = 0$ 
7   $s.\pi = NIL$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11    $u = DEQUEUE(Q)$ 
12   for each  $v \in G.Adj[u]$ 
13     if  $v.color == WHITE$ 
14        $v.color = GRAY$ 
15        $v.d = u.d + 1$ 
16        $v.\pi = u$ 
17       ENQUEUE( $Q, v$ )
18    $u.color = BLACK$ 

```

```

12: for v = 1 to |V|           // 모든 가능한 정점 v에 대해
13:   if A[u][v] == 1           // u에서 v로 가는 에지가 있다면
14:     if v.color == WHITE // (이후 코드는 동일)
15:       v.color = GRAY
16:       v.d = u.d + 1
17:       v.pi = u
18:       ENQUEUE(Q, v)

```

보시다시피, for each  $v$  in  $G.Adj[u]$  부분이 for  $v = 1$  to  $|V|$  로 변경됩니다. 이 변경 때문에 이웃을 찾는 부분의 시간 복잡도가  $O(\text{degree}(u))$ 에서  $O(|V|)$ 로 바뀌게 되고, 이것이 전체 BFS의 시간 복잡도를  $O(|V| + |E|)$ 에서  $O(|V|^2)$ 로 증가시키는 주된 원인이 됩니다.



## Depth First Search DFS : 설명

DFS는 가능할 때마다 그래프에서 '더 깊이' 탐색해서 들어간다.

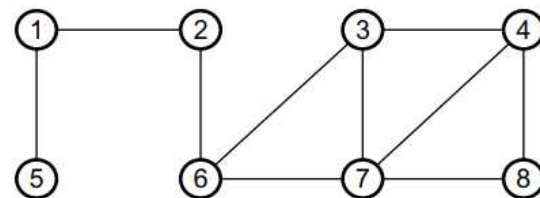
특정 경로에서 최대한 멀리 이동한 다음, 더 이상 갈 곳이 없으면 뒤로 backtracking 하는 재귀 방식 사용

### Each vertex $v$ has two timestamps

- $v.d$ : discovery time (when  $v$  becomes gray)
- $v.f$ : finish time (when  $v$  becomes black)

**v.d** : vertex  $v$ 가 '발견된 시간'

**v.f** : vertex  $v$ 의 탐색이 완료된 시간.



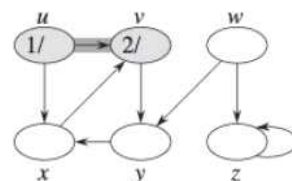
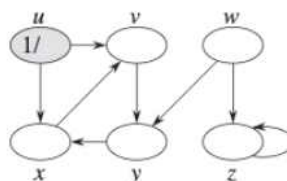
## Procedure of DFS

DFS( $G$ )

```

1 for each vertex  $u \in G.V$ 
2    $u.color = WHITE$ 
3    $u.\pi = NIL$ 
4    $time = 0$ 
5   for each vertex  $u \in G.V$ 
6     if  $u.color == WHITE$ 
7       DFS-VISIT( $G, u$ )
  
```

For every vertex



DFS-VISIT( $G, u$ )

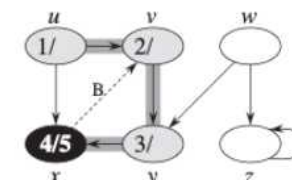
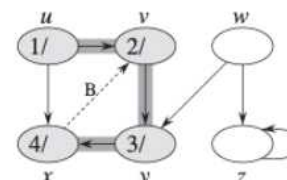
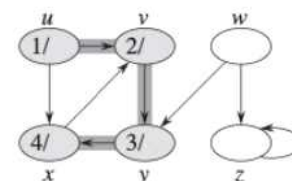
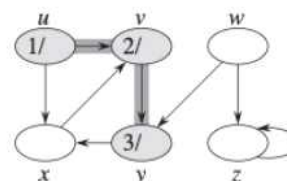
```

1  $time = time + 1$ 
2  $u.d = time$ 
3  $u.color = GRAY$ 
4 for each  $v \in G.Adj[u]$ 
5   if  $v.color == WHITE$ 
6      $v.\pi = u$ 
7     DFS-VISIT( $G, v$ )
8  $u.color = BLACK$ 
9  $time = time + 1$ 
10  $u.f = time$ 
  
```

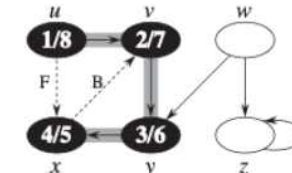
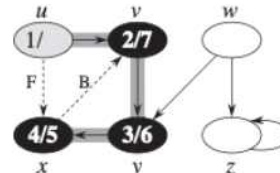
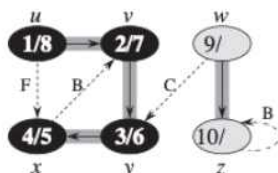
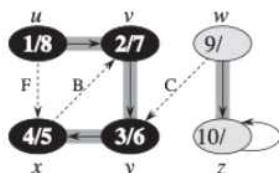
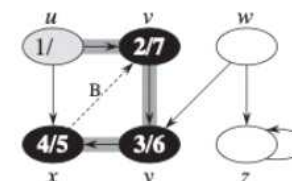
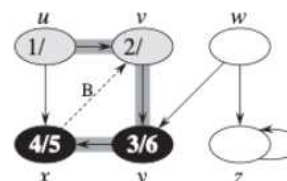
Discovered

Recursively visit  
its 'deeper' neighbors

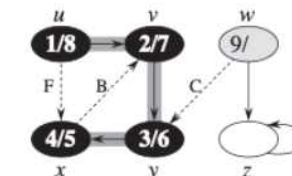
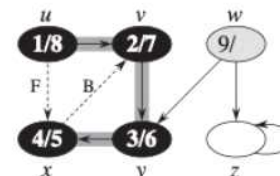
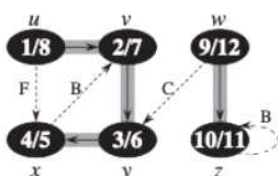
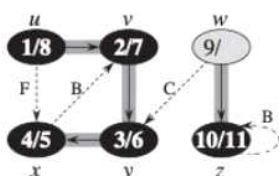
Finished  
After all recursive calls



$N/M == v.d(\text{발견된 시간})/v.f(\text{완료된 시간})$

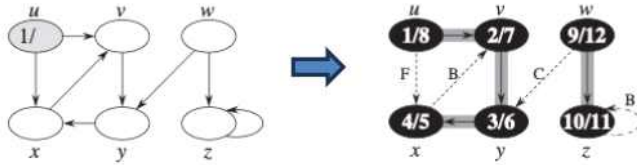


<-----



## Time Complexity of DFS

- Initialization:  $\Theta(|V|)$
- Exploring a graph:  $\Theta(|V| + |E|)$ 
  - ❑ Not  $O(|V| * |E|)$  by the aggregate analysis



```

DFS(G)
1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6    if  $u.color == WHITE$ 
7      DFS-VISIT( $G, u$ )

```

$\Theta(|V|)$  (lines 1-4)  
 $\Theta(|V|)$  (lines 5-7)

```

DFS-VISIT( $G, u$ )
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-VISIT( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 

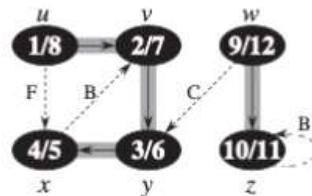
```

$\sum_{v \in V} |Adj[v]| = \Theta(E)$  (lines 4-7)

### DFS의 특성

#### 1. 특성1 : Depth-First Forest

Predecessor subgraph(선행자 서브그래프)는 DFS 과정에서  $u.d = v.d$ 와 같이 설정된 선행자 에지만 모아서 만든 그래프.

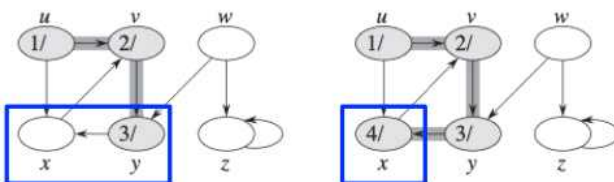


#### 2. 특성2 : Descendant

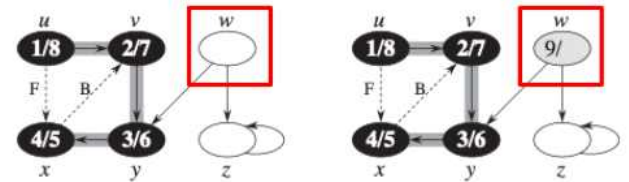
vertex  $v$ 가 vertex  $u$ 의 자손(descendant)인 것은,  $u$ 가 회색인 시간 동안만  $v$ 가 발견되는 경우이다

>>  $u$ 가 회색인동안  $v$ 가 발견된다는 것은,  $v$ 가  $u$ 의 재귀호출 스택에 있다는 것.

>> 즉,  $v$ 는  $u$ 로부터 시작된 탐색 경로의 일부이고, DFS트리에서  $u$ 의 자손이 됨.



Vertex  $x$  is a descendant of vertex  $y$



Vertex  $w$  is not a descendant of vertex  $y$

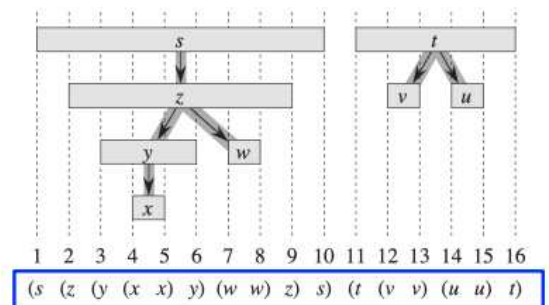
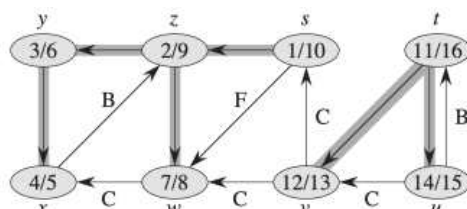
#### 3. 특성3 : 괄호 구조 (Parenthesis Structure)

그래프의 모든 DFS에서, 임의의 두 정점에 대해 다음 세 가지 조건 중 정확히 하나가 성립한다

(1) 두 정점의 시간 간격이 완전히 분리된 경우 (Disjoint Intervals):

(2) 한 정점의 시간 간격이 다른 정점의 시간 간격 안에 완전히 포함되는 경우 (Nested Intervals -  $u$  is descendant of  $v$ ): >>  $u$ 는  $v$ 의 자손이다!  $v.d < u.d < u.f < v.f$

(3) 다른 정점의 시간 간격이 첫 번째 정점의 시간 간격 안에 완전히 포함되는 경우 (Nested Intervals -  $v$  is descendant of  $u$ ): >>  $v$ 는  $u$ 의 자손이다!  $u.d < v.d < v.f < u.f$



Parenthesis Structure

## Edge의 구분

### 1. Tree edge : Depth-first forest에 있는 edge들

새로운 vertex를 발견하고 재귀적으로 DFS-VISIT(v)를 호출할 때 사용되는 edge (u,v)

@그림에서 : 두꺼운 선  $s \rightarrow j$ ,  $z \rightarrow y$

### 2. Back edge : vertex u를 조상(ancestor) v에 연결하는 edge들

non-tree edge들. DFS 탐색 중 정점 u에서 정점 v로 가는 edge(u,v)를 만났는데, v가 이미 회색(GRAY)일 때

@그림에서 : B

### 3. Forward Edges : vertex u를 자손(descendant) v에 연결하는 edge들

non-tree edge들. DFS 탐색 중 정점 u에서 정점 v로 가는 edge(u,v)를 만났는데, v가 이미 검은색(BLACK) 상태이고  $v.d > u.d$ 인 경우

@그림에서 : F

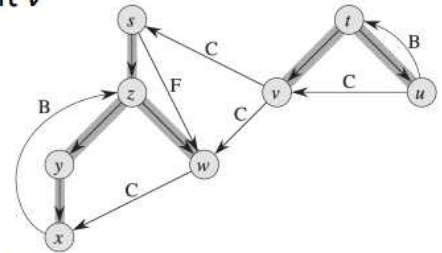
### 4. Cross Edges : 나머지 모든 edge들.

같은 depth-first tree에 있는 vertices 사이에 있는 edge, 서로 다른 depth-first tree에 있는 vertices 사이.

@그림에서 C

descendant v

es



All tree and forward edges going down

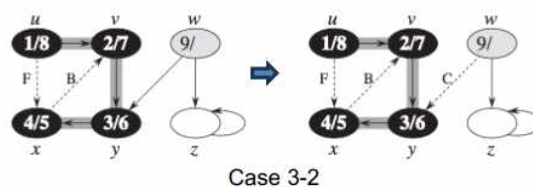
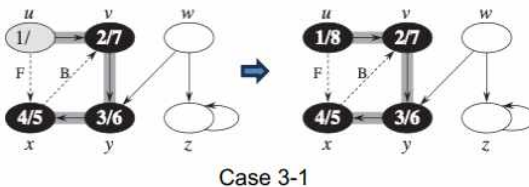
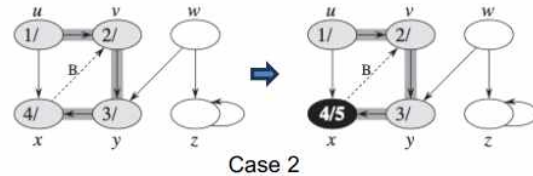
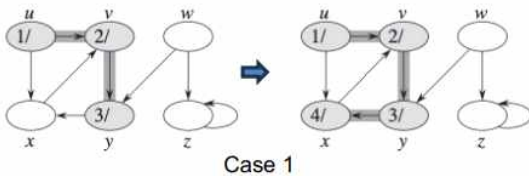
All back edges going up from a descendant to an ancestor

## Key Idea for Edge Classification

■ Case 1: WHITE indicates a tree edge

■ Case 2: GRAY indicates a back edge

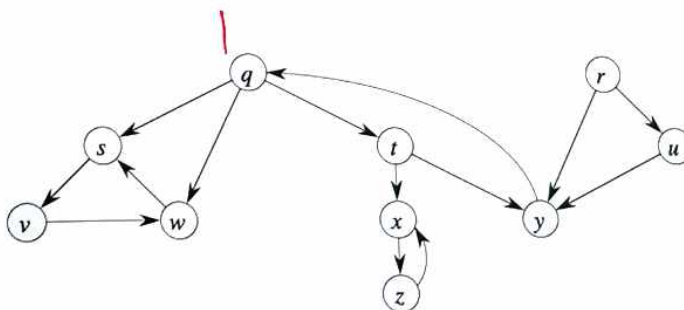
■ Case 3: BLACK indicates a forward or cross edge



## Exercise 6: DFS on a Directed Graph

■ Show how DFS works on the following graph by showing the discovery and finishing times for each vertex, and the classification of each edge

□ Assume that DFS considers the vertices in alphabetical order (start at q)



| Vertex | Discovered | Finished |
|--------|------------|----------|
| q      | 1          | 16       |
| r      | 17         | 20       |
| s      | 2          | 7        |
| t      | 8          | 15       |
| u      | 18         | 19       |
| v      | 3          | 6        |
| w      | 4          | 5        |
| x      | 9          | 12       |
| y      | 13         | 14       |
| z      | 10         | 11       |

Tree edges: (q,s), (s,v), (v,w), (q,t), (t,x), (x,z), (t,y), (r,u)

Forward edges: (q,w)

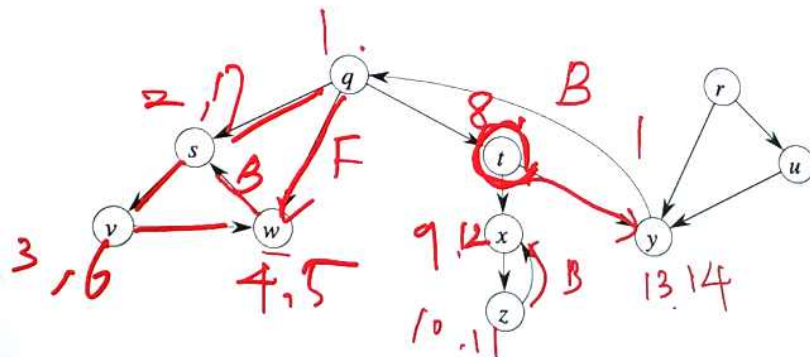
Back edges: (w,s), (y,q), (z,x)

Cross edges: (u,y), (r,y)



## Exercise 6: DFS on a Directed Graph

- Show how DFS works on the following graph by showing the discovery and finishing times for each vertex, and the classification of each edge
- Assume that DFS considers the vertices in alphabetical order (start at q)



| Vertex | Discovered | Finished |
|--------|------------|----------|
| q      | 1          | 16       |
| r      | 17         | 20       |
| s      | 2          | 7        |
| t      | 8          | 15       |
| u      | 18         | 19       |
| v      | 3          | 6        |
| w      | 4          | 5        |
| x      | 9          | 12       |
| y      | 13         | 14       |
| z      | 10         | 11       |

Tree edges: (q,s), (s,v), (v,w), (q,t), (t,x), (x,z), (t,y), (r,u)

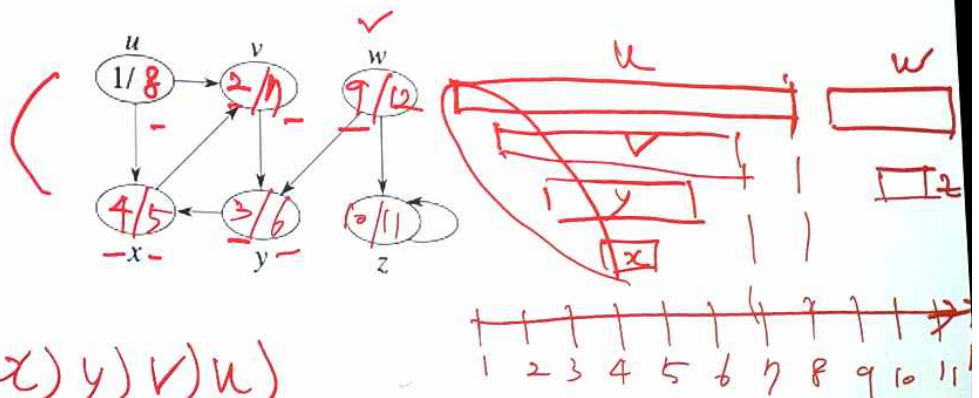
Forward edges: (q,w)

Back edges: (w,s), (y,q), (z,x)

Cross edges: (u,y), (r,y)

## Exercise 7: Parenthesis Structure

- Run DFS on the following directed graph and show its parenthesis structure
- Assume that DFS considers the vertices in alphabetical order



(u (v (y (x x) y) v) u)



## 적용 1 : Connected Component

비방향성 그래프에서는, **Connected Component**는 해당 집합 내의 모든 노드 쌍 사이에 경로가 존재(집합 안에서 임의의 노드 2개를 뽑아도 연결되어있음)하는 **최대 노드 집합**(더 이상 추가될 수 없음. 가장 큰 덩어리)이다.

### 어떻게 모든 Connected Component를 식별하는가?

Key idea : 그래프에서 DFS를 실행하면 Depth-first forest를 생성된다.

왜냐하면 Depth-first Forest는 그래프 G가 가진 Connected component만큼의 트리를 포함하기 때문

### 우리가 해야 할 것(Our mission)

DFS 및 DFS-VISIT 코드를 수정하여 각 정점  $v$ 에 정수 레이블  $v.cc(1부터 k까지)$ 를 할당하도록 하는 것.

Connected component를 식별하는 실질적인 방법. 각 vertex에 해당 vertex가 속한 connected component의 고유 번호를 부여하는 것을 목표로 함. 이 번호는 1부터 시작하여 총 연결 요소의 개수(k)까지 할당된다.

#### DFS(G)

```

1 for each vertex  $u \in G.V$ 
2    $u.color = WHITE$ 
3    $u.\pi = NIL$ 
4  $time = 0$ 
5 for each vertex  $u \in G.V$ 
6   if  $u.color == WHITE$ 
7     DFS-VISIT( $G, u$ )
  
```

$k = 1$

$u.cc = k$   
 $k = k + 1$

#### DFS-VISIT( $G, u$ )

```

1  $time = time + 1$ 
2  $u.d = time$ 
3  $u.color = GRAY$ 
4 for each  $v \in G.Adj[u]$ 
5   if  $v.color == WHITE$ 
6      $v.\pi = u$ 
7     DFS-VISIT( $G, v$ )
8  $u.color = BLACK$ 
9  $time = time + 1$ 
10  $u.f = time$ 
  
```

$v.cc = u.cc$

## 적용 2 : Topological Sort

**Topological Sort란?** : Directed Acyclic graph가 주어졌을 때, 모든 vertices를 선형적인 순서로 생성하는것.

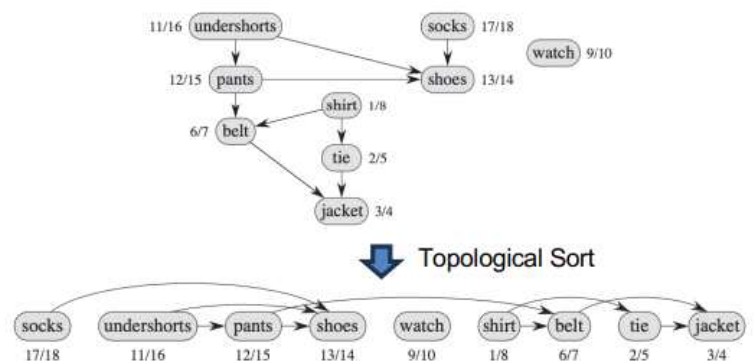
만약 그래프 G가  $edge(u,v)$ 를 포함한다면, 정렬 결과에서 u가 v보다 먼저 나타나도록 모든 vertex의 선형 순서를 생성하는 것.

### 왜 Topological Sort가 DAG에서 중요할까?

DAG를 사용하는 적용 사례들이 이벤트 간의 선행 관계(precedence 또는 의존성 dependency)을 나타낸다.



Real-world Control Systems



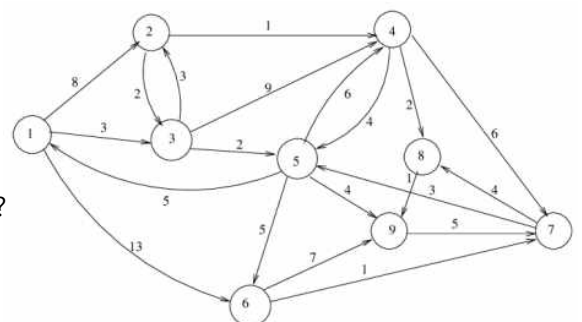
### Topological Sort를 하기 위한 요구사항(Requirements)

그래프가 cycle를 포함한다면, linear ordering이 불가능함.

ex) 오른쪽 그림은 불가능.

그럼 방향 그래프에 순환이 없음을 어떻게 확인할 수 있을까?

>>DFS 알고리즘을 사용



## 이 문제를 어떻게 해결할까?

Idea 1 : 진입 차수(in-degree)가 0인 노드부터 왼쪽에 배치한다.

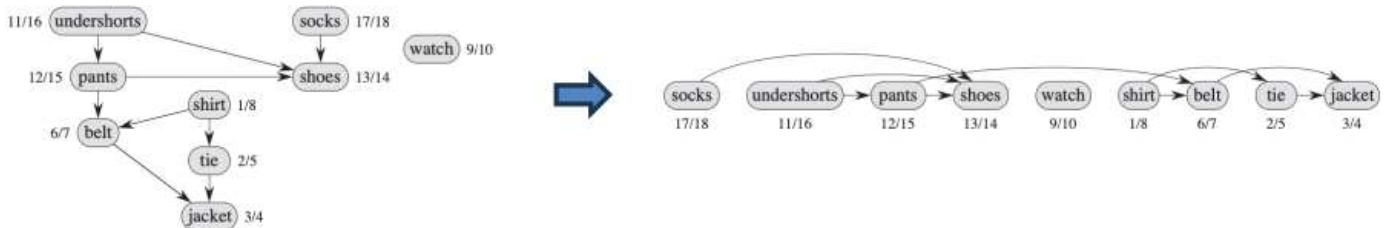
Idea 2 : 그래프 G에 DFS를 실행하고, 노드들을 완료 시간(f time)의 오름차순으로 오른쪽에서부터 배치한다.

### TOPOLOGICAL-SORT( $G$ )

- 1 call DFS( $G$ ) to compute finishing times  $v.f$  for each vertex  $v$  →  $O(|V| + |E|)$
- 2 as each vertex is finished, insert it onto the front of a linked list
- 3 return the linked list of vertices

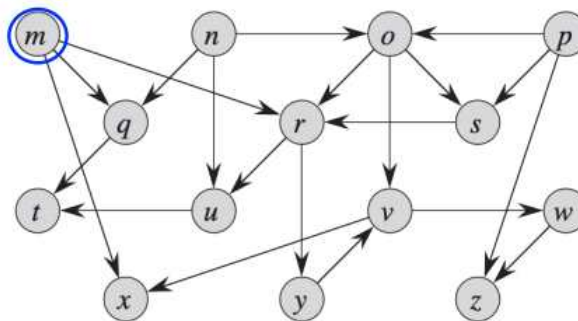
### Topological Sort의 정확도

edge  $(u,v)$ 에 대해  $u.f > v.f$



## Exercise 8: Topological Sort on a DAG

- Show the ordering of vertices produced by the topological sort when it is run on the following DAG
  - Assume that DFS considers the vertices in alphabetical order (start at  $m$ )



## 13주차 - Minimum Spanning Tree

### Spanning Tree란?

순환이 없고(acyclic)이 없고 그래프의 모든 vertices를 연결하는 트리

Spanning tree는 그래프를 "span"한다.

>> span한다는 것은 전체를 아우르다, 걸쳐있다는 의미. 원래 그래프의 모든 정점의 연결하여 덮는 것을 의미.

### Minimum Spanning Tree MST란?

모든 Spanning Tree중에 Weight(가중치, 비용)이 최소인 Spanning tree

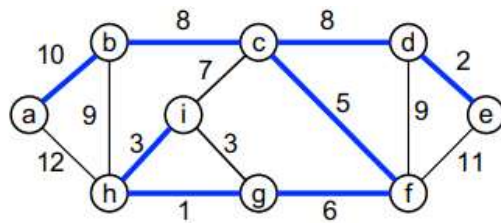
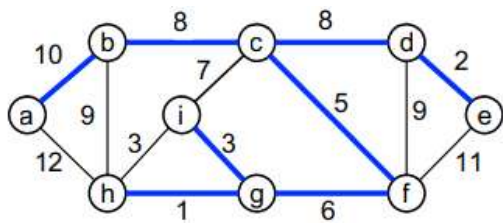
□  $w(u, v)$ : weight on edge  $(u, v)$

$$w(T) = \sum_{(u,v) \in T} w(u, v)$$

$w(u,v)$ 의 의미 :  $(u,v)$ 의 가중치

스몰오메가(T) : 트리 T에 있는 모든 가중치를 더한 값.

Minimum Spanning Tree는 Unique하지 않을 수 있다(유일하지 않을 수 있음)



## 적용 : 네트워크의 적용

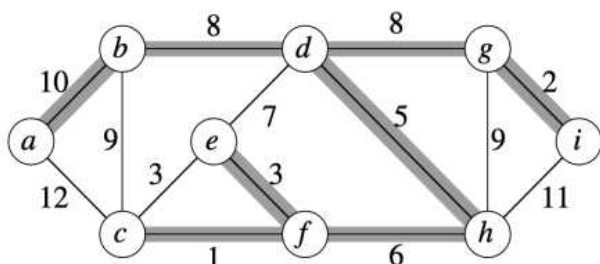
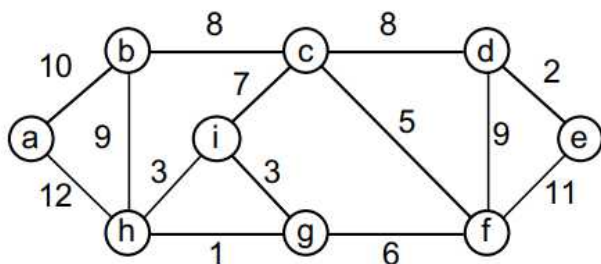
Cable networks(전화, TV, 인터넷 케이블)

제한된 자원이 주어졌을 때, 모든 vertices이 연결된 네트워크를 설계할 수 있을까? 그리고 건설 및 유지보수 비용을 어떻게 하면 최소화할 수 있을까?

## MST 찾기 문제

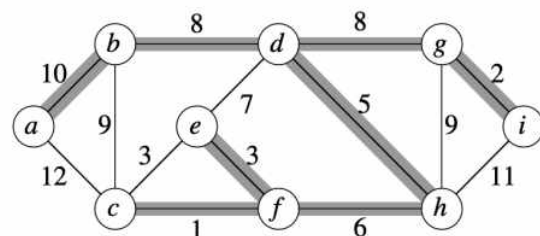
$G=(V,E)$ 를 edge집합  $E$ 에 실수값인 가중치(weight) 함수  $w$ 가 undirected graph라고 하자.

목표는  $G$ 의 모든 vertices를 연결하는 최소 스패닝 트리(MST)를 찾는 것이다.



주어진 그래프  $G = (V,E)$ 의 특징

1. MST는 edges가  $|V|-1$ 개
2. cycle이 없음.
3. unique하지 않을 수 있음



목표 : edge 집합  $A$ 를 만드는 것 (즉, MST)

초기화 :  $A$ 는 빈 집합

$A \subseteq T$  ( $A = \emptyset$ ) where  $T$  is a minimum spanning tree

$T$ 가 스패닝 트리일 때,  $A$ 는 공집합. > 어떤 스패닝 트리라도  $A$ 는  $T$ 의 부분집합.  $A$ 에 edge들이 추가되어도  $A$ 가 항상 어떤 MST의 부분집합이라는 속성을 유지한다!

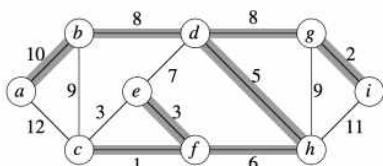
할일 : safe edge( $u,v$ )를  $A$ 에 추가한다 (greedy approach를 사용)

**safe edge** :  $A$ 에 새로운 edge ( $u,v$ )를 추가했을 때, 그 결과 집합이 여전히 어떤 MST의 부분집합이 될 수 있다면, 그 edge( $u,v$ )는 '안전한 edge'이다. MST를 만드는 데 방해가 되지 않음을 의미함.

>> 간선 ( $u,v$ )는  $A \cup \{(u,v)\}$  또한 MST  $T$ 의 부분 집합일 경우에만 안전하다.

$A$ 는 한 번에 하나의 e씩 성장해서 MST를 형성한다.

$A$ 는 항상 MST  $T$ 의 부분집합이어야 한다.



GENERIC-MST( $G, w$ )

```

1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3    find an edge ( $u, v$ ) that is safe for  $A$ 
4     $A = A \cup \{(u, v)\}$ 
5  return  $A$ 

```

⇒ How do we find safe edges?

## Safe Edge를 어떻게 찾을까?

edge  $(u,v)$ 는 그래프의 모든 edge중에서 **최소 가중치(weight)**를 가짐.

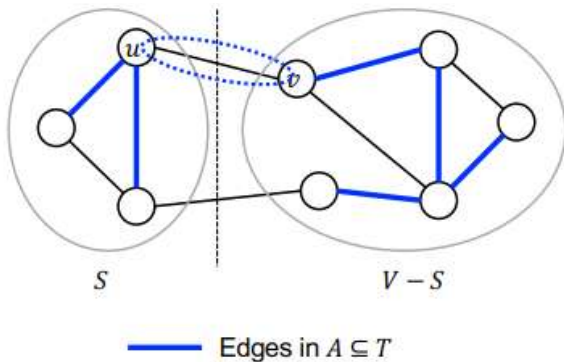
>  $(u,v)$ 는 간선 집합 **A**에 대해서 안전할까?

S가 u를 포함하지만 v는 포함하지 않는 vertices의 부분집합이라고 하자.

MST를 만들기 위해서는, S와 V-S는 적어도 하나의 edge로 연결되어 있어야 한다.

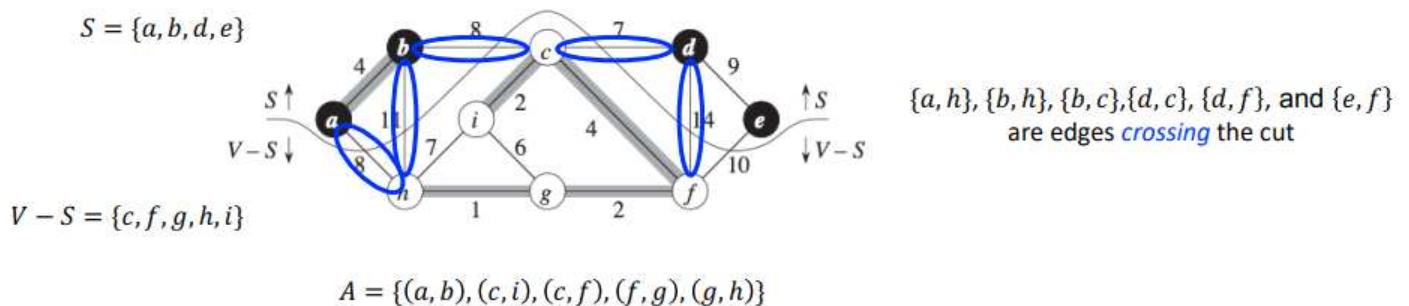
> 왜 최소 가중치를 가진 edge  $(u,v)$ 를 선택하지 않는가? -> 명백히 안전(safe)하기 때문!

>> 이 경우 **S와 V-S를 연결하는 간선 중 최소 가중치 간선을 선택하는 것은 항상 안전하다는 직관**

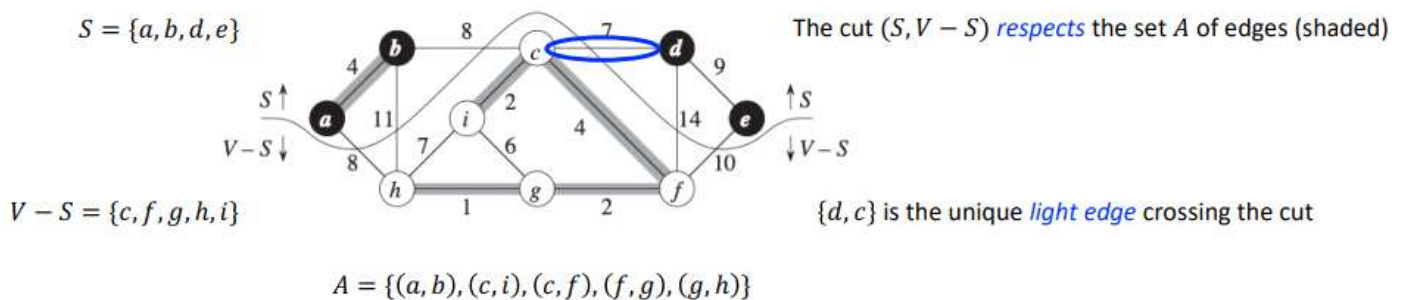


## MST에서의 용어

**cut** (S,V-S)는 disjoint(joint가 안 되어있는, 분리되어 있는)인 집합 S와 V-S로 나누는 것  
 edge  $(u,v)$ 는 한쪽 끝점이 S에 있고 다른 쪽 끝점이 V-S에 있다면 **cut(S,V-S)**을 **cross**한다.



cut이 edge 집합 **A**가 cut을 **cross**하는 edge를 포함하지 않을 때 **A**를 **respect**한다고 한다!  
 edge가 그것의 weight가 cut을 cross하는 edge중에서 **최소**이면 **light**이라고 한다.

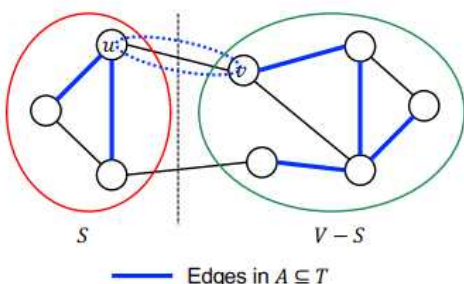


## Theorem 정리 1

$G(V,E)$ 를 edge 집합 E에 실수 값 가중치 함수 **w**가 정의된 **undirected** graph라고 하자.

**A**를 G의 어떤 MST에 포함된 **edge** 집합이라 하자. 그리고 (S,V-S)를 **A**를 **respect**하는 G의 어떤 cut이라 하자  
 $(u,v)$ 를 (S, V-S)를 **cross**하는 **light edge** 즉, 라 하자

그러면,  $(u,v)$ 는 A에 safe하다. >> (즉,  $A \cup \{(u,v)\}$ 는 T의 부분 집합이다).



If we prove that this theorem is true,  
**We can safely choose a light edge to form an MST at each iteration**

만약 이 정리가 참이라면, 각 반복에서 MST를 형성하기 위해 **safe**하게 **light edge**를 선택할 수 있다.



## Theorem 1 증명 개요

T를 edge 집합 A를 포함하는 MST라고 하자. 가정을 함으로서 증명 시작.

증명하고 싶은 것 : light edge  $(u,v)$ 는 A에 대해 safe하다.

가정 : T는  $(u,v)$ 를 포함하지 않는다.

★증명하고 싶은 명제와 반대되는 것을 가정합니다. (모순에 의한 증명)

여기서 그 반대 가정이 바로 "light 간선  $(u,v)$ 가 T에 포함되어 있지 않다"는 것.

모순이 발생하면  $(u,v)$ 는 반드시 T에 포함되어야 한다는 결론에 도달.

목표 :  $A \cup \{(u,v)\}$ 를 포함하는 또 다른 MST T'를 구성하는 것이다.

light edge  $(u,v)$ 가 A에 safe하다는 것을 보여야 한다.

>> 만약, T'를 성공적으로 구성하여 T가 T'와 동일한 최소 가중치를 가지면서  $(u,v)$ 를 포함한다면, 이는 light edge  $(u,v)$ 를 A에 추가하는 것이 항상 safe한 선택이었음을 의미.(증명 성공)

T'가 MST라면,  $(u,v)$ 를 포함해도 최소 가중치를 유지할 수 있다는 뜻.

## Theorem 1 증명

MST T가 u와 v사이에 unique한 path p를 포함한다.

spanning tree가 두 vertices 사이에 unique한 path(vertices 여러개)를 가진다

path p는 반드시  $(S, V-S)$ 를 cross해야 한다.

$u \rightarrow v$ 로의 경로 p가 존재하는데, u와 v는 서로 다른 vertices set에 있으므로 p는 반드시  $(S, V-S)$ 를 가로질러야 한다.

T의 적어도 하나의 edge는 simple path p에 포함된다.

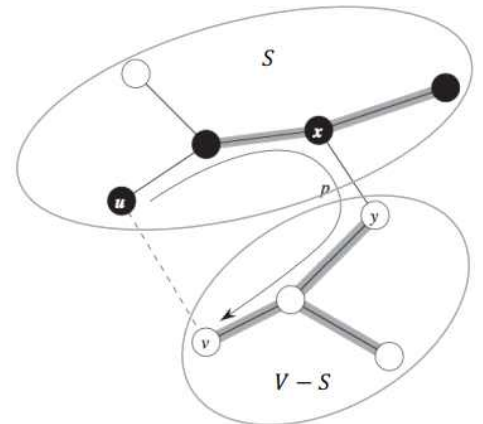
p의 edge 중 S와 V-S를 cross하는 것이 반드시 존재한다.

$(x,y)$ 를 경로 p의 중에 cut  $(S, V-S)$ 를 cross하는 edge라고 하자.

edge  $(x,y)$ 가 A에 없다. (반대로 가정)

반대로 가정..!

cut  $(S, V-S)$ 는 A를 respect한다. (S와 V-S가 분리됨...!)



또 다른 MST를 만들기 위해서는

T에는  $(u,v)$ 가 없다고 가정했으니 T를 변형해서  $(u,v)$ 를 포함하는 T'를 만들 A is a subset of T (shaded edges)

$(x,y)$ 를 제거해서 T를 두개의 components로 나눈다.

$(u,v)$ 를 추가해서 두 개의 components를 다시 합친다.(새로운 MST)

$T' = T - \{(x,y)\} \cup \{(u,v)\}$

T'이 MST임을 보이기 위해서

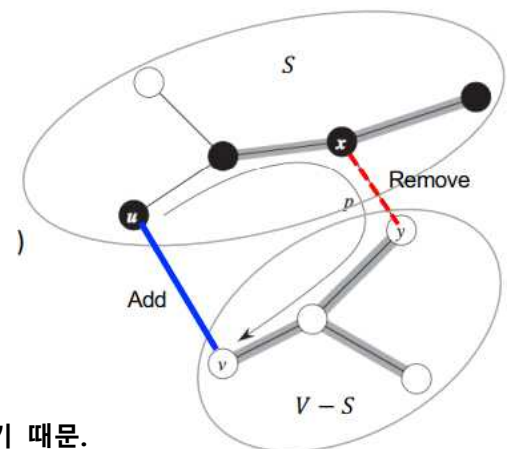
$w(T') = w(T) - w(x,y) + w(u,v) \leq w(T)$

더 큰걸 빼고 (최소)작은걸 더함

T'의 가중치는 T의 가중치에서  $(x,y)$ 의 가중치를 빼고  $(u,v)$ 의 가중치를 더한 것과 같고, 이는 T의 가중치보다 작거나 같다.

★ $w(u,v) \leq w(x,y)$  : 왜냐하면,  $(u,v)$ 는 cut을 가로지르는 light edge이기 때문.

$(u,v)$ 는 가로지르는 모든 edge중 가중치가 가장 작거나 같음.



A is a subset of T (shaded edges)

그런데, 증명 맨 처음에  $T$ 는 MST라고 함.

그러니  $w(T) \leq w(T')$ 이다. >>모순이다...! 그러므로 동시에 성립하려면  $w(T)=w(T')$ 이 된다.

$\therefore T$ 도 MST이고, light edge( $u,v$ )는  $A$ 에 대해서 safe하다.

설명: 증명의 최종 결론입니다.

- $T'$ 의 가중치가  $T$ 의 가중치와 같다는 것은,  $T'$  또한 '최소 스패닝 트리'라는 것을 의미합니다.
- 그런데  $T'$ 는 우리가 '경량 간선 ( $u, v$ )가  $T$ 에 포함되지 않는다'고 가정했을 때 만들어진 MST입니다. 그리고 이  $T'$ 는 ( $u, v$ )를 포함합니다.
- 이는 우리가 애초에 세웠던 가정("경량 간선 ( $u, v$ )는 MST  $T$ 에 포함되어 있지 않다")이 틀렸음을 의미하는 모순입니다.
- 따라서, 경량 간선 ( $u, v$ )는 항상 MST에 포함될 수 있으며, 이를  $A$ 에 추가하는 것은 '안전한' 선택입니다. 이로써 정리 1이 증명됩니다.

## Corollary 보조 정리 2

$G=(V,E)$ 를 edge집합  $E$ 에 실수 가중치 함수  $w$ 가 정의된 **connected, undirected graph**라 하자.

그리고  $A$ 를  $G$ 의 어떤 MST에 포함된 **edge들의 부분집합**이라고 하자.

**Let  $C = (V_c, E_c)$  be a connected component (tree) in the forest  $G_A = (V, A)$  and  $(u, v)$  is a light edge connecting  $C$  to some other component in  $G_A$ , then  $(u, v)$  is safe for  $A$**

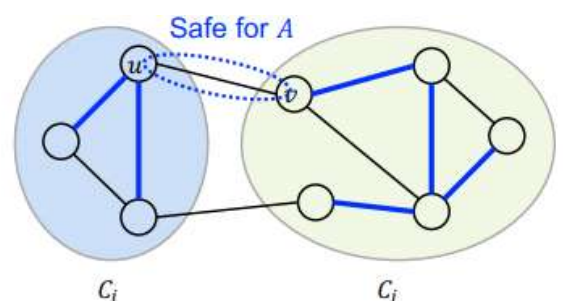
해석:  $C = (V_c, E_c)$ 를 포리스트  $G_A = (V, A)$ 에 있는 연결 요소(트리)라고 하고,  $(u, v)$ 가  $C$ 를  $G_A$ 의 다른 어떤 연결 요소에 연결하는 경량 간선이라면,  $(u, v)$ 는  $A$ 에 대해 안전하다.

설명: 이것이 보조 정리 2의 핵심 내용입니다.

- **Forest  $G_A = (V, A)$ :** 현재까지 우리가 선택한 간선 집합  $A$ 와 모든 정점  $V$ 로 구성된 그래프를  $G_A$ 라고 합니다.  $A$ 에 있는 간선들은 사이클을 형성하지 않으므로,  $G_A$ 는 여러 개의 '트리'로 구성된 '포리스트(forest)'가 됩니다. 이 각 트리가 하나의 연결 요소입니다.
- **$C = (V_c, E_c)$  is a connected component (tree) in the forest  $G_A$ :**  $C$ 는 이 포리스트  $G_A$ 내의 하나의 연결 요소(즉, 하나의 트리)입니다.
- **$(u, v)$  is a light edge connecting  $C$  to some other component in  $G_A$ :**  $(u, v)$ 는  $C$ 에 속한 정점과  $C$ 에 속하지 않는 다른 연결 요소의 정점을 연결하는 간선입니다. 특히, 이 간선은  $C$ 와 다른 연결 요소 사이를 연결하는 간선들 중에서 가중치가 가장 작은 '경량 간선'입니다.
- **then  $(u, v)$  is safe for  $A$ :** 결론적으로, 이러한 경량 간선  $(u, v)$ 는 항상 현재 간선 집합  $A$ 에 추가해도 MST를 형성하는 데 안전하다는 것입니다.

### GENERIC-MST( $G, w$ )

```
1  $A = \emptyset$  → Start with  $|V|$  components (trees)
2 while  $A$  does not form a spanning tree
3   find an edge  $(u, v)$  that is safe for  $A$ 
4    $A = A \cup \{(u, v)\}$ 
5 return  $A$  → Merge two components into one by
               choosing the light edge that connects them
```



## Theorem 1의 역 (p->q인데, q->p일까) 분석

정리 1 "light edge이면 safe하다."  $\leftrightarrow$  "safe하면 light edge일까?"

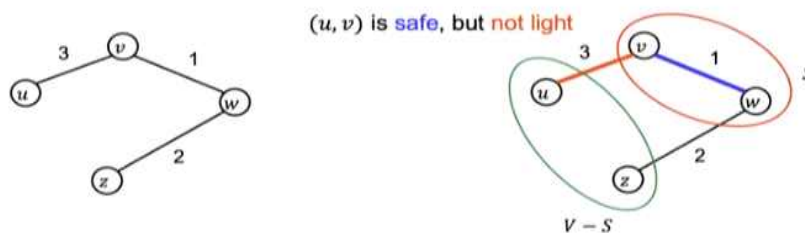
$G(V,E)$ 를 edge 집합  $E$ 에 실수 값 가중치 함수  $w$ 가 정의된 **undirected** graph라고 하자.

$A$ 를  $G$ 의 어떤 MST에 포함된 **edge 집합**이라 하자. 그리고  $(S, V-S)$ 를  $A$ 를 **respect**하는  $G$ 의 어떤 cut이라 하자  
( $u,v$ )를  $A$ 에 대해 **safe**하다고 하자 (Theorem에서 결론이었던 것이 조건이 됨!)

그러면,  $(u,v)$ 는 cut  $(S, V-S)$ 를 가로지르는 **light edge**이다.

>> Theorem 1의 역은 참인가? 아니라면 반례를 들어라.

- Let  $G = (V, E)$  be a connected, undirected graph with a real-valued weight function  $w$  defined on  $E$ .
- Let  $A$  be a set of edges that is included in some minimum spanning tree (MST) for  $G$ , let  $(S, V - S)$  be any cut of  $G$  that respects  $A$ ,
- Let edge  $(u, v)$  be **safe** for  $A$  (that is,  $A \cup \{(u, v)\} \subseteq T$ )
- Then,  $(u, v)$  is a **light** edge crossing the cut  $(S, V - S)$



$S$ 가 빨간색,  $V-S$ 가 초록색이라고 하면,

$(u,v)$ 는 **safe**(SP되려면 뽑혀야 함)하지만,  $(z,w)$ 라는 반례가 있기 때문에 **light** 하지는 않는다!

역은 거짓이다.

## MST를 해결 알고리즘

Kruskal's Algorithm

Prim's Algorithm

### Kruskal's Algorithm

MST를 찾는 대표적인 알고리즘 중 하나.

#### 1) 각 vertex가 자신만의 component인 상태로 시작한다.

알고리즘의 초기 상태. 어떤 edge도 선택되지 않았으므로 그래프의 모든 vertex는 독립적임.

#### 2) edge들의 가중치(weight)를 오름차순으로 정렬한다.

이 정렬 작업을 통해 두 component들을 연결하는 가장 light edge를 효율적으로 찾을 수 있음  
가중치가 작은 edge부터 순차적으로 살펴보면서 MST에 포함시킬 edge를 선택할 수 있음

>> Greedy approach의 기반이 됨.

#### 3) 최소 가중치를 가진 edge를 선택하여 두 components를 하나로 합친다.

edge  $A$ 에 추가

선택하려는 edge는 두 가지 경우를 발생시킬 수 있음.

경우 1. 같은 component 내의 두 vertex를 연결하는 경우

> 같은 component에 2개의 vertex가 있거나 cycle을 형성하게 될 수 있음.

경우 2. 두 개의 다른 component를 연결하는 경우.

> 이 edge를 추가하면 두 개의 분리된 연결 요소가 하나로 합쳐지면서 cycle은 생성하지 않음.

>> edge를 safe하게 MST에 추가할 수 있음!

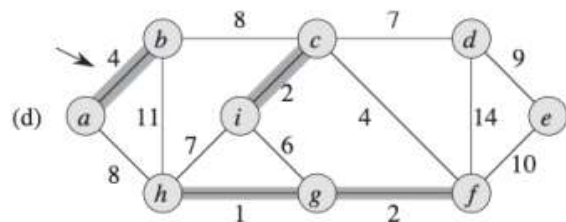
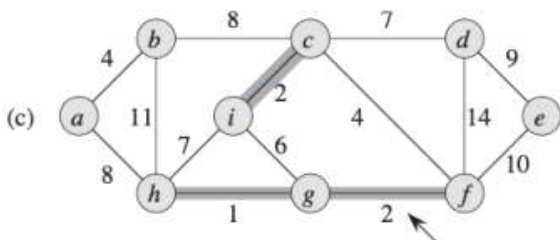
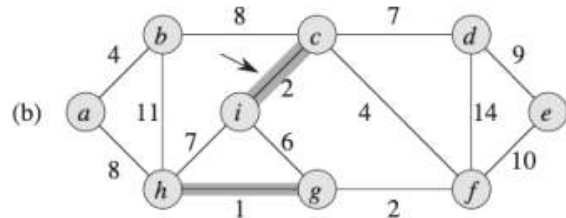
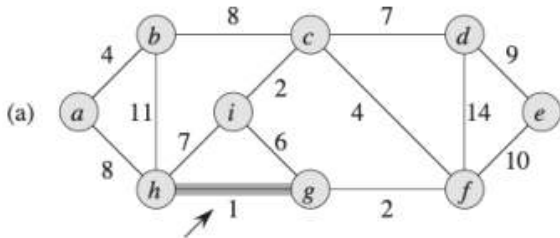
disjoint된 집합 자료구조는 어떤 두 vertex가 같은 component에 속하는지 판별하고, component를 합치는 작업을 수행하는데 사용될 수 있음.

# MST-KRUSKAL( $G, w$ )

```

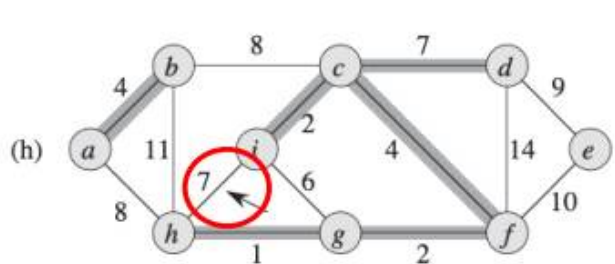
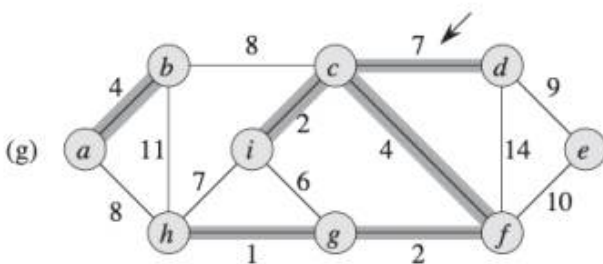
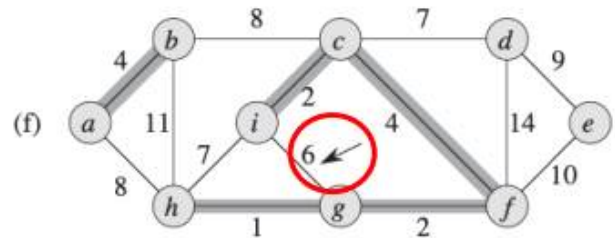
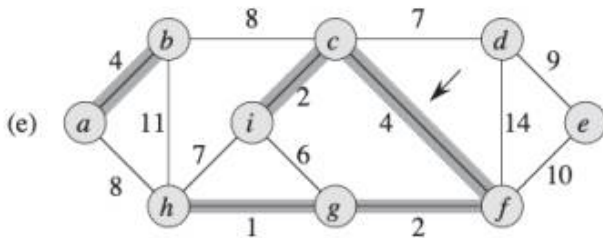
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3    MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6    if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7       $A = A \cup \{(u, v)\}$ 
8      UNION( $u, v$ )
9  return  $A$ 

```



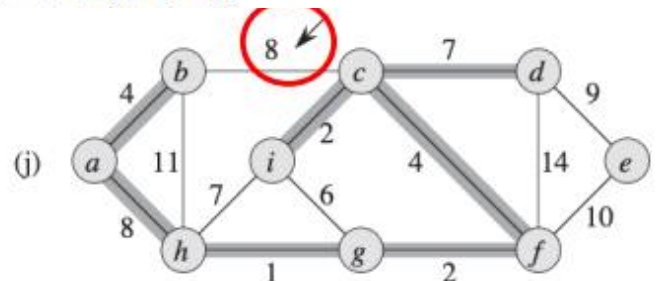
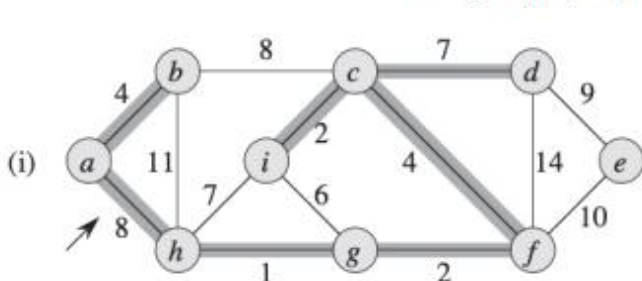
Sorted edges =  $\langle 1, 2, 2, 4, 4, 6, 7, 7, 8, 8, 9, 10, 11, 14 \rangle$

$A = \{(h, g), (c, i), (g, f), (a, b)\}$

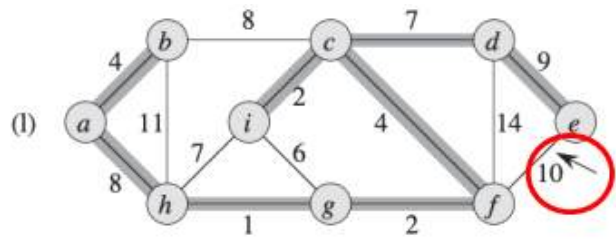
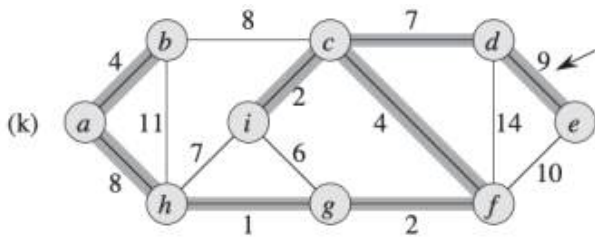


Sorted edges =  $\langle 1, 2, 2, 4, 4, 6, 7, 7, 8, 8, 9, 10, 11, 14 \rangle$

$A = \{(h, g), (c, i), (g, f), (a, b), (c, f), (c, d)\}$

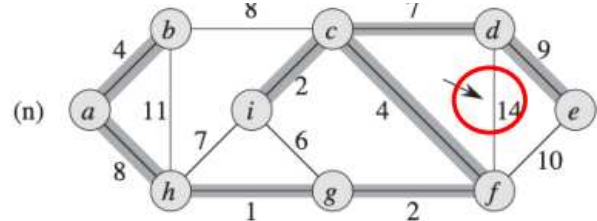
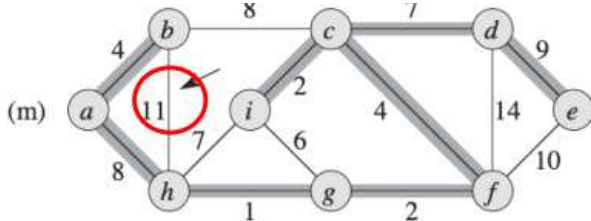






Sorted edges =  $\langle 1, 2, 2, 4, 4, 6, 7, 7, 8, 8, 9, 10, 11, 14 \rangle$

$A = \{(h, g), (c, i), (g, f), (a, b), (c, f), (c, d), (a, h), (d, e)\}$



Sorted edges =  $\langle 1, 2, 2, 4, 4, 6, 7, 7, 8, 8, 9, 10, 11, 14 \rangle$

$A = \{(h, g), (c, i), (g, f), (a, b), (c, f), (c, d), (a, h), (d, e)\}$

$$|A| = |V| - 1$$

### Prim's Algorithm

MST를 찾는 대표적인 알고리즘 중 하나. Kruskal's와 다르게 component 고려X

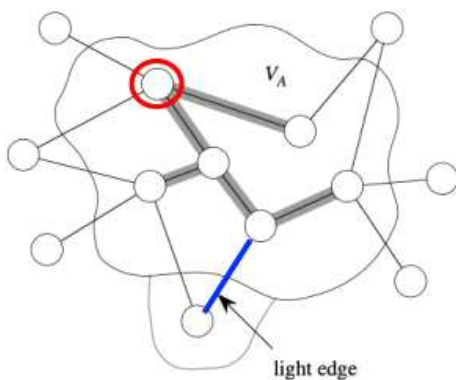
Prim's Algorithm은 edge 집합 A에 있는 edge들이 **하나의 단일 트리**를 형성한다.

>> 시작부터 끝까지 1개의 트리로!

여러개의 component들을 연결하는 Kruskal's과는 다름!

임의의(Arbitrary) root r에서 시작한다.

cut ( $V - V_A$ )를 cross하는 light edge를 집합 A에 추가한다. >> priority queue가 필요!

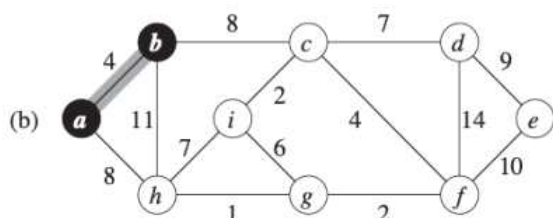
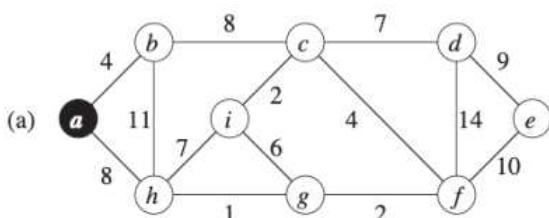


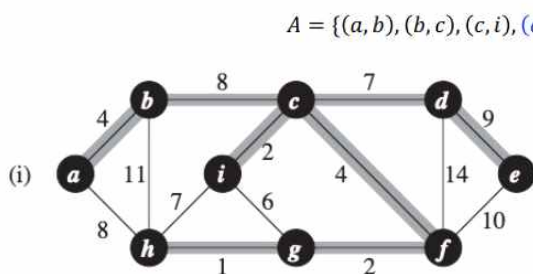
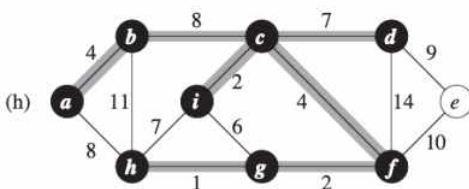
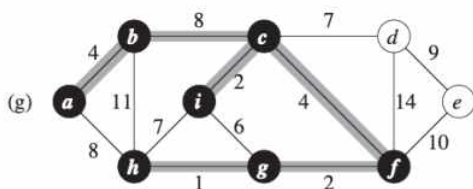
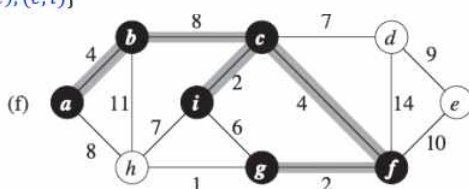
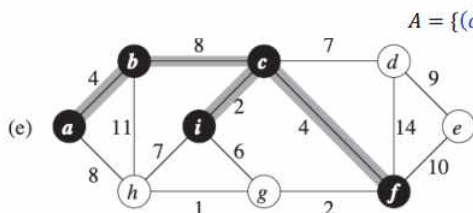
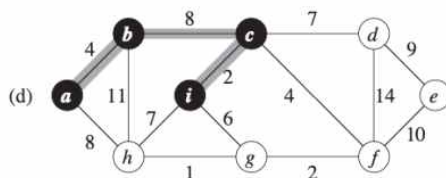
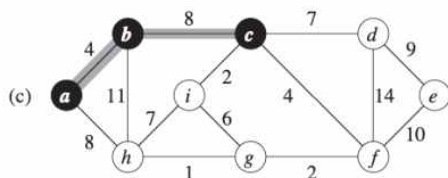
MST-PRIM( $G, u, r$ )

```

1 for each  $u \in G.V$ 
2    $u.key = \infty$ 
3    $u.\pi = \text{NIL}$ 
4  $r.key = 0$ 
5  $Q = G.V$ 
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.Adj[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.key$ 
10       $v.\pi = u$ 
11       $v.key = w(u, v)$ 

```





단계를 거듭하면서 V와 V-S로 나눠보면서  
어떻게 light edge인지 찾기!

$A = \{(a, b), (b, c), (c, i), (c, f), (f, g), (h, g), (c, d), (d, e)\}$

Two spanning tree computed by Kruskal's algorithm  
 $A = \{(h, g), (c, i), (g, f), (a, b), (c, f), (c, d), (a, h), (d, e)\}$

## Exercise 1: MST with a Different Weight Function

- Given a graph  $G$  and an MST  $T$ , suppose that the weight of one edge in  $T$  decreases. In this case, is  $T$  still an MST for  $G$ ?
- Formally, let  $T$  be an MST for  $G$  and the original weight function  $w$ . Then, choose one edge  $(x, y) \in T$  and a positive number  $k$ , and define the weight function  $w'$  by

$$w'(u, v) = \begin{cases} w(u, v) & \text{if } (u, v) \neq (x, y) \\ w(x, y) - k & \text{if } (u, v) = (x, y) \end{cases}$$

- Show that  $T$  is an MST for  $G$  with edge weights defined by  $w'$

That is, show that  $w'(T)$  is minimized

$w'$ 은 기존 가중치 함수  $w$ 에서 딱 한 간선의 가중치만을 줄인 새로운 함수

$T$ 는 MST가 아닌 다른 트리.

MST에서 edge를 하나( $x, y$ )의 가중치를 줄여도  $T$ 가 여전히 MST일까?

ex. 5->3으로 줄임

>> 새로운 가중치 함수  $w'$ 을 정의

>>  $w'(T)$   $w(T)$ 라면 참이다.. 증명을 해보자

## □ Exercise 1: MST with a Different Weight Function

- Let  $w(T) = \sum_{(x,y) \in T} w(x,y)$  be the sum of the weights of the edges in  $T$ .
  - We have  $w'(T) = w(T) - k$  ( $w(T) > w'(T)$ ) since we decrease the weight of a single edge  $(x,y)$  by  $k$

$$w'(u,v) = \begin{cases} w(u,v) & \text{if } (u,v) \neq (x,y) \\ w(x,y) - k & \text{if } (u,v) = (x,y) \end{cases}$$

- Consider any other spanning tree  $T'$ , that is,  $w(T) \leq w(T')$ 
  - If the selected edge  $(x,y)$  is not in  $T'$ , then  $w'(T') = w(T') \geq w(T) > w'(T)$

$$\rightarrow w'(T') > w'(T)$$

$T$  is an MST for  $G$  with edge weights defined by  $w'$

- If the selected edge  $(x,y)$  is in  $T'$ , then  $w'(T') = w(T') - k \geq w(T) - k = w'(T)$

$$\rightarrow w'(T') \geq w'(T)$$

$T$  is an MST for  $G$  with edge weights defined by  $w'$

| 구간                                                | 의미                                               |
|---------------------------------------------------|--------------------------------------------------|
| $(u,v) \neq (x,y) \Rightarrow w(u,v) \neq w(x,y)$ | 이 간선이 우리가 줄이려는 간선이 <b>아닌 경우</b> → 원래 가중치 그대로 사용! |
| $(u,v) = (x,y) \Rightarrow w(u,v) = w(x,y)$       | 이 간선이 우리가 <b>가중치를 줄이려는 그 간선</b> → $k$ 만큼 줄여서 사용! |

## □ Exercise 2: Subset of MST

- Let  $T$  be a minimum spanning tree of a graph  $G = (V, E)$  and  $V'$  be a subset of  $V$
- Let  $T'$  be the subgraph of  $T$  induced by  $V'$  and  $G'$  be the subgraph of  $G$  induced by  $V'$
- Show that if  $T'$  is connected, then  $T'$  is a minimum spanning tree of  $G'$ 
  - Use the contraction

MST에서 일부 MST만 뽑아서 subset을 만들어도, 그것이 MST인가?

$V'$ 을  $V$ 의 subset이라 하자

$T'$ 이 연결되어 있다면,  $T'$ 은  $G'$ 의 MST이다.

- Let  $T$  be a minimum spanning tree of a graph  $G = (V, E)$  and  $V'$  be a subset of  $V$
- Let  $T'$  be the subgraph of  $T$  induced by  $V'$  and  $G'$  be the subgraph of  $G$  induced by  $V'$
- Show that if  $T'$  is connected, then  $T'$  is a minimum spanning tree of  $G'$ 
  - Use the contraction

>>모순을 통한 증명

$$w(S \cup T'') = w(S) + w(T'') < w(S) + w(T') = w(S \cup T') = w(T).$$

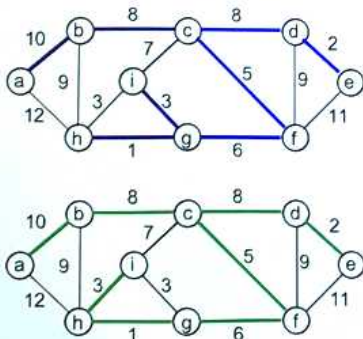
- 어떤 사람이  $G'$ 에서  $T'$ 보다 더 가벼운 트리  $T''$ 를 찾았다고 해봐. >> 실제로는  $T''$ 이 없음!
- 그럼  $T''$ 와 나머지  $T$ 의 일부  $S$ 를 합치면, 전체  $T$ 보다 더 가벼운 트리가 생김.
- 근데  $T$ 는 MST여야 하니까 모순이 생김! >>  $w(S \cup T'')$ 이 더 작을 수가 없음.

**결론 :**  $T'$ 은  $G'$ 에서도 여전히 MST이다!



### ❑ Exercise 3: Multiple Minimum Spanning Trees

- Let  $T$  be a minimum spanning tree of a graph  $G$ , and let  $L$  be the sorted list of the edge weights of  $T$
- For any other minimum spanning tree  $T'$ , is the list  $L$  equal to the sorted list  $L'$  of edge weights of  $T'$ ?

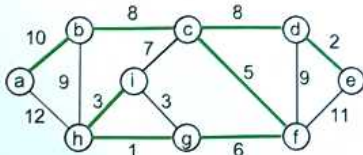


$(u, v)$  in  $T$ , but not in  $T'$

$L = \langle w_1, w_2, w_3, w_4, w_5, \dots, w_{|V|-1} \rangle$

$$w_4(u, v) \neq w'_4(x, y)$$

$(x, y)$  in  $T'$ , but not in  $T$   
 $L' = \langle w'_1, w'_2, w'_3, w'_4, w'_5, \dots, w'_{|V|-1} \rangle$



Adding in  $(u, v)$  to  $T'(L')$

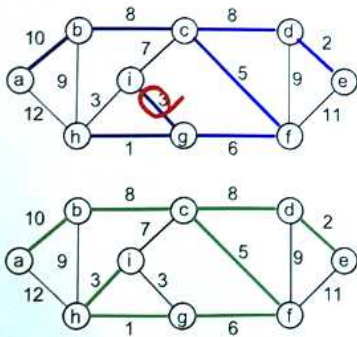
A cycle is formed by  $(u, v)$

(Case 1) If there exists an edge  $w(a, b) > w(u, v)$

By replacing  $(a, b)$  with  $(u, v)$ ,  
We can get a spanning tree with lower cost

Contradiction!

$$\rightarrow w(u, v) \geq a(a, b)$$

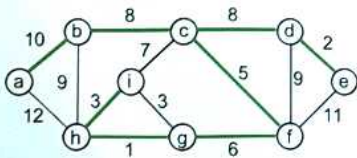


$(u, v)$  in  $T$ , but not in  $T'$

$L = \langle w_1, w_2, w_3, \mathbf{w}_4, w_5, \dots, w_{|V|-1} \rangle$

$$w_4(u, v) \neq w'_4(x, y)$$

$(x, y)$  in  $T'$ , but not in  $T$   
 $L' = \langle w'_1, w'_2, w'_3, w'_4, w'_5, \dots, w'_{|V|-1} \rangle$



Removing  $(u, v)$  from  $T(L)$

$T$  is divided into two components

**(Case 2)** If  $w(u, v) > w(a, b)$

By connecting the two components with the edge  $(a, b)$  in the cycle  
We can get a spanning tree with lower cost

Contradiction!

$$\rightarrow w(u, v) = w(a, b)$$

Continuing this way...  $\rightarrow$  eventually they must have every edge in common

Therefore, all minimum spanning trees have the same sorted list of edge weights!

MST T와 또 다른 MST T'이 있을 때 T의 edge를 sorting한 L과 T'의 edge를 sorting한 L'이 있다고 할 때, L과 L'은 같은가?

$(u,v)$ 는  $T$ 에만 있고,  $(x,y)$ 는  $T'$ 에만 있다.

$w_4(u,v)$ 와  $w'_4(x,y)$ 는 다르다

### CASE 1 : Adding edge

1. T에 **(u,v)**를 추가한다.
2. Spanning tree에 edge를 추가했으니까 **cycle**이 생겨버린다!
3. 그 cycle안에는 여러 edge들이 있을텐데, 제일 **작은걸** 바꿔보자!  $w(a,b) > w(u,v)$  라면?
4. 원래 있던 (a,b)를 빼고 **(u,v)**를 넣으면 다시 트리가 된다! >> 그런데 비용이 더 싸졌네?  
>> 그런데 애초에 T는 MST이므로, **모순이다**  $\therefore w(u,v) \leq w(a,b)$

## CASE 2 : Removing

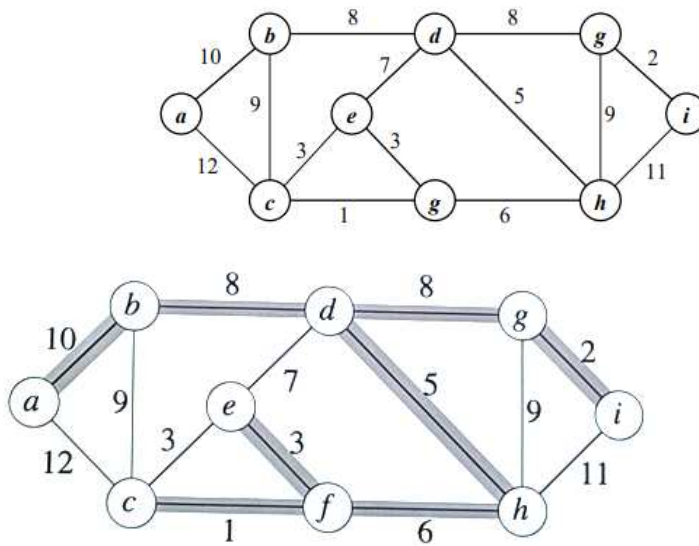
1.  $(u,v)$ 를  $T(L)$ 에서 제거한다. >> Tree는 연결된 그래프이기 때문에 두 개의 component로 쪼개진다!
2.  $T$ 가 두 component로 쪼개진다
3.  $T$ 에는  **$(a,b)$** 라는 edge가 있는데, 이 edge가 두 컴포넌트를 다시 연결시켜준다  
>> 그런데 만약  **$w(u,v) > w(a,b)$**  즉,  **$a,b$ 로 연결한게 더 싸다면?**
4. 가중치가 더 낮아지므로 새로운 MST가 생기게 된다!  
>> 그런데 애초에  $T$ 는 MST이므로, **모순이다**

∴ 따라서 모순이 생기지 않으려면 모든 MST의 edge weight list는 같을 수밖에 없다! >> 합치면  $w(u,v)=w(a,b)$



## Exercise 4: Kruskal's Algorithm

- Show how Kruskal's algorithm works on the following directed graph and write down the set  $A$  of edges included a minimum spanning tree

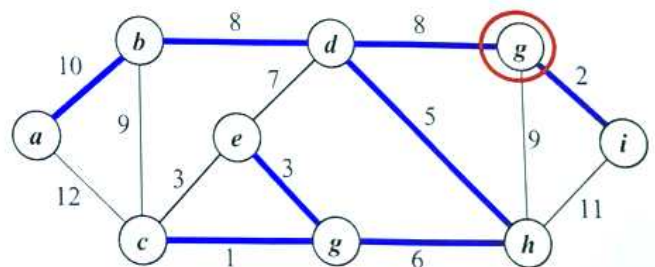
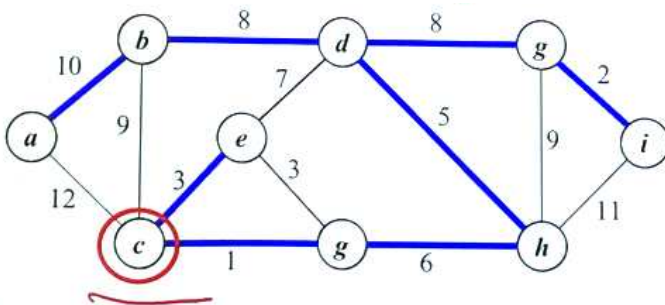
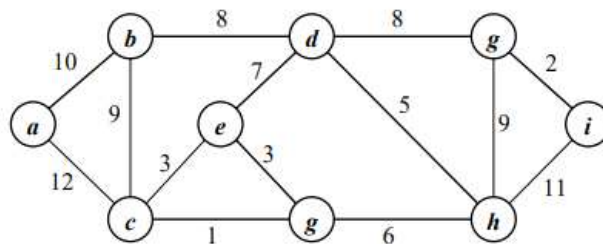


$(c, f)$  : safe  
 $(g, i)$  : safe  
 $(e, f)$  : safe  
 $(c, e)$  : reject  
 $(d, h)$  : safe  
 $(f, h)$  : safe  
 $(e, d)$  : reject  
 $(b, d)$  : safe  
 $(d, g)$  : safe  
 $(b, c)$  : reject  
 $(g, h)$  : reject  
 $(a, b)$  : safe

Can be exchanged

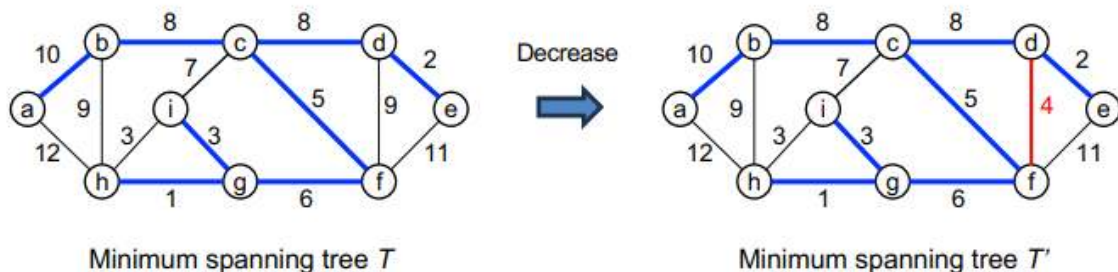
## Exercise 5: Prim's Algorithm

- Show how Prim's algorithm works on the following directed graph and write down the set  $A$  of edges included a minimum spanning tree
  - Start from different random roots ( $c$ , and  $g$ )

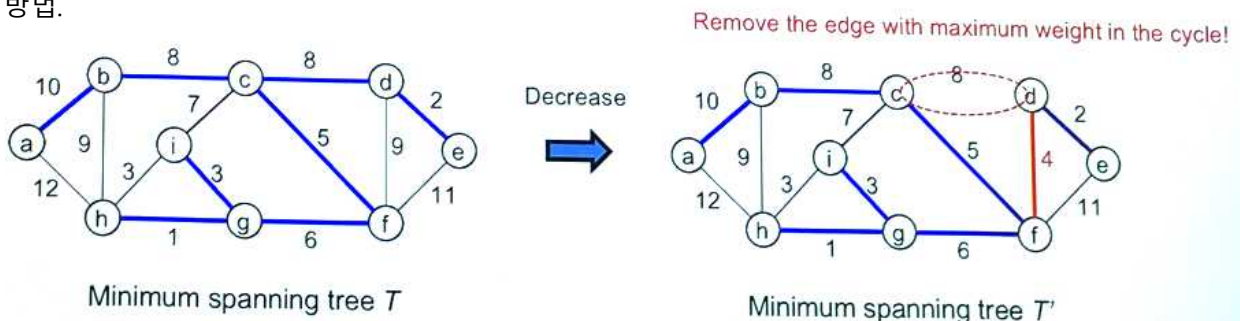


## Exercise 6: Decreasing Edge Weights

- Given a graph  $G$  and a minimum spanning tree  $T$ , suppose that we decrease the weight of one of the edges not in  $T$
- Give an algorithm for finding the minimum spanning tree in the modified graph
  - Do not run an MST algorithm on the modified graph



기존 MST를 새로 계산하지 않고, MST에 포함되지 않았던 edge의 weight가 줄어들었을 때, 새로운 MST를 빠르게 업데이트하는 방법.



weight가 줄어든 edge를 넣음. -> 그럼 cycle이 생기겠네?

cycle 안에서 가장 가중치가 큰 간선을 제거하면 된다! >>  $T'$

## 14주차 - Shortest Path Algorithm

directed graph  $G=(V,E)$ 와 그 weight function  $w: E \rightarrow \mathbb{R}$ (각 간선에 대해 실수 할당됨)이 주어졌을 때

- The **weight of path**  $p = \langle v_0, v_1, \dots, v_k \rangle$  is computed by

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

>> 경로의 가중치는 경로를 지나가는 edge들의 weight의 합.

- The **shortest-path weight**  $\delta(u, v)$  from  $u$  to  $v$

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \xrightarrow{p} v\} & \text{if there is a path from } u \text{ to } v, \\ \infty & \text{otherwise.} \end{cases}$$

>> 델타(u,v)에서 u->v까지 경로가 있다면 경로중 최솟값  
>> 없다면 무한.

- A **shortest path** from vertex  $u$  to vertex  $v$  is defined as

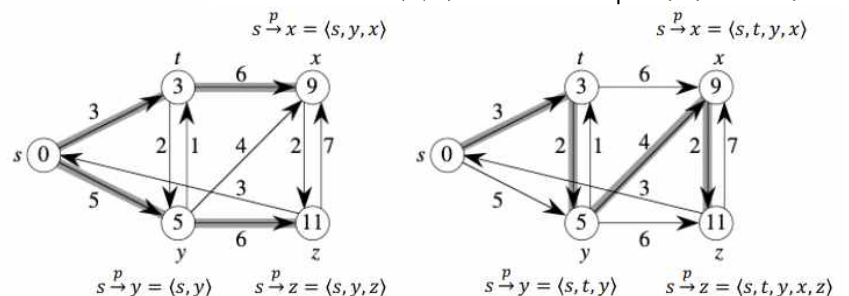
any path  $p$  with weight  $w(p) = \delta(u, v)$

>> u->v 경로 중  $(u, v)$ 인 경로. unique하지 않을 수도!

Source Vertex  $s$ 로부터의 최단 거리

MST처럼 unique하지 않을 수 있다!

----->



### □ Single-Source Shortest-Paths (SSSP) Problem

- To find a shortest path from a given **source** vertex  $s \in V$  to each vertex  $v \in V$  source vertex  $s$ 에서 시작해 모든 vertex까지의 최단 경로 구하기

### □ Single-Destination Shortest-Paths Problem

- To find a shortest path to a given **destination** vertex  $t \in V$  from each node  $v \in V$  모든 vertex에서 시작해 도착 vertex  $t$ 까지의 최단 경로 구하기

### □ Single-Pair Shortest-Path Problem

- To find a shortest path from  $u$  to  $v$  for **a given pair** of vertices  $u$  and  $v$

### □ All-Pair Shortest-Paths Problem

- To find a shortest path from  $u$  to  $v$  for **every pair** of vertices  $u$  and  $v$

#### 최단거리 : 조건

$s \rightarrow g$  까지 최단 경로를 정의할 수 있을까?

모든 정점이 음수여야 할 필요는 없음  $\rightarrow$  음수여도 됨!

하지만 **Negative-weight cycle**이 있으면 안 됨!

#### □ Positive-weight cycle

- No reason to pass the cycle, just omit the cycle

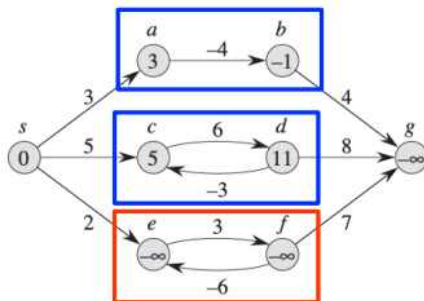
>> 최단거리니 통과할 이유가 없음

#### □ Negative-weight cycle

- We can reduce the path weight  $w(s, g)$  by passing through the cycle repeatedly

- However, we get  $w(s, g) = -\infty$  for all  $v$  on the cycle

>> 사이클을 통과할수록  $p$ 의 weight가 작아짐.. 문제다!



Single-source shortest paths can be defined if there are **NOT** any negative-weight cycles reachable from the source

#### Lemma 1 Shortest path의 subpath 역시 shortest path이다. - Optimal Substructure

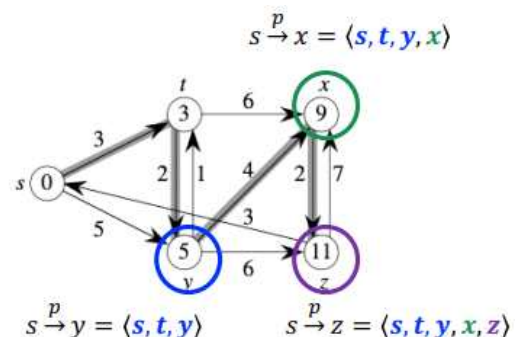
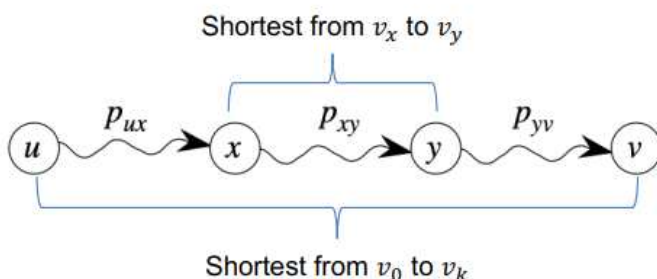
directed graph  $G=(V,E)$ 와 그 weight function  $w: E \rightarrow \mathbb{R}$ (각 간선에 대해 실수 할당됨)이 주어졌을 때 부터  $v_k$ 까지의 최단 경로를  $p = \langle v_0, v_1, \dots, v_k \rangle$ 라 하자.

$$v_0 = u \text{ and } v_k = v$$

0  $x$   $y$   $k$ 인  $x, y$ 가 있다고 할 때,  $p_{xy} = \langle v_x, v_{x+1}, \dots, v_y \rangle$ 를  $v_x$ 부터  $v_y$ 까지의  $p$ 의 subpath라고 하면

$p_{xy}$ 는  $v_x$ 부터  $v_y$ 까지의

최단 경로이다.





## 증명

p를 세 개 부분으로 나눈다.

$$= p_x + p_{xy} + p_{yv}$$

전체 경로는  $u \rightarrow x \rightarrow y \rightarrow v$  이다.

$$w(p) = w(p_{ux}) + w(p_{xy}) + w(p_{yv})$$

$w(p'_{xy}) < w(p_{xy})$  인  $p'_{xy}$  가 있다고 가정하자.

그러면  $w(p)$ 보다 작은 path가 생긴다. >> 최단 경로보다 짧은 경로가 생겨버렸네? 모순이다.

■ Then,  $u \xrightarrow{p_{ux}} x \xrightarrow{p'_{xy}} y \xrightarrow{p_{yv}} v$  is a path whose weight is less than  $w(p)$

$$\square w(p_{ux}) + w(p_{xy}) + w(p_{yv}) > w(p_{ux}) + w(p'_{xy}) + w(p_{yv})$$



We can get shortest paths from the source to other vertices based on the previously computed shortest paths

∴ 아하  $p_{xy}$  는 최단경로이구나! >

이전에 구한 최단 경로정보를 활용해 이후의 최단 경로를 계산할 수 있구나!

## 최단거리를 어떻게 출력할 것인가?

SSSP 알고리즘의 출력을 어떻게 표현할 것인가?

1. 최단 거리의 weight

각 vertex까지의 거리  $\delta(s, v)$

2. 최단 경로 상의 vertices

## predecessor 서브그래프 : 최단거리 Tree

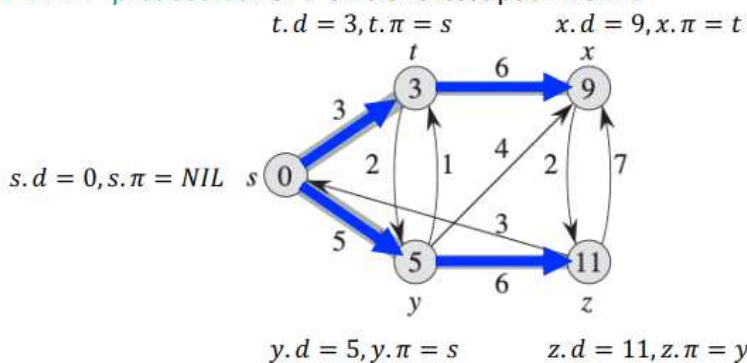
각각의 vertex는 두 가지 attributes가 있다.

□  $v.d = \text{Shortest-path estimate}$  ( $v.d \geq \delta(s, v)$ )

<- v.d 는 estimate라 계속 줄어든다.

■ i.e.,  $v.d$  reduces as a SSSP algorithm progresses until  $v.d = \delta(s, v)$

□  $v.\pi = \text{predecessor}$  of  $v$  on a shortest path from  $s$



## SSSP Single Source Shortest Path 알고리즘 프로세스

SSSP 알고리즘의 General Process

1. 초기화 Initialization : INIT-SINGLE-SOURCE 함수로 시작된다

2. 완화 Relaxation : 그런 다음 반복적으로 더 짧은 경로를 발견하면 거리 추정치를 update.

### 초기화 Initialization

>> 모든 Shortest path는 이렇게 시작함

각각의 vertex와 predecessor를 초기화한다

$v.d$ 는  $\infty$

$v.\pi$ 는 NIL(비어 있음)

$s.d$

시작 vertex의  $d$ 는 0으로 초기화.

INIT-SINGLE-SOURCE( $G, s$ )

for each  $v \in G.V$

$v.d = \infty$

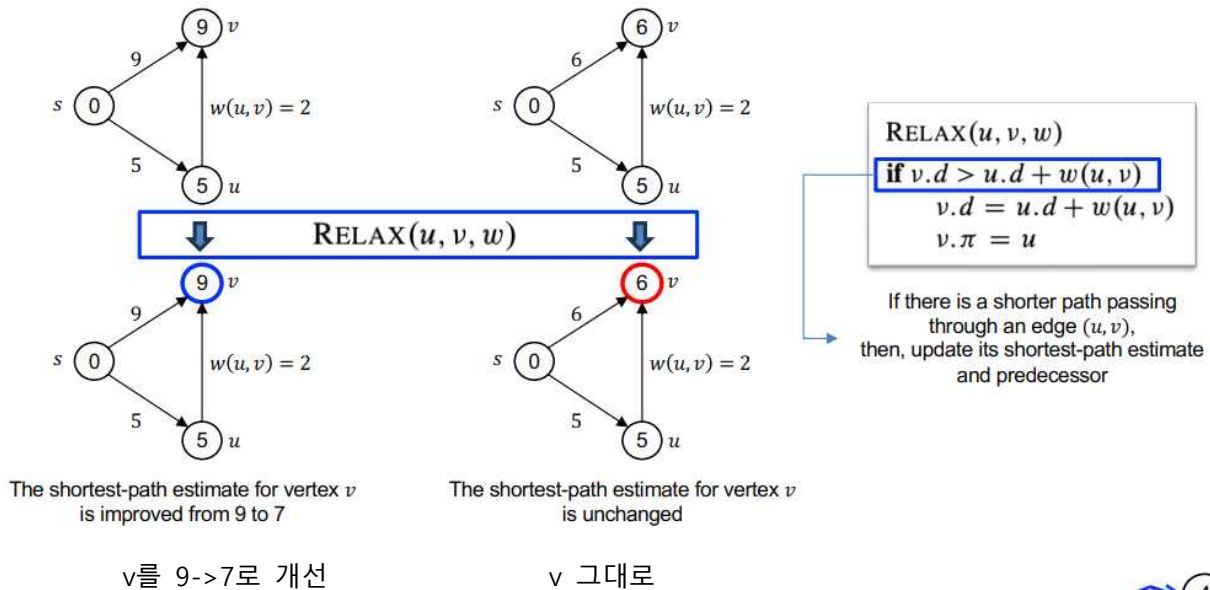
$v.\pi = \text{NIL}$

$s.d = 0$



## 완화 Relaxation

edge  $(u,v)$ 을 relaxing함으로서 vertex  $v$ 의 Shortest path estimate을 향상시킴.



## Shortest Path의 3가지 핵심 성질

### 1. Triangle Inequality 삼각부등식

$$\delta(s, v) \leq \delta(s, u) + w(u, v)$$

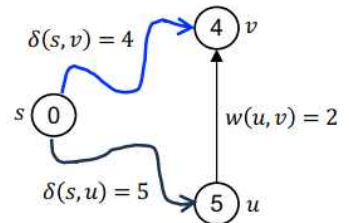
모든 edge  $(u,v)$ 에 대해서,  $s \rightarrow v$ 까지의 최단 경로는  $s \rightarrow u \rightarrow v$  경로(우회하는 것보다)를 거쳤을 때보다 짧거나 같다.

### 2. Upper-Bound Property 상한 특성

모든 vertex  $v$ 에 대해 현재의 거리 추정값  $v.d$ 는,  $\delta(s, v)$ 보다 크거나 같다  
 그리고 한번  $v.d = \delta(s, v)$ 가 되면 바뀌지 않는다.

### 3. Path Relaxation Property 경로 완화 성질

만약  $p = \langle v, v_1, \dots, v_k \rangle$ 가  $v_0$  서  $v_k$ 까지의 최단 경로라면,  
 이 경로의 vertices를 순서대로 Relax하면 최종적으로  $v_k.d = \delta(s, v_k)$ 가 된다.



## Dijkstra's Algorithm : Solving the SSSP Problem

source에서부터 가장 가까운 vertex들을 선택하는 **Greedy Approach**를 사용한다.

**모든 edge는 non-negative여야 한다!** -> negative가 있으면 bellman-ford 사용해야 함.

INIT-SINGLE-SOURCE( $G, s$ )

**for** each  $v \in G.V$   
 $v.d = \infty$   
 $v.\pi = \text{NIL}$   
 $s.d = 0$

RELAX( $u, v, w$ )

**if**  $v.d > u.d + w(u, v)$   
 $v.d = u.d + w(u, v)$   
 $v.\pi = u$

DIJKSTRA( $G, w, s$ )

INIT-SINGLE-SOURCE( $G, s$ )

$S = \emptyset$

$Q = G.V$

**while**  $Q \neq \emptyset$

$u = \text{EXTRACT-MIN}(Q)$

$S = S \cup \{u\}$

**for** each vertex  $v \in G.Adj[u]$

RELAX( $u, v, w$ )

//  $S$ 는 최단거리 확정된 vertices 집합

// Priority queue is sorted in the increasing order by the distance from the source

// the closest vertex to the source is picked

//인접 vertices에 대해 Relaxing 실행!

$u = \text{Extract-Min}(Q)$ 를 하면  $s$ 가 나옴

**DIJKSTRA**( $G, w, s$ )

**INIT-SINGLE-SOURCE**( $G, s$ )

$S = \emptyset$  //  $S$ 는 최단거리 확정된 vertices 집합

$Q = G.V$  // 거리순으로 정렬해놓은 Priority Queue

**while**  $Q \neq \emptyset$

$u = \text{EXTRACT-MIN}(Q)$

$S = S \cup \{u\}$

**for each vertex**  $v \in G.\text{Adj}[u]$

**RELAX**( $u, v, w$ )

// 정렬되고 EXTRACT 하니 최솟값 나옴

**INIT-SINGLE-SOURCE**( $G, s$ )

**for each**  $v \in G.V$

$v.d = \infty$

$v.\pi = \text{NIL}$

$s.d = 0$

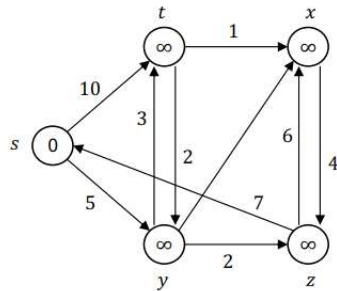
**RELAX**( $u, v, w$ )

**if**  $v.d > u.d + w(u, v)$

$v.d = u.d + w(u, v)$

$v.\pi = u$

| $s$ | $t$      | $y$      | $x$      | $z$      |
|-----|----------|----------|----------|----------|
| 0   | $\infty$ | $\infty$ | $\infty$ | $\infty$ |

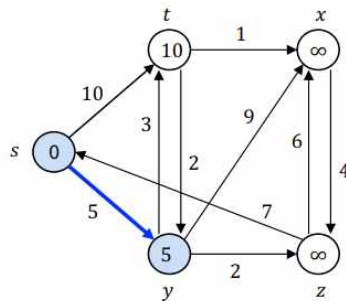


| $s$ | $t$      | $y$      | $x$      | $z$      |
|-----|----------|----------|----------|----------|
| 0   | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
|     | 10       | 5        | -        | -        |

$S = \{s\}$

$Q = \{s, t, x, y, z\} \rightarrow$  Sorted by distance

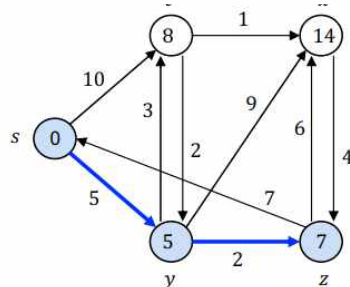
| $s$ | $t$      | $y$      | $x$      | $z$      |
|-----|----------|----------|----------|----------|
| 0   | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
|     | 10       | 5        | -        | -        |



$S = \{s, y\} \rightarrow y$  is finished

$Q = \{t, x, z\}$

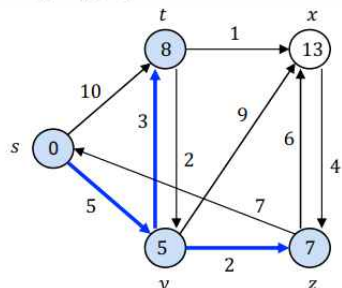
| $s$ | $t$      | $y$      | $x$      | $z$      |
|-----|----------|----------|----------|----------|
| 0   | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
|     | 10       | 5        | -        | -        |
|     | 8        |          | 14       | 7        |



$S = \{s, y, z\} \rightarrow z$  is finished

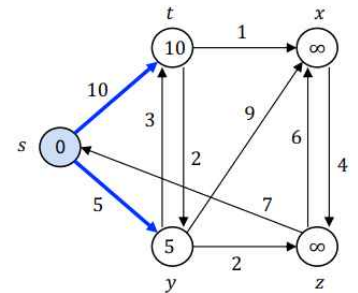
$Q = \{t, x\}$

| $s$ | $t$      | $y$      | $x$      | $z$      |
|-----|----------|----------|----------|----------|
| 0   | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
|     | 10       | 5        | -        | -        |
|     | 8        |          | 14       | 7        |
|     | 8        |          | 13       |          |



$S = \{s, y, z, t\} \rightarrow t$  is finished

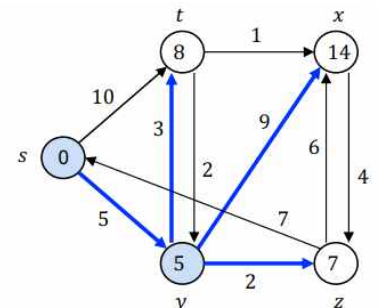
$Q = \{x\}$



$S = \{s\}$

$Q = \{y, t, x, z\} \rightarrow$  Re-sorted

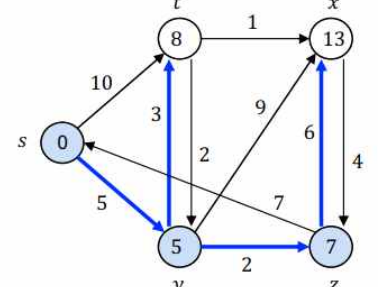
| $s$ | $t$      | $y$      | $x$      | $z$      |
|-----|----------|----------|----------|----------|
| 0   | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
|     | 10       | 5        | -        | -        |
|     | 8        |          | 14       | 7        |



$S = \{s, y\}$

$Q = \{z, t, x\} \rightarrow$  Re-sorted

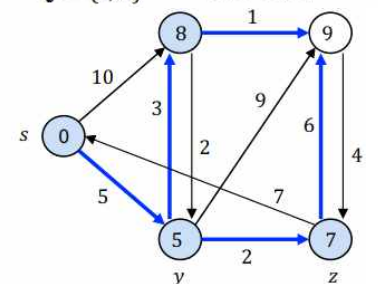
| $s$ | $t$      | $y$      | $x$      | $z$      |
|-----|----------|----------|----------|----------|
| 0   | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
|     | 10       | 5        | -        | -        |
|     | 8        |          | 14       | 7        |
|     | 8        |          | 13       |          |



$S = \{s, y, z\}$

$Q = \{t, x\} \rightarrow$  Re-sorted

| $s$ | $t$      | $y$      | $x$      | $z$      |
|-----|----------|----------|----------|----------|
| 0   | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
|     | 10       | 5        | -        | -        |
|     | 8        |          | 14       | 7        |
|     | 8        |          | 13       |          |
|     |          |          | 9        |          |



$S = \{s, y, z, t\}$

$Q = \{x\}$

| s | t        | y        |
|---|----------|----------|
| 0 | $\infty$ | $\infty$ |
|   | 10       | 5        |
|   | 8        |          |
|   | 8        |          |
|   |          |          |

INIT-SINGLE-SOURCE( $G, s$ )

for each  $v \in G.V$

$v.d = \infty$

$v.\pi = \text{NIL}$

$s.d = 0$

RELAX( $u, v, w$ )

if  $v.d > u.d + w(u, v)$

$v.d = u.d + w(u, v)$

$v.\pi = u$

BELLMAN-FORD( $G, w, s$ )

INIT-SINGLE-SOURCE( $G, s$ )

for  $i = 1$  to  $|G.V| - 1$

for each edge  $(u, v) \in G.E$

RELAX( $u, v, w$ )

for each edge  $(u, v) \in G.E$

if  $v.d > u.d + w(u, v)$

return FALSE

return TRUE

To check negative-weight cycles

$Q = \{\}$

## Dijkstra's Algorithm 복잡도 분석

시간복잡도 : priority queue의 구현에 따라서 달라짐

배열 기반 구현

$O(V^2)$

각각의 연산이  $V$ 번씩 걸리기 때문.

Binary heap-based 구현

$O(V \lg V + E \lg V)$

-> 각각의 연산이  $O(\lg V)$ 들로 이루어져있기 때문

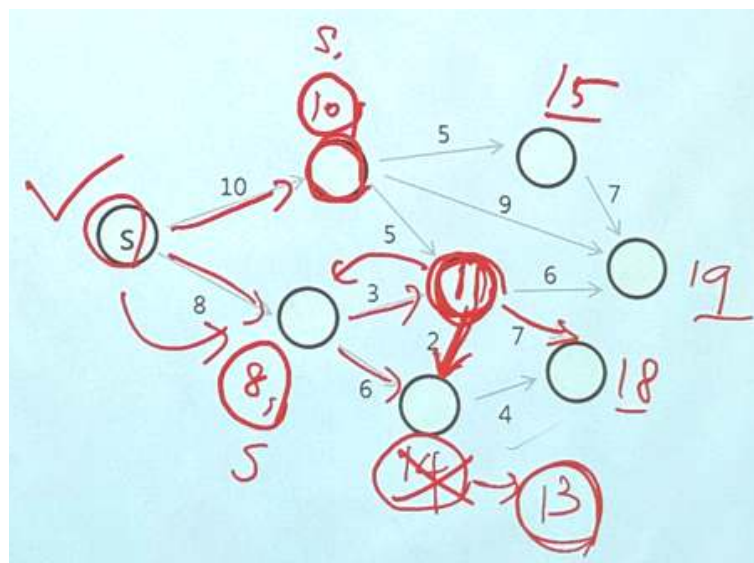
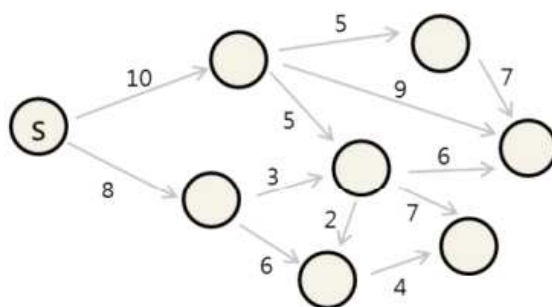
Extract-Min( $Q$ ) :  $O(\lg V)$

Decrease-Key( $Q, v$ ) :  $O(\lg V)$

building min heap :  $O(V)$

## Exercise 1: Dijkstra's Algorithm

- Run Dijkstra's algorithm on the directed graph, using vertex  $s$  as the source
- Show the  $v.d$  and  $v.\pi$  values



## Bellman-Ford's Algorithm

shortest-path problem에서 쓸 수 있는 general한 알고리즘  
edge가 negative여도 사용할 수 있음!

INIT-SINGLE-SOURCE( $G, s$ )

for each  $v \in G.V$   
 $v.d = \infty$   
 $v.\pi = \text{NIL}$   
 $s.d = 0$

RELAX( $u, v, w$ )

if  $v.d > u.d + w(u, v)$   
 $v.d = u.d + w(u, v)$   
 $v.\pi = u$

BELLMAN-FORD( $G, w, s$ )

INIT-SINGLE-SOURCE( $G, s$ )

for  $i = 1$  to  $|G.V| - 1$

for each edge  $(u, v) \in G.E$

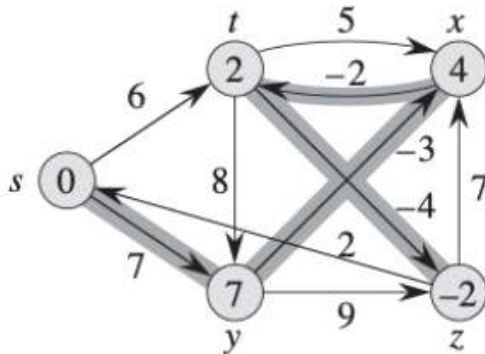
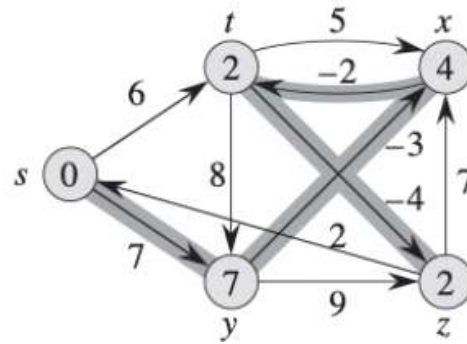
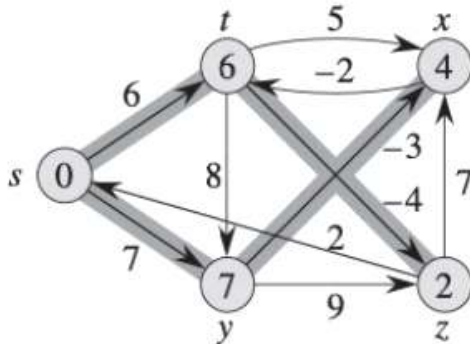
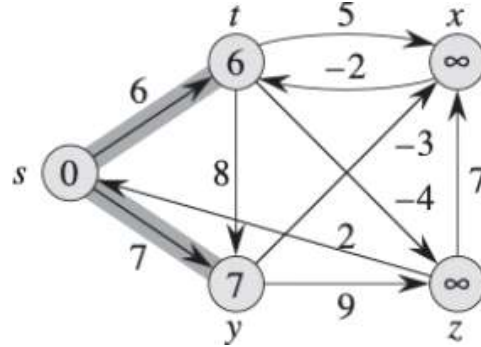
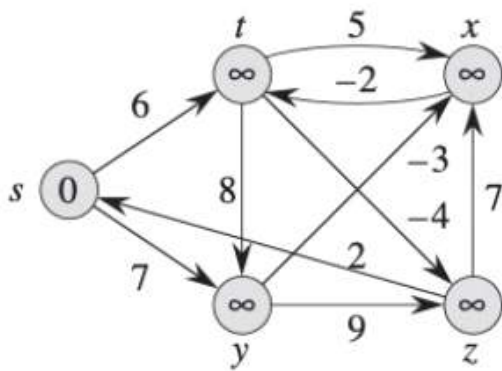
RELAX( $u, v, w$ )

for each edge  $(u, v) \in G.E$

if  $v.d > u.d + w(u, v)$   
 return FALSE

To check negative-weight cycles

return TRUE



## Bellman-ford Algorithm 복잡도 분석

시간복잡도 :  $O(VE)$  - 모든 노드의 각 edge relaxing하기  
 $O(E)$  - negative-weight cycle이 있는지 체크

정확도 : 이 알고리즘이 제대로된 결과를 나타내나?

>  $v.d = (s, v)$ 와 가 shortest paths tree인가?

이 알고리즘이 제대로된 값을 return하는가?

> negative-weight cycle이 없으면 TRUE.

BELLMAN-FORD( $G, w, s$ )

INIT-SINGLE-SOURCE( $G, s$ )

for  $i = 1$  to  $|G.V| - 1$

for each edge  $(u, v) \in G.E$

RELAX( $u, v, w$ )

for each edge  $(u, v) \in G.E$

if  $v.d > u.d + w(u, v)$

return FALSE

return TRUE

$O(V)$

$O(VE)$

$O(E)$



## Bellman-ford 의 정확도

가중치가 있는 directed graph  $G=(V,E)$  그리고 source  $s$ 와 weight function으로 Bellman-ford를 실행한다고 하자  
 $s$ 로 도달 가능한 vertices 사이에 **negative-weight cycle**이 없으면, 알고리즘은 **TRUE**를 반환한다.

-> 이때 모든 vertex  $v$ 에 대해  $v.d = \delta(s,v)$ 가 성립하며, predecessor subgraph 는  $s$ 를 root로 하는 MST이다  
 $s$ 로 도달 가능한 vertices 사이에 **negative-weight cycle**이 있으면, 알고리즘은 **FALSE**를 반환한다.

### Lemma 1

weight directed graph  $G(V,E)$ 와 source  $s$ 와 weight function  $w$ 가 있다고 하자

$G$ 는 negative-weight cycle을 포함하지 않는다고 하자.

$|V|-1$ 번만큼 반복을 돌린다. 모든 vertices  $v$ 에 대해  $v.d = \delta(s, v)$ 이다.

### Lemma 1 증명

$p = \langle v_0, v_1, \dots, v_k \rangle$  가  $s \rightarrow v$ 까지의 임의의 최단 경로라고 하자.

$p$ 는 최대  $|V|-1$  개의 edge를 가진다. ( $k \leq |V| - 1$ ) ----  $k$ 는 index변수  
 simple(한 vertex에 두 번 이상 못오기) 때문!

모든  $|E|$  edge는 2-4줄에서 relax된다.

모든 vertices  $v$ 에 대해  $v.d = \delta(s, v)$ 이다.

**path Relaxation Property** : 만약  $p = \langle v_0, v_1, \dots, v_k \rangle$  가  $s$ 로부터  $v_k$ 까지의 최단 경로이고, 그 경로의 상의 간선들을  $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$  순서로 relax하면 최종적으로  $v_k.d = \delta(s, v_k)$ 가 성립한다.

```
BELLMAN-FORD( $G, w, s$ )
INIT-SINGLE-SOURCE( $G, s$ )
for  $i = 1$  to  $|G.V| - 1$ 
  for each edge  $(u, v) \in G.E$ 
    RELAX( $u, v, w$ )
for each edge  $(u, v) \in G.E$ 
  if  $v.d > u.d + w(u, v)$ 
    return FALSE
return TRUE
```

**Case 1** :  $G$ 에 Negative-Weight Cycle이 없음.

$$\begin{aligned} v.d &= \delta(s, v) \\ &\leq \delta(s, u) + w(u, v) \quad (\text{by the triangle inequality}) \\ &= u.d + w(u, v), \end{aligned}$$

FALSE가 반환될 일이 없음 -> 항상 TRUE만 반환한다.

**Case 2** :  $G$ 에 Negative-Weight Cycle이 있음.

negative weight cycle이 없다면 Sigma가 항상 0과 같거나 커야 한다.

사이클  $c = \langle v_0, v_1, \dots, v_k \rangle$  이 존재하며  $v_0 = v_k$ 라 하자. (한바퀴 돌고 출발점으로 돌아옴)

■ Let  $c = \langle v_0, v_1, \dots, v_k \rangle$  be any cycle ( $v_0 = v_k$ )  $\rightarrow \sum_{i=1}^k w(v_{i-1}, v_i) < 0$ .

$$\begin{aligned} \sum_{i=1}^k v_i.d &\leq \sum_{i=1}^k (v_{i-1}.d + w(v_{i-1}, v_i)) \quad \text{Can it return TRUE?} \\ &= \sum_{i=1}^k v_{i-1}.d + \sum_{i=1}^k w(v_{i-1}, v_i). \quad \sum_{i=1}^k v_i.d = \sum_{i=1}^k v_{i-1}.d. \\ &\quad \text{Contradiction!} \end{aligned}$$

### 증명 흐름

1. 사이클 전체에 relax를 한 번씩 적용한다고 가정해 봅시다.  
 각 단계마다

$$v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i)$$

이 성립해야 하죠.

("relax 조건을 만족하면 값을 갱신하거나 같게 유지")

2. 이걸  $i = 1$ 부터  $k$ 까지 더하면,

$$\sum_{i=1}^k v_i.d \leq \sum_{i=1}^k (v_{i-1}.d + w(v_{i-1}, v_i))$$

3. 우변을 전개하면

$\sum v_i.d + \sum w(v_{i-1}, v_i)$ 가 되고,  
 좌변에도  $\sum v_i.d$ 가 있으니 빼면

$$0 \leq \sum_{i=1}^k w(v_{i-1}, v_i).$$

즉 "사이클 전체 비용은 0 이상"이 되어야 합니다.

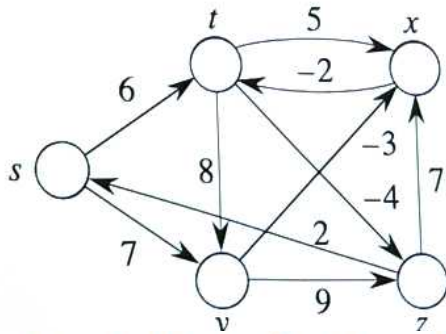
4. 하지만 음수 사이클이면 위 합이 0보다 작아야 하므로

$$0 \leq \sum w(\dots) < 0$$

▶ 모순(Contradiction)!

## Exercise: Bellman-Ford Algorithm

- Run the Bellman-ford algorithm on the following directed graph, using vertex  $z$  as the source
  - In each pass, relax edges in the same order
- Show the  $v.d$  and  $v.\pi$  values after each pass

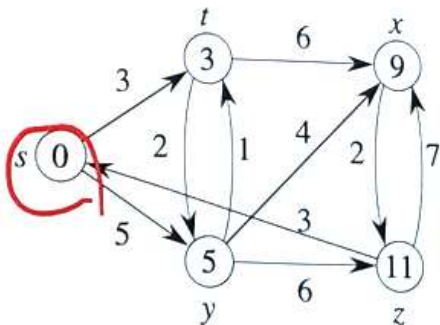
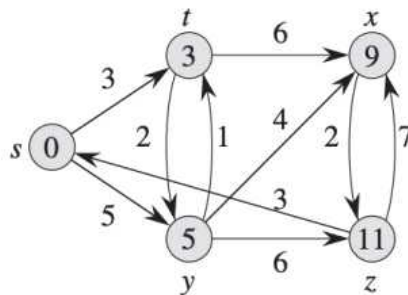


| $s$      | $t$      | $x$      | $y$      | $z$ |
|----------|----------|----------|----------|-----|
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | 0   |
| 2        | $\infty$ | 7        | $\infty$ | 0   |
| 2        | 5        | 7        | 9        | 0   |
| 2        | 5        | 6        | 9        | 0   |
| 2        | 4        | 6        | 9        | 0   |

| $s$ | $t$ | $x$ | $y$ | $z$ |
|-----|-----|-----|-----|-----|
| NIL | NIL | NIL | NIL | NIL |
| z   | NIL | z   | NIL | NIL |
| z   | x   | z   | s   | NIL |
| z   | x   | y   | s   | NIL |
| z   | x   | y   | s   | NIL |

## Exercise: Dijkstra's Algorithm

- Run Dijkstra's algorithm on the directed graph, using vertex  $s$  as the source
- Show the  $v.d$  and  $v.\pi$  values



| $s$ | $t$ | $x$      | $y$ | $z$      |
|-----|-----|----------|-----|----------|
| 0   | 3   | $\infty$ | 5   | $\infty$ |
| 0   | 3   | 9        | 5   | $\infty$ |
| 0   | 3   | 9        | 5   | 11       |
| 0   | 3   | 9        | 5   | 11       |
| 0   | 3   | 9        | 5   | 11       |

| $s$ | $t$ | $x$ | $y$ | $z$ |
|-----|-----|-----|-----|-----|
| NIL | s   | NIL | NIL | NIL |
| NIL | s   | t   | s   | NIL |
| NIL | s   | t   | s   | y   |
| NIL | s   | t   | s   | y   |
| NIL | s   | t   | s   | y   |

음이 아닌 가중치 방향 그래프에서의 SSSP

### Dijkstra's Algorithm

**Greedy approach**로 시작점에서부터 가장 가까운 정점을 순서대로 선택한다.

-> 모든 edge가 non-negative일 때만 정상적으로 동작

### Dijkstra's Algorithm 시간 복잡도

priority queue 구현에 따라 시간복잡도가 달라진다

배열기반 구현 시

$O(V^2)$

binary heap기반 구현 시

$O(V \lg V + E \lg V)$

- Each operation takes  $O(\lg V)$
- Extract-Min(Q):  $O(\lg V)$
- Decrease-Key(Q, v):  $O(\lg V)$
- Build-Min-Heap:  $O(V)$

DIJKSTRA( $G, w, s$ )

INIT-SINGLE-SOURCE( $G, s$ )

$S = \emptyset$

$Q = G.V$

**while**  $Q \neq \emptyset$

$u = \text{EXTRACT-MIN}(Q)$

$S = S \cup \{u\}$

**for each** vertex  $v \in G.Adj[u]$

RELAX( $u, v, w$ )

## 15주차 - All-Pairs Shortest-Paths

### SSSP in Directed Acyclic Graphs (DAG)

명칭 상 negative-weight cycle이 존재할 수 없음.

DAG-SHORTEST-PATHS( $G, w, s$ )

1 **topologically sort the vertices of  $G$**

그래프  $G$ 의 정점들을 위상 정렬한다

2 INITIALIZE-SINGLE-SOURCE( $G, s$ )

INITIALIZE-SINGLE-SOURCE( $G, s$ )

3 **for each vertex  $u$ , taken in topologically sorted order**

위상 정렬된 순서대로 각 정점

4 **for each vertex  $v \in G.Adj[u]$**

$u$ 와 그 인접한 vertex에 대해

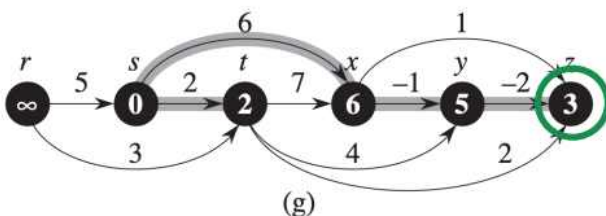
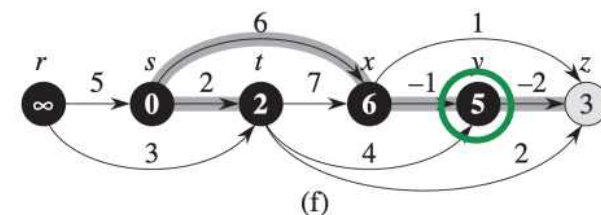
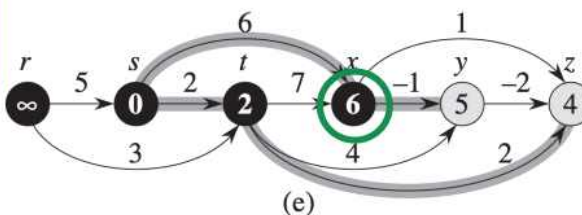
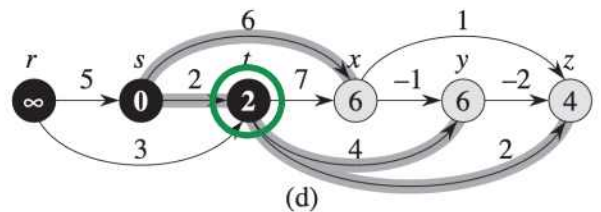
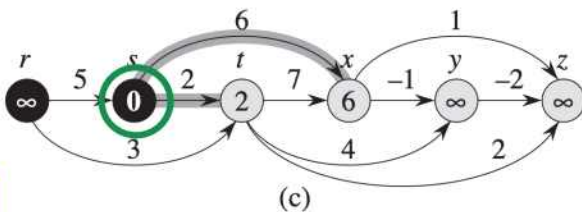
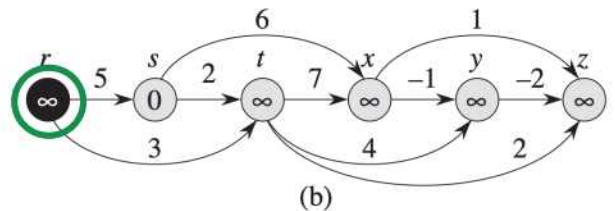
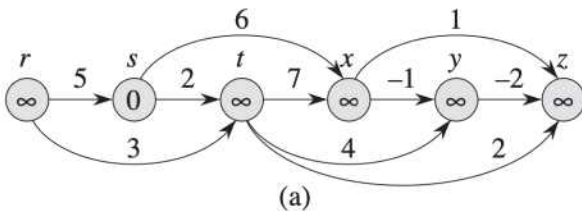
5 RELAX( $u, v, w$ )

RELAX( $u, v, w$ ) 수행



$r$ 에서 시작해  $s, t, x, y, z$  순(위상정렬 한 순)으로 거리를 갱신  
위상 정렬 순서대로 한 번만 relax해도 최단 거리가 확정됨

$s$ 가 source. 시작점이자 기준점





시간 복잡도 :  $O(V+E)$

### DAG-SHORTEST-PATHS( $G, w, s$ )

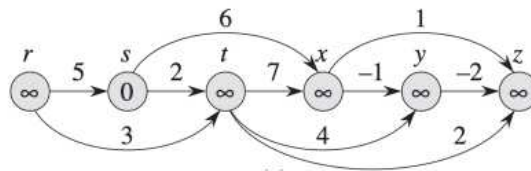
```

1  topologically sort the vertices of  $G$    $O(V + E) \rightarrow DFS$ 
2  INITIALIZE-SINGLE-SOURCE( $G, s$ )   $O(V)$ 
3  for each vertex  $u$ , taken in topologically sorted order
4      for each vertex  $v \in G.Adj[u]$ 
5          RELAX( $u, v, w$ )

```

$O(V + E)$

(by aggregate analysis)



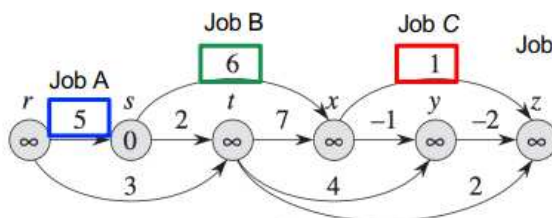
### PERT(Program Evaluation and Review Technique) Chart

edge : 수행해야 할 작업

edge weight : 수행하는데 걸리는 시간

path : 특정 순서대로 수행해야 할 작업의 연속

critical path : DAG를 통과하는 가장 긴 경로! 작업을 마치는데 필요한 최소 소요 시간의 lower bound 제공



Job B (edge (s,x)) must be performed before Job C (edge (x,z))

Q. How can we find a critical path?

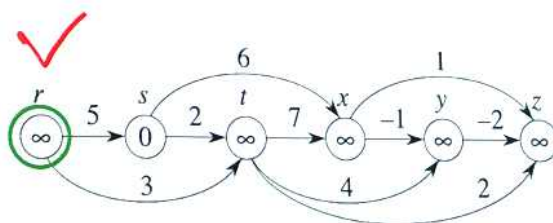
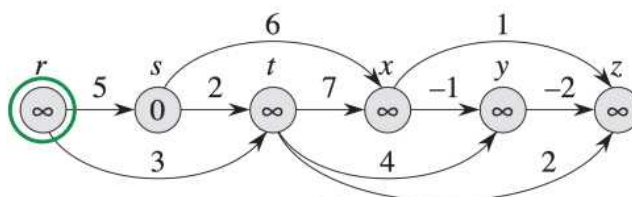
Job A (edge (r,s)) must be performed before Job B (edge (s,x))

Q) 어떻게 critical path를 찾을 수 있을까?

>> A) Shortest path 찾는 방법에서 부호를 뒤집으면 됨.

### Exercise: DAG-Shortest-Paths

- Run DAG-Shortest-Paths on the directed graph, using vertex  $r$  as the source
- Show the  $v.d$  and  $v.\pi$  values after each pass



| $r$ | $s$      | $t$      | $x$      | $y$      | $z$      |
|-----|----------|----------|----------|----------|----------|
| 0   | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| 0   | 5        | 3        | $\infty$ | $\infty$ | $\infty$ |
| 0   | 5        | 3        | 11       | $\infty$ | $\infty$ |
| 0   | 5        | 3        | 10       | 7        | 5        |
| 0   | 5        | 3        | 10       | 7        | 5        |
| 0   | 5        | 3        | 10       | 7        | 5        |

| $r$ | $s$ | $t$ | $x$ | $y$ | $z$ |
|-----|-----|-----|-----|-----|-----|
| NIL | NIL | NIL | NIL | NIL | NIL |
| NIL | $r$ | $r$ | NIL | NIL | NIL |
| NIL | $r$ | $r$ | $s$ | NIL | NIL |
| NIL | $r$ | $r$ | $t$ | $t$ | $t$ |
| NIL | $r$ | $r$ | $t$ | $t$ | $t$ |
| NIL | $r$ | $r$ | $t$ | $t$ | $t$ |



## All-Pairs Shortest Paths Problem

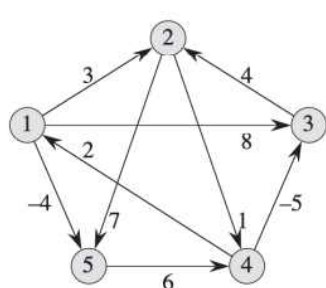
### Given

directed graph  $G=(V,E)$ , weight function  $w: E \rightarrow \mathbb{R}$ 이 있을 때,  $|V| = n$ 이고, 정점에 1,2, ..., n 번호가 매겨져 있어 입력 그래프를  $n \times n$ 행렬  $W$ 로 표현할 수 있다.

### Goal

모든 쌍  $(i,j)$ 에 대해 최단경로 길이  $d(i,j)$ 를 담은  $n \times n$  행렬  $D = [d_{ij}]$ ,  $d_{ij} = \delta(i,j)$

그리고 최단경로 상의 바로 Predecessor를 담은  $n \times n$  행렬  $\Pi$



$W$

$$\begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

How can we compute the results?

$$w_{ij} = \begin{cases} 0 & \text{if } i = j, \\ \text{weight of } (i, j) & \text{if } i \neq j, (i, j) \in E, \\ \infty & \text{if } i \neq j, (i, j) \notin E. \end{cases}$$

| Shortest-path distances |    |    |   |    |  |
|-------------------------|----|----|---|----|--|
| 0                       | 1  | -3 | 2 | -4 |  |
| 3                       | 0  | -4 | 1 | -1 |  |
| 7                       | 4  | 0  | 5 | 3  |  |
| 2                       | -1 | -5 | 0 | -2 |  |
| 8                       | 5  | 1  | 6 | 0  |  |

| Shortest-path distances |     |     |     |     |  |
|-------------------------|-----|-----|-----|-----|--|
| NIL                     | 3   | 4   | 5   | 1   |  |
| 4                       | NIL | 4   | 2   | 1   |  |
| 4                       | 3   | NIL | 2   | 1   |  |
| 4                       | 3   | 4   | NIL | 1   |  |
| 4                       | 3   | 4   | 5   | NIL |  |

### APSP를 SSSP를 사용해서 해결하는 방법

모든 정점을 source 정점으로 치고  $|V|$ 회수만큼 SSSP를 하면 APSP를 풀 수 있다.

### Non-Negative Edges가 있는 그래프

Dijkstra's algorithm :  $(V^2 \lg V + VE \lg V)$

만약 그래프가 dense하다면,  $O(V^3 \lg V)$ 가 될 것이다

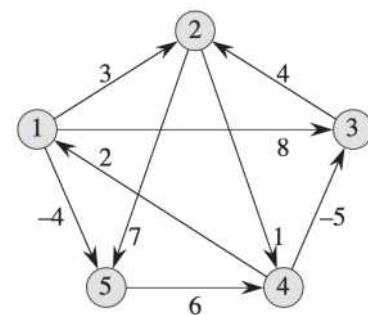
### Negative Edges가 있는 그래프

Bellman-Ford's algorithm :  $O(V^2 E)$

만약 그래프가 dense하다면,  $O(V^4)$ 가 될 것이다

★ 그래프가 dense 하다는 것 :  $E$ 가  $V(V-1)$ 이라는 것  $\rightarrow E = O(V^2)$

>>>  $O(V^3)$ 정도로 개선할 수 는 없을까?



Run Bellman-Ford's algorithm 5 times

### ASAP 해결 방법

#### Shortest Path 길이의 Upper bound

최단경로는 최대  $|V|-1$ 개의 edge로 이루어진다.

#### Naive한 해결방법

edge 1개를 포함한 모든 최단경로를 계산한다.

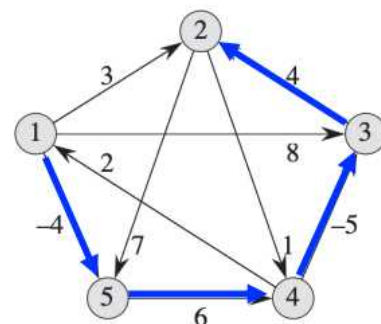
edge 2개를 포함한 모든 최단경로를 계산한다.

edge 3개를 포함한 모든 최단경로를 계산한다.

...

edge  $n-1$ 개를 포함한 모든 최단경로를 계산한다.

>> 모든  $(i,j)$ 에 대해 최대 edge개수는  $n-1$ 이다.



The length of the shortest path  $p_{1,2}$  is 4

어떻게 하면  $m$ 개의 edge를 가진 최단경로를 계산할 수 있을까?

$l_{ij}^{(m)}$ 은  $n \times n$  matrix이고,

$l_{ij}^{(m)}$ 은 정점  $i$ 에서  $j$ 로 가는 경로 중 edge의 개수가 최대  $m$ 개인 것들 중에서 weight값이 최소인 값을 뜻함.

그래서  $L^{(1)} = W$ 이고,  $W$ 는 adjacency matrix로, 각 원소에는 edge( $i, j$ )까지의 weight  $w_{ij}$ 가 담겨 있다.

만약 경로가 존재하지 않으면 무한.

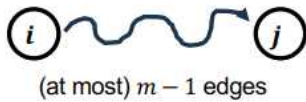
$L^{(m)} = l_{ij}^{(m)}$ 은 아래와 같은 방정식으로 나타낼 수 있다.

$$\begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

$$l_{ij}^{(m)} = \min \left( l_{ij}^{(m-1)}, \min_{1 \leq k \leq n} \{ l_{ik}^{(m-1)} + w_{kj} \} \right) \text{ Relaxation}$$

The minimum weight of a path from  $i$  to  $j$  consisting of at most  $m-1$  edges

The minimum weight of a path from  $i$  to  $j$  consisting of at most  $m$  edges, while passing through the vertex  $k$

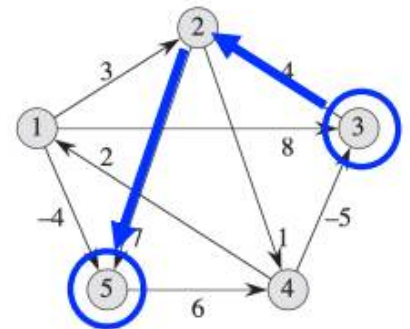


"최대  $m$ 개"라는 조건을 두 가지 경우로 나눠 생각합니다.

$$l_{ij}^{(m)} = \min \left( \underbrace{l_{ij}^{(m-1)}}_{\text{(A) 이미 최대 } m-1 \text{ 개로 가장 짧게 왔던 경우}}, \underbrace{\min_{1 \leq k \leq n} (l_{ik}^{(m-1)} + w_{kj})}_{\text{(B) 마지막에 정점 } k \text{ 를 거쳐 간선 하나 추가한 경우}} \right).$$

• **(A)**는 "굳이  $m$ 번째 간선을 안 쓰고, 이미 계산된 최대  $m-1$ 개짜리 최단거리"

• **(B)**는 "우선 최대  $m-1$ 개로  $i \rightarrow k$ 까지 왔던 최단거리  $l_{ik}^{(m-1)}$ 에, 마지막으로 한 개 간선 ( $k \rightarrow j$ )의 가중치  $w_{kj}$ 를 더한 값"을 모든  $k$ 에 대해 계산해 그중 최소를 택한 것



$$\begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 0 & 3 & 8 & 2 & -4 \\ 3 & 0 & -4 & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & \infty & 1 & 6 & 0 \end{pmatrix}$$

$L^{(1)} = W \qquad \qquad \qquad W \qquad \qquad \qquad L^{(2)} = L^{(1)} \cdot W$

Similar to matrix multiplication  $l_{35}^{(2)} = \min((\infty - 4), (4 + 7), (0 + \infty), (\infty + \infty), (\infty + 0)) = 11$

$$\begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 0 & 3 & 8 & 2 & -4 \\ 3 & 0 & -4 & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & \infty & 1 & 6 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 0 & 3 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

$L^{(1)} = W \qquad \qquad L^{(2)} = L^{(1)} \cdot W \qquad \qquad L^{(3)} = L^{(2)} \cdot W \qquad \qquad L^{(4)} = L^{(3)} \cdot W$

## Naive Method의 복잡도

Naive Method의 복잡도 :  $O(V^3)$

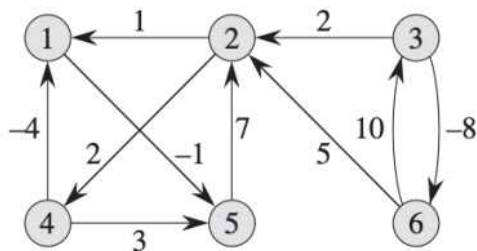
Matrix-Matrix 곱 :  $O(V^3)$

$(m)_{ij}$  계산하는데  $O(V^2)$

이 계산을  $|V|$ 번 반복해야 함!

## Exercise 1: Naïve solution to the APSP problem

- Run the naïve method on the following weighted and directed graph, showing the matrices that result for each iteration. (showing 4 matrices  $L^{(2)} \sim L^{(5)}$ )



$$L^{(1)} = W = \begin{pmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & \infty & \infty \\ \infty & 2 & 0 & \infty & \infty & -8 \\ -4 & \infty & \infty & 0 & 3 & \infty \\ \infty & 7 & \infty & \infty & 0 & \infty \\ \infty & 5 & 10 & \infty & \infty & 0 \end{pmatrix}$$

$$L^{(1)} \quad W \quad \Rightarrow \quad L^{(2)} = \begin{pmatrix} 0 & 6 & \infty & \infty & -1 & \infty \\ -2 & 0 & \infty & 2 & 0 & \infty \\ 3 & -3 & 0 & 4 & \infty & -8 \\ -4 & 10 & \infty & 0 & -5 & \infty \\ 8 & 7 & \infty & 9 & 0 & \infty \\ 6 & 5 & 10 & 7 & \infty & 0 \end{pmatrix}$$

$$L^{(2)} \quad W \quad \Rightarrow \quad L^{(3)} = \begin{pmatrix} 0 & 6 & \infty & 8 & -1 & \infty \\ -2 & 0 & \infty & 2 & 0 & \infty \\ -2 & -3 & 0 & -1 & 2 & -8 \\ -4 & 2 & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 5 & 0 \end{pmatrix}$$

$$L^{(3)} \quad W \quad \Rightarrow \quad L^{(4)} = \begin{pmatrix} 0 & 6 & \infty & 8 & -1 & \infty \\ -2 & 0 & \infty & 2 & -3 & \infty \\ -5 & -3 & 0 & -1 & -3 & -8 \\ -4 & 2 & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 2 & 0 \end{pmatrix}$$

$$L^{(4)} \quad W \quad \Rightarrow \quad L^{(5)} = \begin{pmatrix} 0 & 6 & \infty & 8 & -1 & \infty \\ -2 & 0 & \infty & 2 & -3 & \infty \\ -5 & -3 & 0 & -1 & -6 & -8 \\ -4 & 2 & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 2 & 0 \end{pmatrix}$$

## Floyd-Warshall Algorithm

Dynamic-Programming 기반의 APSP 알고리즘.

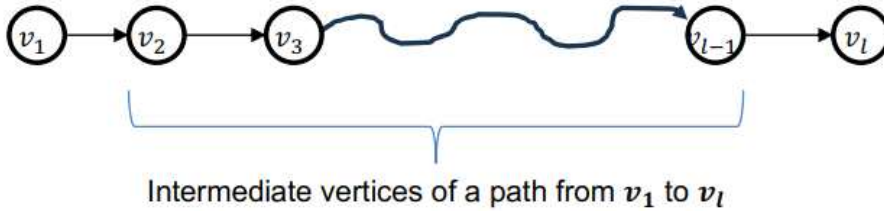
$V$ ) 시간복잡도를 가진다!

negative-weight edge는 있어도 되지만 negative-weight cycle은 있으면 안됨

### 최단경로의 구조 분해(Characterizing the Structure)

Floyd-Warshall 알고리즘은 시작점과 도착점 사이에 어떤 **중간(intermediate)** 정점을 지난다.

**Intermediate vertex** :  $v_1$ 과  $v_l$ 사이에 있는  $\{v_2, v_3, \dots, v_{l-1}\}$  vertex



임의의 두 정점  $i$ 와  $j$ 에 대하여, **중간 정점**이 모두 집합  $\{1, 2, \dots, k\}$ 에 속하는  $i$ 에서  $j$ 로 가는 모든 경로를 고려한다.

>> 중간에 어떤 정점들(1~k번 정점)을 거칠수 있는지를 기준으로 문제를 나눔

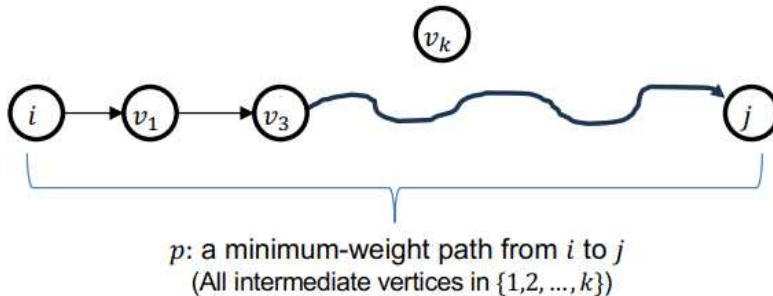
$p$ 를  $i$ 에서  $j$ 로 가는 **최단 경로**라 하고, 그 **경로의 가중치**를  $d_{ij}^{(k)}$ 라 하자.

#### Case1 : 만약 정점 $k$ 가 경로 $p$ 의 중간에 포함되지 않는다면

모든 **중간 정점**은  $\{1, 2, \dots, k-1\}$ 에 속하며,  $\{1, 2, \dots, k\}$ 는 속하지 않는다.

>>  $k$ 번 정점은 쓰지 않았음. 허용 집합을  $\{1, \dots, k-1\}$ 로 줄여도 경로는 그대로일 것이다!

그래서 중간 정점 집합  $\{1, 2, \dots, k-1\}$ 일 때의  $i \rightarrow j$  최단경로와  $\{1, 2, \dots, k\}$ 일 때의 **최단 거리**가 동일하다.



$$d_{ij}^{(k)} = d_{ij}^{(k-1)}$$

#### Case2 : 만약 정점 $k$ 가 경로 $p$ 의 중간 정점이라면, $p$ 를 $p_1(i \rightarrow k)$ 와 $p_2(k \rightarrow j)$ 로 분해된다.

Lemma 1에 따라서,  $p_1$ 과  $p_2$ 는 shortest path이다. (전체가 최단경로이면 부분도 최단경로이다)

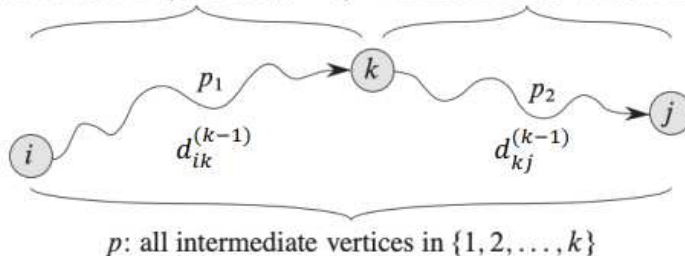
경로  $p_1$ 과  $p_2$ 의 **중간 정점**들은 모두  $\{1, 2, \dots, k-1\}$ 에 속한다.  $k$ 는 둘다 속할수 없기 때문에 제외

The weight of  $p_1 = d_{ik}^{(k-1)}$  and the weight of  $p_2 = d_{kj}^{(k-1)}$

출발 정점( $i$ )와 도착 정점( $j$ )는 중간 정점이 될 수 없으므로

따라서 중간 정점 허용집합 $\{1, 2, \dots, k\}$ 일때  $i \rightarrow j$  최단 경로는 다음과 같이 계산할 수 있다.

all intermediate vertices in  $\{1, 2, \dots, k-1\}$     all intermediate vertices in  $\{1, 2, \dots, k-1\}$



$$d_{ij}^{(k)} = d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$$



## APSP에 대한 재귀적 해결책

앞에서의 분석을 바탕으로, naïve method와는 다른 최단 경로 추정치의 표현식을 정의한다.

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = 0, \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) & \text{if } k \geq 1. \end{cases} \quad O(1)$$

(Case 1)  
Vertex  $k$  is not an intermediate vertex of  $p$

(Case 2)  
Vertex  $k$  is an intermediate vertex of  $p$

Naïve solution:  $l_{ij}^{(m)} = \min(l_{ij}^{(m-1)}, \min_{1 \leq k \leq n} \{l_{ik}^{(m-1)} + w_{kj}\})$   $O(V)$

### FLOYD-WARSHALL( $W$ )

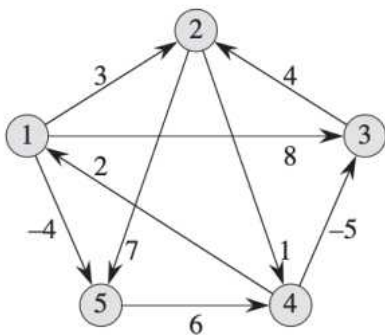
```

1   $n = W.rows$ 
2   $D^{(0)} = W$  Triply nested for loops  $\rightarrow O(V^3)$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) \rightarrow O(1)$ 
8  return  $D^{(n)}$ 
```

### Floyd-Warshall 알고리즘의 Process

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = 0, \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) & \text{if } k \geq 1. \end{cases} \quad \pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)} & \text{if } d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)}, \\ \pi_{kj}^{(k-1)} & \text{if } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)}. \end{cases}$$

Updated only when a shortest-path estimate is reduced



$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$\Pi^{(0)} = \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & \text{NIL} & 4 & \text{NIL} & \text{NIL} \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

$$D^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$\Pi^{(1)} = \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix}$$

Shortest-path weight matrix

Predecessor matrix



## Transitive Closure 전이 폐쇄

directed graph  $G=(V,E)$ , weight function  $w : E \rightarrow \mathbb{R}$ 가 있을 때,

**G의 transitive closure를  $G^*=(V,E^*)$ 라 한다.** 여기서 반드시 직접 연결된(i,j)를 간선이 있다!

모든 쌍의 vertices 사이에 path가 존재하게 된다.

주어진 G의 Transitive Closure를 어떻게 계산(또는 확인)할 수 있을까?

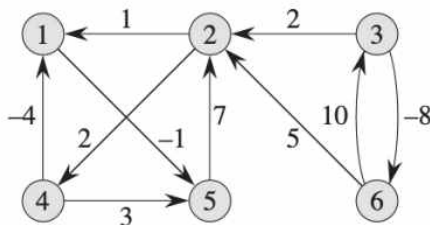
각 정점 i에서 다른 모든 정점 j로 갈 수 있는지 여부를 판별해서 그래프로 만드는 작업

Floyd-Marshall 알고리즘 사용...

-> 결과에 무한이 하나라도 있으면 도달가능하지 않음.

## Exercise 2: Floyd-Warshall Algorithm

- Run the Floyd-Warshall algorithm on the following weighted and directed graph, showing the matrix  $D^{(k)}$  that results for each iteration. (showing 6 matrices)



$$D^{(0)} = \begin{pmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & \infty & \infty \\ \infty & 2 & 0 & \infty & \infty & -8 \\ -4 & \infty & \infty & 0 & 3 & \infty \\ \infty & 7 & \infty & \infty & 0 & \infty \\ \infty & 5 & 10 & \infty & \infty & 0 \end{pmatrix}$$

$$D^{(0)} = \begin{pmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & \infty & \infty \\ \infty & 2 & 0 & \infty & \infty & -8 \\ -4 & \infty & \infty & 0 & 3 & \infty \\ \infty & 7 & \infty & \infty & 0 & \infty \\ \infty & 5 & 10 & \infty & \infty & 0 \end{pmatrix}$$

$$D^{(1)} = \begin{pmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & 0 & \infty \\ \infty & 2 & 0 & \infty & \infty & -8 \\ -4 & \infty & \infty & 0 & -5 & \infty \\ \infty & 7 & \infty & \infty & 0 & \infty \\ \infty & 5 & 10 & \infty & \infty & 0 \end{pmatrix}$$

$$D^{(2)} = \begin{pmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & 0 & \infty \\ 3 & 2 & 0 & 4 & 2 & -8 \\ -4 & \infty & \infty & 0 & -5 & \infty \\ 8 & 7 & \infty & 9 & 0 & \infty \\ 6 & 5 & 10 & 7 & 5 & 0 \end{pmatrix}$$

$$D^{(1)} = \begin{pmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & 0 & \infty \\ \infty & 2 & 0 & \infty & \infty & -8 \\ -4 & \infty & \infty & 0 & -5 & \infty \\ \infty & 7 & \infty & \infty & 0 & \infty \\ \infty & 5 & 10 & \infty & \infty & 0 \end{pmatrix}$$

$$D^{(2)} = \begin{pmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & 0 & \infty \\ 3 & 2 & 0 & 4 & 2 & -8 \\ -4 & \infty & \infty & 0 & -5 & \infty \\ 8 & 7 & \infty & 9 & 0 & \infty \\ 6 & 5 & 10 & 7 & 5 & 0 \end{pmatrix}$$

$$D^{(3)} = \begin{pmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & 0 & \infty \\ 3 & 2 & 0 & 4 & 2 & -8 \\ -4 & \infty & \infty & 0 & -5 & \infty \\ 8 & 7 & \infty & 9 & 0 & \infty \\ 6 & 5 & 10 & 7 & 5 & 0 \end{pmatrix}$$

$$D^{(3)} = \begin{pmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ 1 & 0 & \infty & 2 & 0 & \infty \\ 3 & 2 & 0 & 4 & 2 & -8 \\ -4 & \infty & \infty & 0 & -5 & \infty \\ 8 & 7 & \infty & 9 & 0 & \infty \\ 6 & 5 & 10 & 7 & 5 & 0 \end{pmatrix}$$

$$D^{(4)} = \begin{pmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ -2 & 0 & \infty & 2 & -3 & \infty \\ 0 & 2 & 0 & 4 & -1 & -8 \\ -4 & \infty & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 2 & 0 \end{pmatrix}$$

$$D^{(5)} = \begin{pmatrix} 0 & 6 & \infty & 8 & -1 & \infty \\ -2 & 0 & \infty & 2 & -3 & \infty \\ 0 & 2 & 0 & 4 & -1 & -8 \\ -4 & 2 & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 2 & 0 \end{pmatrix}$$

$$D^{(4)} = \begin{pmatrix} 0 & \infty & \infty & \infty & -1 & \infty \\ -2 & 0 & \infty & 2 & -3 & \infty \\ 0 & 2 & 0 & 4 & -1 & -8 \\ -4 & \infty & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 2 & 0 \end{pmatrix}$$

$$D^{(5)} = \begin{pmatrix} 0 & 6 & \infty & 8 & -1 & \infty \\ -2 & 0 & \infty & 2 & -3 & \infty \\ 0 & 2 & 0 & 4 & -1 & -8 \\ -4 & 2 & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 2 & 0 \end{pmatrix}$$

$$D^{(6)} = \begin{pmatrix} 0 & 6 & \infty & 8 & -1 & \infty \\ -2 & 0 & \infty & 2 & -3 & \infty \\ -5 & -3 & 0 & -1 & -6 & -8 \\ -4 & 2 & \infty & 0 & -5 & \infty \\ 5 & 7 & \infty & 9 & 0 & \infty \\ 3 & 5 & 10 & 7 & 2 & 0 \end{pmatrix}$$